UNIVERSIDAD TECNOLÓGICA DEL PERÚ

FACULTAD DE INGENIERÍA

CURSO INTEGRADOR I: SISTEMAS - SOFTWARE

SISTEMA DE SEGUIMIENTO DE ACTIVIDADES DE ATENCIÓN – VETERINARIA "PETCARE"

INTEGRANTES:

	2	2	2	2	2	2	2	14	20
	SEMANA 9 SESIÓN 1	SEMANA 9 SESIÓN 2	SEMANA 10 SESIÓN 1	SEMANA 10 SESIÓN 2	SEMANA 11 SESIÓN 1	SEMANA 11 SESIÓN 2	SEMANA 12 SESIÓN 1	PORCENTAJE DE PARTICIPACIÓN	NOTA
Juliams Schneider Lazaro Ruiz	0	2	2	2	0	2	2	71%	14.3
Camila Noemi Narvaez Francia	2	2	2	2	2	2	2	100%	20.0
Jhon Kevin Noriega Pardo	2	2	0	2	0	2	2	71%	14.3
Sofia Alarcón Bardales	2	2	2	2	2	2	2	100%	20.0

CICLO ACADÉMICO

2026-II

SECCIÓN

39134

DOCENTE:

Leonardo Ramos, Milagros Elisa

Lima, Perú

28 de octubre del 2026

ÍNDICE

1. Diseño de la solución 6

1.1. Diagrama de arquitectura 6

1.2. Tabla de principios aplicados por requerimiento (MVC, TDD, DAO, SOLID) 6

RF01 — Iniciar sesión con correo electrónico y contraseña 6

RF02 — Identificar el rol del usuario y mostrar sus funciones 6

RF03 — Recuperar la contraseña mediante el correo electrónico registrado 7

RF04 — Mantener activa la sesión mientras las credenciales sean válidas 7

RF05 — Consultar y actualizar la información del perfil del usuario 8

RF06 — Cambiar la contraseña del usuario 8

RF07 — Cerrar sesión de forma segura 9

RF08 — Visualizar el panel principal con el resumen de atenciones y actividades 9

RF09 — Visualizar en el panel principal las atenciones activas 10

RF10 — Visualizar la cantidad de actividades pendientes, completadas y no realizadas 10

RF11 — Mostrar avisos visuales cuando una actividad pendiente está próxima a vencer 11

RF12 — Registrar los datos de un propietario 11

RF13 — Consultar la información de un propietario 12

RF14 — Consultar las mascotas asociadas a un propietario 12

RF15 — Editar los datos de un propietario 13

RF16 — Buscar una mascota mediante la barra de búsqueda 13

RF17 — Registrar una mascota y asociarla a un propietario 14

RF18 — Modificar la información registrada de una mascota 14

RF19 — Consultar la información de una mascota 14

RF20 — Consultar el historial de atención de una mascota 15

RF21 — Filtrar las atenciones según su estado activo o finalizado 15

RF22 — Registrar una nueva atención veterinaria 16

RF23 — Registrar automáticamente la fecha y hora de inicio de una atención 16

RF24 — Consultar el detalle completo de una atención veterinaria 17

RF25 — Identificar si una atención está activa o finalizada 17

RF26 — Mostrar aviso de atención lista para finalizar 18

RF27 — Finalizar una atención veterinaria mediante confirmación 18

RF28 — Registrar automáticamente la fecha y hora de finalización 18

RF29 — Registrar una nueva actividad dentro de una atención 19

RF30 — Seleccionar el tipo de actividad y mostrar sus campos 19
RF31 — Consultar las actividades registradas en una atención 20
RF32 — Modificar la información registrada de una actividad 20
RF33 — Eliminar una actividad registrada por error 21
RF34 — Asignar automáticamente el estado "Pendiente" a una nueva actividad 21
RF35 — Cambiar el estado de una actividad de "Pendiente" a "Completada" 21
RF36 — Marcar como "No realizada" una actividad pendiente con su motivo 22
RF37 — Reactivar una actividad "No realizada" devolviéndola a "Pendiente" 22
RF38 — Calcular el porcentaje de avance de una atención 23
RF39 — Mostrar una barra de progreso del avance de una atención 23
RF40 — Consultar las actividades de una atención clasificadas según su estado 24
RF41 — Visualizar la línea de tiempo con los eventos de una atención veterinaria 24
RF42 — Registrar automáticamente los cambios realizados en la información 25
RF43 — Actualizar en tiempo real la información del seguimiento de una atención 25
RF44 — Crear y editar las cuentas de los usuarios 26
RF45 — Activar o desactivar las cuentas de los usuarios 26
RF46 — Gestionar el estado de las mascotas registradas 26
RF47 — Activar o desactivar el registro de un propietario 27
RF48 — Consultar un resumen general de la información de la veterinaria 27
RF49 — Consultar indicadores históricos de las atenciones veterinarias 28
RF50 — Consultar los cambios realizados en la información registrada 28
RF51 — Programar y cancelar citas de atención veterinaria 29
RF52 — Gestionar el catálogo de medicamentos 29
RF53 — Registrar la posología de los medicamentos en una actividad 30
RF54 — Registrar la dieta indicada a una mascota dentro de una atención 30
RF55 — Registrar los hallazgos de una ecografía por órgano evaluado 30
RF56 — Gestionar el catálogo de tipos de actividad 31
1.3. Diagramas de secuencia 31
Escenario de Éxito · Diagrama de secuencias 56
Escenario de Éxito · Diagrama de secuencias 59
Escenario de Éxito · Diagrama de secuencias 63
Escenario de Éxito · Diagrama de secuencias 67

Escenario de Éxito · Diagrama de secuencias.....71
Escenario de Éxito · Diagrama de secuencias.....75
Escenario de Éxito · Diagrama de secuencias.....79
Escenario de Éxito · Diagrama de secuencias.....82
Escenario de Éxito · Diagrama de secuencias.....85
Escenario de Éxito · Diagrama de secuencias.....90
Escenario de Éxito · Diagrama de secuencias.....93
Escenario de Éxito · Diagrama de secuencias.....97
Escenario de Éxito · Diagrama de secuencias.....102
Escenario de Éxito · Diagrama de secuencias.....105
Escenario de Éxito · Diagrama de secuencias.....109
Escenario de Éxito · Diagrama de secuencias.....112
Escenario de Éxito · Diagrama de secuencias.....116
Escenario de Éxito · Diagrama de secuencias.....119
Escenario de Éxito · Diagrama de secuencias.....123
Escenario de Éxito · Diagrama de secuencias.....127
Escenario de Éxito · Diagrama de secuencias.....131
Escenario de Éxito · Diagrama de secuencias.....135
Escenario de Éxito · Diagrama de secuencias.....138
Escenario de Éxito · Diagrama de secuencias.....142
Escenario de Éxito · Diagrama de secuencias.....147
Escenario de Éxito · Diagrama de secuencias.....150
Escenario de Éxito · Diagrama de secuencias.....155
Escenario de Éxito · Diagrama de secuencias.....159
Escenario de Éxito · Diagrama de secuencias.....163
Escenario de Éxito · Diagrama de secuencias.....167
Escenario de Éxito · Diagrama de secuencias.....171
Escenario de Éxito · Diagrama de secuencias.....175
Escenario de Éxito · Diagrama de secuencias.....179
Escenario de Éxito · Diagrama de secuencias.....183
Escenario de Éxito · Diagrama de secuencias.....186
Escenario de Éxito · Diagrama de secuencias.....190
Escenario de Éxito · Diagrama de secuencias.....193
Escenario de Éxito · Diagrama de secuencias.....196
Escenario de Éxito · Diagrama de secuencias.....200
Escenario de Éxito · Diagrama de secuencias.....204

Escenario de Éxito · Diagrama de secuencias 208
Escenario de Éxito · Diagrama de secuencias 212
Escenario de Éxito · Diagrama de secuencias 216
Escenario de Éxito · Diagrama de secuencias 219
Escenario de Éxito · Diagrama de secuencias 222
Escenario de Éxito · Diagrama de secuencias 228
Escenario de Éxito · Diagrama de secuencias 233
Escenario de Éxito · Diagrama de secuencias 237
Escenario de Éxito · Diagrama de secuencias 241
Escenario de Éxito · Diagrama de secuencias 246
2. Uso de recursos del lenguaje elegido 251
2.1. Lista de librerías utilizadas 251
2.2. Condiciones de seguridad 251
2.3. Evidencias en código 251
3. Control de versiones (GitHub) 252
3.1. Enlace al repositorio 252
3.2. Captura del historial 252
3.3. Captura de participantes 252
4. Implementación de interfaces gráficas 253
4.1. Tabla de cobertura 253
4.2. Capturas por requerimiento 263
4.3. Modelo lógico final de BD 263
4.3.1. Diagrama del modelo lógico 264
4.3.2. Notación relacional 264
4.3.3. Relaciones y cardinalidad 266
4.3.4. Reglas de integridad implementadas 268
5. Construcción del producto 269

1. Diseño de la solución

1.1. Diagrama de arquitectura

1. Descripción general

PetCare es una aplicación móvil para la gestión de una veterinaria, desarrollada con React, Expo y JavaScript. Su código está organizado en capas siguiendo el patrón MVC (Modelo, Vista, Controlador), complementado con el patrón DAO para aislar el acceso a los datos. Este documento muestra qué carpetas y clases pertenecen a cada capa y cómo se comunican entre sí.

2. Diagrama de capas

Figura 1. Arquitectura por capas de PetCare (MVC + DAO).

3. Distribución del código por capa

Capa	Carpeta (src/)	Responsabilidad / archivos principales
Vista	app	Pantallas y rutas de la aplicación (Expo). [Completar con los archivos de la carpeta]
Vista	atenciones	Pantallas del módulo de atenciones. [Completar]
Vista	components, hooks, styles	Componentes reutilizables de interfaz, hooks de UI y estilos. [Completar]
Controlador	controllers	Lógica de cada requerimiento: valida los datos de la Vista, coordina el flujo y llama a los DAO. [Completar]
Controlador	services	Servicios de apoyo usados por los controladores. [Completar]
Modelo	models	Entidades del dominio (por ejemplo Usuario, Mascota, Atención). [Completar]
Modelo	dao	Objetos de acceso a datos: único punto que consulta y modifica la base de datos. [Completar]
Transversal	utils, constants	Logger, validadores y constantes compartidas por las capas. [Completar]
Pruebas	__tests__, tests	Pruebas automatizadas (TDD) de controladores, DAO y utilidades. [Completar]

4. Reglas de comunicación entre capas

- La Vista solo se comunica con el Controlador; nunca accede directamente a la base de datos.
- El Controlador contiene la lógica del requerimiento y usa las entidades del Modelo y los DAO.

- Solo los DAO consultan o modifican la base de datos; el resto del código no conoce cómo se almacenan los datos.
- Las utilidades (logger, validadores) y las constantes pueden usarse desde cualquier capa, pero no dependen de ellas.

5. Relación con los diagramas de secuencia

Los diagramas de secuencia del proyecto usan tres tipos de objetos que corresponden directamente a las capas:

Prefijo	Capa	Ejemplo en los diagramas
I_ (Interfaz)	Vista	I_Inicio: pantalla donde el usuario ingresa y ve los avisos.
C_ (Control)	Controlador	C_Login: valida los datos y decide el resultado de la operación.
E_ (Entidad)	Modelo (entidades y DAO)	E_Usuario: consulta y devuelve los datos de usuarios de la base de datos.

Ejemplo, flujo «Iniciar sesión»: el usuario interactúa con I_Inicio (Vista), que envía la solicitud a C_Login (Controlador); este consulta a E_Usuario (Modelo/DAO), evalúa el resultado y ordena a la interfaz mostrar el panel o el aviso correspondiente.

6. Principios aplicados

- **MVC:** separa la interfaz, la lógica y los datos, de modo que cada parte puede cambiar sin afectar a las demás.
- **DAO:** concentra el acceso a datos en una sola capa, lo que facilita cambiar de fuente de datos y probar los controladores con datos simulados.
- **SOLID:** cada clase tiene una responsabilidad única (por ejemplo, el DAO solo accede a datos y el controlador solo coordina la lógica), y las capas dependen de abstracciones simples en lugar de detalles de otras capas.
- **TDD:** las carpetas de pruebas permiten verificar controladores, DAO y utilidades de forma independiente de la interfaz.

1.2. Tabla de principios aplicados por requerimiento (MVC, TDD, DAO, SOLID)

RF01 — Iniciar sesión con correo electrónico y contraseña

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	iniciarSesionControlador.js	Vista: app/index.jsx y FormularioInicioSesion.jsx, que solo muestra datos y captura las acciones del usuario. iniciarSesionControlador.js actúa como controlador: recibe la acción y la delega en autenticacionServicio.js, sin que la vista contenga lógica de negocio.

Principio aplicado	Clase o componente	Cómo se aplicó
DAO	perfilDAO.js	Centraliza el acceso a Supabase para iniciar sesión en Supabase Auth y consultar el estado de la cuenta. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	autenticacionServicio.js	Módulo con una única responsabilidad: validar las credenciales y controlar los intentos fallidos. No mezcla la lógica de otros módulos.
TDD	RF01.iniciarSesion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: éxito, campos vacíos, correo inválido, credenciales incorrectas, error de conexión y bloqueo por intentos. Se evidenció primero en rojo y luego en verde.

RF02 — Identificar el rol del usuario y mostrar sus funciones

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	rolUsuarioControlador.js	Vista: app/_layout.jsx, que solo muestra datos y captura las acciones del usuario. rolUsuarioControlador.js actúa como controlador: recibe la acción y la delega en autenticacionServicio.js, sin que la vista contenga lógica de negocio.
DAO	perfilDAO.js	Centraliza el acceso a Supabase para leer el rol y el estado de la tabla perfiles. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	autenticacionServicio.js	Módulo con una única responsabilidad: determinar las funciones visibles según el rol. No mezcla la lógica de otros módulos.
TDD	RF02.rolUsuario.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: Administrador, Asistente, rol no válido, cuenta inactiva y error al cargar funciones. Se evidenció primero en rojo y luego en verde.

RF03 — Recuperar la contraseña mediante el correo electrónico registrado

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	recuperarContrasenaControlador.js	Vista: formularioRecuperarContrasena.jsx, que solo muestra datos y captura las acciones del usuario. recuperarContrasenaControlador.js actúa como controlador: recibe la acción y la delega en recuperacionContrasenaServicio.js, sin que la vista contenga lógica de negocio.

Principio aplicado	Clase o componente	Cómo se aplicó
DAO	perfilDAO.js	Centraliza el acceso a Supabase para solicitar el enlace de recuperación y establecer la nueva contraseña. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	recuperacionContrasenaServicio.js	Módulo con una única responsabilidad: enviar el enlace de recuperación y limitar las solicitudes. No mezcla la lógica de otros módulos.
TDD	RF03.recuperarContrasena.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: envío exitoso, correo no registrado, límite excedido, enlace expirado, formato inválido y error de envío. Se evidenció primero en rojo y luego en verde.

RF04 — Mantener activa la sesión mientras las credenciales sean válidas

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	sesionControlador.js	Vista: app/_layout.jsx, que solo muestra datos y captura las acciones del usuario. sesionControlador.js actúa como controlador: recibe la acción y la delega en sesionPersistenciaServicio.js, sin que la vista contenga lógica de negocio.
DAO	perfilDAO.js	Centraliza el acceso a Supabase para obtener la sesión guardada y validar su token. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	sesionPersistenciaServicio.js	Módulo con una única responsabilidad: verificar que la sesión guardada siga vigente. No mezcla la lógica de otros módulos.
TDD	RF04.mantenerSesion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: sesión vigente, no encontrada, expirada e inválida. Se evidenció primero en rojo y luego en verde.

RF05 — Consultar y actualizar la información del perfil del usuario

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	perfilControlador.js	Vista: PantallaEditarPerfil.jsx, que solo muestra datos y captura las acciones del usuario. perfilControlador.js actúa como controlador: recibe la acción y la delega en perfilServicio.js, sin que la vista contenga lógica de negocio.

Principio aplicado	Clase o componente	Cómo se aplicó
DAO	perfilDAO.js	Centraliza el acceso a Supabase para consultar y actualizar el registro del usuario en perfiles. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	perfilServicio.js	Módulo con una única responsabilidad: validar y actualizar los datos del perfil. No mezcla la lógica de otros módulos.
TDD	RF05.actualizarPerfil.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: actualización exitosa, campos incompletos, correo o teléfono inválido, correo repetido, confirmación de correo y error al guardar. Se evidenció primero en rojo y luego en verde.

RF06 — Cambiar la contraseña del usuario

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	perfilControlador.js	Vista: ModalCambiarContrasena.jsx, que solo muestra datos y captura las acciones del usuario. perfilControlador.js actúa como controlador: recibe la acción y la delega en autenticacionServicio.js, sin que la vista contenga lógica de negocio.
DAO	perfilDAO.js	Centraliza el acceso a Supabase para verificar la contraseña actual y actualizar la nueva. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	autenticacionServicio.js	Módulo con una única responsabilidad: validar las reglas de la nueva contraseña. No mezcla la lógica de otros módulos.
TDD	RF06autenticacion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: cambio exitoso, campos incompletos, contraseña actual incorrecta, contraseñas distintas, contraseña inválida e igual a la actual. Se evidenció primero en rojo y luego en verde.

RF07 — Cerrar sesión de forma segura

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	cerrarSesionControlador.js	Vista: MenuLateralDerecho.jsx, que solo muestra datos y captura las acciones del usuario. cerrarSesionControlador.js actúa como controlador: recibe la acción y la delega en cerrarSesionServicio.js, sin que la vista contenga lógica de negocio.
DAO	perfilDAO.js	Centraliza el acceso a Supabase para cerrar la sesión activa en Supabase Auth. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	cerrarSesionServicio.js	Módulo con una única responsabilidad: finalizar la sesión y restringir el acceso protegido. No mezcla la lógica de otros módulos.
TDD	RF07.cerrarSesion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: cierre exitoso, acción cancelada, sesión expirada y error durante el cierre. Se evidenció primero en rojo y luego en verde.

RF08 — Visualizar el panel principal con el resumen de atenciones y actividades

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	inicioControlador.js	Vista: app/inicio.jsx, que solo muestra datos y captura las acciones del usuario. inicioControlador.js actúa como controlador: recibe la acción y la delega en panelInicioServicio.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js y actividadesDAO.js	Centraliza el acceso a Supabase para contar atenciones y actividades registradas. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	panelInicioServicio.js	Módulo con una única responsabilidad: consolidar el resumen que muestra el panel principal. No mezcla la lógica de otros módulos.
TDD	RF08.visualizarPanel.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: panel cargado, error de atenciones, error de actividades y error de conexión. Se evidenció primero en rojo y luego en verde.

RF09 — Visualizar en el panel principal las atenciones activas

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	inicioControlador.js	Vista: tarjetaIndicador.jsx, que solo muestra datos y captura las acciones del usuario. inicioControlador.js actúa como controlador: recibe la acción y la delega en panelInicioServicio.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para contar las atenciones con estado Activa. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	panelInicioServicio.js	Módulo con una única responsabilidad: contar las atenciones en estado Activa. No mezcla la lógica de otros módulos.
TDD	RF09.visualizarAtencionesActivas.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: contador con valor, contador en 0, error de consulta y error al cargar el listado. Se evidenció primero en rojo y luego en verde.

RF10 — Visualizar la cantidad de actividades pendientes, completadas y no realizadas

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	inicioControlador.js	Vista: tarjetaIndicador.jsx, que solo muestra datos y captura las acciones del usuario. inicioControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para contar las actividades por estado. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: agrupar las actividades por estado. No mezcla la lógica de otros módulos.
TDD	RF10.visualizarActividadesPorEstado.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: resumen por estado, sin actividades, estado sin resultados y error de carga. Se evidenció primero en rojo y luego en verde.

RF11 — Mostrar avisos visuales cuando una actividad pendiente está próxima a vencer

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	inicioControlador.js	Vista: avisoActividad.jsx, que solo muestra datos y captura las acciones del usuario. inicioControlador.js actúa como controlador: recibe la acción y la delega en panelInicioServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para consultar las actividades pendientes con su fecha prevista. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	panelInicioServicio.js	Módulo con una única responsabilidad: evaluar si una actividad pendiente vence en las próximas 24 horas. No mezcla la lógica de otros módulos.
TDD	RF11.avisoVencimiento.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: aviso por vencer, error al mostrar, fecha inválida y error al verificar. Se evidenció primero en rojo y luego en verde.

RF12 — Registrar los datos de un propietario

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	propietariosControlador.js	Vista: RegistrarPropietario.jsx, que solo muestra datos y captura las acciones del usuario. propietariosControlador.js actúa como controlador: recibe la acción y la delega en validacionesPropietario.js, sin que la vista contenga lógica de negocio.
DAO	propietarioDAO.js	Centraliza el acceso a Supabase para insertar propietarios y verificar DNI y correo existentes. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	validacionesPropietario.js	Módulo con una única responsabilidad: validar y registrar al propietario evitando duplicados. No mezcla la lógica de otros módulos.
TDD	RF12.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: registro exitoso, campos incompletos, DNI inválido o repetido, y correo inválido o repetido. Se evidenció primero en rojo y luego en verde.

RF13 — Consultar la información de un propietario

Principio aplicado	Clase o componente	Cómo se aplicó

MVC	propietariosControlador.js	Vista: fichaPropietario.jsx, que solo muestra datos y captura las acciones del usuario. propietariosControlador.js actúa como controlador: recibe la acción y la delega en propietarioServicio.js, sin que la vista contenga lógica de negocio.
DAO	propietarioDAO.js	Centraliza el acceso a Supabase para consultar un propietario por identificador. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	propietarioServicio.js	Módulo con una única responsabilidad: obtener los datos del propietario con sus mascotas. No mezcla la lógica de otros módulos.
TDD	RF13.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: consulta exitosa, sin propietarios, propietario no encontrado y error de consulta. Se evidenció primero en rojo y luego en verde.

RF14 — Consultar las mascotas asociadas a un propietario

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	propietariosControlador.js	Vista: fichaMascotaPropietario.jsx, que solo muestra datos y captura las acciones del usuario. propietariosControlador.js actúa como controlador: recibe la acción y la delega en propietarioServicio.js, sin que la vista contenga lógica de negocio.
DAO	mascotaDAO.js	Centraliza el acceso a Supabase para consultar mascotas filtradas por propietario. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	propietarioServicio.js	Módulo con una única responsabilidad: obtener las mascotas asociadas a un propietario. No mezcla la lógica de otros módulos.
TDD	RF14.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: mascotas mostradas, propietario sin mascotas, propietario no encontrado y error de consulta. Se evidenció primero en rojo y luego en verde.

RF15 — Editar los datos de un propietario

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	propietariosControlador.js	Vista: editarPropietario.jsx, que solo muestra datos y captura las acciones del usuario. propietariosControlador.js actúa como controlador: recibe la acción y la delega en validacionesPropietario.js, sin que la vista contenga lógica de negocio.

DAO	propietarioDAO.js	Centraliza el acceso a Supabase para actualizar el registro del propietario. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	validacionesPropietario.js	Módulo con una única responsabilidad: validar y actualizar los datos del propietario. No mezcla la lógica de otros módulos.
TDD	RF15.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: edición exitosa, campos incompletos, formato inválido, DNI repetido y error al actualizar. Se evidenció primero en rojo y luego en verde.

RF16 — Buscar una mascota mediante la barra de búsqueda

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	mascotasControlador.js	Vista: ListaMascotas.jsx, que solo muestra datos y captura las acciones del usuario. mascotasControlador.js actúa como controlador: recibe la acción y la delega en validacionesBusquedaMascotas.js, sin que la vista contenga lógica de negocio.
DAO	mascotaDAO.js	Centraliza el acceso a Supabase para buscar mascotas por nombre u otro dato de identificación. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	validacionesBusquedaMascotas.js	Módulo con una única responsabilidad: validar el dato de búsqueda y obtener las coincidencias. No mezcla la lógica de otros módulos.
TDD	RF16.validacionesBusquedaMascotas.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: búsqueda con coincidencias, formato inválido, sin coincidencias y campo vacío. Se evidenció primero en rojo y luego en verde.

RF17 — Registrar una mascota y asociarla a un propietario

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	mascotasControlador.js	Vista: nuevaMascota.jsx, que solo muestra datos y captura las acciones del usuario. mascotasControlador.js actúa como controlador: recibe la acción y la delega en validacionesRegistroMascota.js, sin que la vista contenga lógica de negocio.
DAO	mascotaDAO.js	Centraliza el acceso a Supabase para insertar la mascota con la clave del propietario. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.

SOLID — SRP	validacionesRegistroMascota.js	Módulo con una única responsabilidad: validar y registrar la mascota asociada a su propietario. No mezcla la lógica de otros módulos.
TDD	RF17.registrarMascota.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: registro exitoso, campos incompletos, propietario no seleccionado, datos inválidos y error al registrar. Se evidenció primero en rojo y luego en verde.

RF18 — Modificar la información registrada de una mascota

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	mascotasControlador.js	Vista: editarMascota.jsx, que solo muestra datos y captura las acciones del usuario. mascotasControlador.js actúa como controlador: recibe la acción y la delega en validacionesEdicionMascota.js, sin que la vista contenga lógica de negocio.
DAO	mascotaDAO.js	Centraliza el acceso a Supabase para actualizar el registro de la mascota. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	validacionesEdicionMascota.js	Módulo con una única responsabilidad: validar y actualizar los datos de la mascota. No mezcla la lógica de otros módulos.
TDD	RF18.editarMascota.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: modificación exitosa, campos incompletos, datos inválidos, cancelación y error al actualizar. Se evidenció primero en rojo y luego en verde.

RF19 — Consultar la información de una mascota

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	mascotasControlador.js	Vista: fichaMascota.jsx, que solo muestra datos y captura las acciones del usuario. mascotasControlador.js actúa como controlador: recibe la acción y la delega en validacionesConsultaMascota.js, sin que la vista contenga lógica de negocio.
DAO	mascotaDAO.js	Centraliza el acceso a Supabase para consultar una mascota con su propietario. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	validacionesConsultaMascota.js	Módulo con una única responsabilidad: obtener los datos básicos de la mascota y su propietario. No mezcla la lógica de otros módulos.

TDD	RF19.consultarMascota.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: consulta exitosa, sin coincidencias, sin atenciones, información incompleta y error de consulta. Se evidenció primero en rojo y luego en verde.

RF20 — Consultar el historial de atención de una mascota

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: historiaMascota.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en seguimientoServicio.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para consultar las atenciones de una mascota. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	seguimientoServicio.js	Módulo con una única responsabilidad: ordenar cronológicamente las atenciones de una mascota. No mezcla la lógica de otros módulos.
TDD	RF20_historialAtencion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: historial ordenado, mascota sin historial, mascota no encontrada y error de consulta. Se evidenció primero en rojo y luego en verde.

RF21 — Filtrar las atenciones según su estado activo o finalizado

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: app/atenciones.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en atencionesFiltroServicio.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para consultar atenciones filtradas por estado. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	atencionesFiltroServicio.js	Módulo con una única responsabilidad: aplicar el filtro de estado sobre las atenciones. No mezcla la lógica de otros módulos.
TDD	RF21.filtrarAtenciones.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: filtro por estado, sin resultados, error de carga y opción Todas por defecto. Se evidenció primero en rojo y luego en verde.

RF22 — Registrar una nueva atención veterinaria

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: registrarAtencion.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en validacionesAtencion.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para insertar la atención y verificar la mascota. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	validacionesAtencion.js	Módulo con una única responsabilidad: validar la mascota y crear la atención en estado Activa. No mezcla la lógica de otros módulos.
TDD	RF22.registrarAtencion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: registro exitoso, mascota no seleccionada, mascota no disponible y error al registrar. Se evidenció primero en rojo y luego en verde.

RF23 — Registrar automáticamente la fecha y hora de inicio de una atención

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: formularioAtencion.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en fechaAtencion.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para guardar la fecha y hora de inicio de la atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	fechaAtencion.js	Módulo con una única responsabilidad: obtener y asignar la fecha y hora de inicio. No mezcla la lógica de otros módulos.
TDD	RF23.fechaInicio.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: fecha de inicio correcta, error al obtener la fecha, error al guardar y datos incompletos. Se evidenció primero en rojo y luego en verde.

RF24 — Consultar el detalle completo de una atención veterinaria

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: detalleAtencion.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en

		seguimientoServicio.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para consultar una atención con sus actividades. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	seguimientoServicio.js	Módulo con una única responsabilidad: reunir estado, fechas, actividades, progreso y observaciones. No mezcla la lógica de otros módulos.
TDD	RF24.detalleAtencion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: detalle completo, atención no encontrada, información incompleta y error de carga. Se evidenció primero en rojo y luego en verde.

RF25 — Identificar si una atención está activa o finalizada

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: tarjetaAtencion.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para leer el estado de la atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: determinar el estado actual de la atención. No mezcla la lógica de otros módulos.
TDD	RF25.estadoAtencion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: estado identificado, estado no definido, error de consulta y atención no encontrada. Se evidenció primero en rojo y luego en verde.

RF26 — Mostrar aviso de atención lista para finalizar

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: detalleAtencion.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para contar las actividades pendientes de la atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.

SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: verificar que no queden actividades pendientes. No mezcla la lógica de otros módulos.
TDD	RF26.verificarAtencion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: aviso mostrado, actividades pendientes, actividades no disponibles y atención sin actividades. Se evidenció primero en rojo y luego en verde.

RF27 — Finalizar una atención veterinaria mediante confirmación

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: detalleAtencion.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en atencionesServicio.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para actualizar el estado de la atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	atencionesServicio.js	Módulo con una única responsabilidad: validar y cambiar la atención a estado Finalizada. No mezcla la lógica de otros módulos.
TDD	RF27.finalizarAtencion.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: finalización exitosa, actividad pendiente, confirmación cancelada y atención ya finalizada. Se evidenció primero en rojo y luego en verde.

RF28 — Registrar automáticamente la fecha y hora de finalización

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	atencionesControlador.js	Vista: detalleAtencion.jsx, que solo muestra datos y captura las acciones del usuario. atencionesControlador.js actúa como controlador: recibe la acción y la delega en fechaAtencion.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para guardar la fecha y hora de finalización. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	fechaAtencion.js	Módulo con una única responsabilidad: obtener y asignar la fecha y hora de finalización. No mezcla la lógica de otros módulos.

TDD	RF28.fechaFin.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: fecha de fin correcta, atención no finalizable, error de fecha y error al actualizar. Se evidenció primero en rojo y luego en verde.

RF29 — Registrar una nueva actividad dentro de una atención

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: registrarActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para insertar la actividad asociada a la atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: validar y registrar la actividad dentro de la atención. No mezcla la lógica de otros módulos.
TDD	RF29.registrarActividad.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: registro exitoso, campos incompletos, datos inválidos y error durante el registro. Se evidenció primero en rojo y luego en verde.

RF30 — Seleccionar el tipo de actividad y mostrar sus campos

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: formularioActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	tiposActividadDAO.js	Centraliza el acceso a Supabase para consultar el catálogo de tipos de actividad. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: determinar los campos del formulario según el tipo. No mezcla la lógica de otros módulos.
TDD	RF30.tipoActividad.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: campos según tipo, tipo no seleccionado, cambio de tipo y error al cargar campos. Se evidenció primero en rojo y luego en verde.

RF31 — Consultar las actividades registradas en una atención

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: listaActividadesEstado.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para consultar actividades por atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: obtener las actividades de una atención. No mezcla la lógica de otros módulos.
TDD	RF31.consultarActividades.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: consulta exitosa, atención no encontrada, sin actividades y error de información. Se evidenció primero en rojo y luego en verde.

RF32 — Modificar la información registrada de una actividad

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: detalleActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para actualizar el registro de la actividad. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: validar y actualizar los datos de la actividad. No mezcla la lógica de otros módulos.
TDD	RF32.modificarActividad.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: modificación exitosa, campos incompletos, datos inválidos, error al actualizar y cancelación. Se evidenció primero en rojo y luego en verde.

RF33 — Eliminar una actividad registrada por error

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: detalleActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en

		actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para marcar la actividad como eliminada (eliminación lógica). Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: validar y eliminar lógicamente la actividad. No mezcla la lógica de otros módulos.
TDD	RF33.eliminarActividad.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: eliminación exitosa, eliminación cancelada, actividad no encontrada y error al eliminar. Se evidenció primero en rojo y luego en verde.

RF34 — Asignar automáticamente el estado “Pendiente” a una nueva actividad

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: formularioActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para guardar la actividad con estado Pendiente. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: asignar el estado inicial a toda actividad nueva. No mezcla la lógica de otros módulos.
TDD	RF34.estadoPendiente.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: estado Pendiente asignado, registro incompleto, error al asignar y atención no activa. Se evidenció primero en rojo y luego en verde.

RF35 — Cambiar el estado de una actividad de “Pendiente” a “Completada”

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: tarjetaActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para actualizar el estado de la actividad. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.

SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: validar la transición Pendiente a Completada. No mezcla la lógica de otros módulos.
TDD	RF35.completarActividad.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: actividad completada, actividad no pendiente, cancelación y error al actualizar. Se evidenció primero en rojo y luego en verde.

RF36 — Marcar como “No realizada” una actividad pendiente con su motivo

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: detalleActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para actualizar el estado y guardar el motivo. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: validar el motivo y la transición a No realizada. No mezcla la lógica de otros módulos.
TDD	RF36.noRealizada.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: actividad no realizada, motivo vacío, actividad no pendiente, cancelación y error al actualizar. Se evidenció primero en rojo y luego en verde.

RF37 — Reactivar una actividad “No realizada” devolviéndola a “Pendiente”

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: detalleActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en actividadesServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para actualizar el estado de la actividad. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	actividadesServicio.js	Módulo con una única responsabilidad: validar la transición No realizada a Pendiente. No mezcla la lógica de otros módulos.
TDD	RF37.reactivarActividad.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: reactivación exitosa, actividad con otro estado, cancelación y error

		al actualizar. Se evidenció primero en rojo y luego en verde.

RF38 — Calcular el porcentaje de avance de una atención

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	seguimientoControlador.js	Vista: barraProgreso.jsx, que solo muestra datos y captura las acciones del usuario. seguimientoControlador.js actúa como controlador: recibe la acción y la delega en seguimientoServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para contar actividades totales y completadas de la atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	seguimientoServicio.js	Módulo con una única responsabilidad: calcular el porcentaje de actividades completadas. No mezcla la lógica de otros módulos.
TDD	RF38.porcentajeAvance.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: avance del 50%, sin actividades, información incompleta y error de consulta. Se evidenció primero en rojo y luego en verde.

RF39 — Mostrar una barra de progreso del avance de una atención

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	seguimientoControlador.js	Vista: barraProgreso.jsx, que solo muestra datos y captura las acciones del usuario. seguimientoControlador.js actúa como controlador: recibe la acción y la delega en seguimientoServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para obtener el estado de las actividades de la atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	seguimientoServicio.js	Módulo con una única responsabilidad: entregar a la vista el porcentaje listo para dibujar. No mezcla la lógica de otros módulos.
TDD	RF39.barraProgreso.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: barra al 50%, sin actividades, 100% completadas y error al cargar. Se evidenció primero en rojo y luego en verde.

RF40 — Consultar las actividades de una atención clasificadas según su estado

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	seguimientoControlador.js	Vista: listaActividadesEstado.jsx, que solo muestra datos y captura las acciones del usuario. seguimientoControlador.js actúa como controlador: recibe la acción y la delega en seguimientoServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para consultar actividades por atención y estado. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	seguimientoServicio.js	Módulo con una única responsabilidad: clasificar las actividades por estado y aplicar el filtro. No mezcla la lógica de otros módulos.
TDD	RF40.actividadesPorEstado.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: clasificación correcta, categoría vacía, estado sin actividades y consulta de una actividad. Se evidenció primero en rojo y luego en verde.

RF41 — Visualizar la línea de tiempo con los eventos de una atención veterinaria

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	seguimientoControlador.js	Vista: lineaTiempo.jsx, que solo muestra datos y captura las acciones del usuario. seguimientoControlador.js actúa como controlador: recibe la acción y la delega en seguimientoServicio.js, sin que la vista contenga lógica de negocio.
DAO	registroCambiosDAO.js	Centraliza el acceso a Supabase para consultar los eventos registrados de una atención. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	seguimientoServicio.js	Módulo con una única responsabilidad: ordenar cronológicamente los eventos de la atención. No mezcla la lógica de otros módulos.
TDD	RF41.lineaTiempo.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: eventos ordenados, sin eventos, información incompleta y error de consulta. Se evidenció primero en rojo y luego en verde.

RF42 — Registrar automáticamente los cambios realizados en la información

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	auditoriaControlador.js	Vista: (sin pantalla propia), que solo muestra datos y captura las acciones del usuario. auditoriaControlador.js actúa como

		controlador: recibe la acción y la delega en auditoriaServicio.js, sin que la vista contenga lógica de negocio.
DAO	registroCambiosDAO.js	Centraliza el acceso a Supabase para insertar el registro de cambio. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	auditoriaServicio.js	Módulo con una única responsabilidad: registrar usuario, acción, información afectada y fecha de cada cambio. No mezcla la lógica de otros módulos.
TDD	RF42.registroCambios.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: cambio registrado, error de trazabilidad, usuario no identificado e información incompleta. Se evidenció primero en rojo y luego en verde.

RF43 — Actualizar en tiempo real la información del seguimiento de una atención

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	seguimientoControlador.js	Vista: app/seguimientos/seguimiento.jsx, que solo muestra datos y captura las acciones del usuario. seguimientoControlador.js actúa como controlador: recibe la acción y la delega en seguimientoServicio.js, sin que la vista contenga lógica de negocio.
DAO	atencionesDAO.js	Centraliza el acceso a Supabase para suscribirse a cambios de la atención y sus actividades (Supabase Realtime). Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	seguimientoServicio.js	Módulo con una única responsabilidad: suscribirse a los cambios y refrescar estado y avance. No mezcla la lógica de otros módulos.
TDD	RF43.tiempoReal.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: actualización automática, cambio no detectado, error de actualización y atención no disponible. Se evidenció primero en rojo y luego en verde.

RF44 — Crear y editar las cuentas de los usuarios

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	administracionControlador.js	Vista: nuevoUsuario.jsx, que solo muestra datos y captura las acciones del usuario. administracionControlador.js actúa como controlador: recibe la acción y la delega en usuariosService.js, sin que la vista contenga lógica de negocio.

DAO	perfilDAO.js	Centraliza el acceso a Supabase para crear y actualizar cuentas en Auth y perfiles. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	usuariosService.js	Módulo con una única responsabilidad: validar y guardar las cuentas con su rol. No mezcla la lógica de otros módulos.
TDD	RF44.gestionarUsuarios.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: éxito, campos incompletos, datos inválidos, correo registrado, contraseñas distintas y error al guardar. Se evidenció primero en rojo y luego en verde.

RF45 — Activar o desactivar las cuentas de los usuarios

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	administracionControlador.js	Vista: gestionUsuario.jsx, que solo muestra datos y captura las acciones del usuario. administracionControlador.js actúa como controlador: recibe la acción y la delega en administracionService.js, sin que la vista contenga lógica de negocio.
DAO	perfilDAO.js	Centraliza el acceso a Supabase para actualizar el estado de la cuenta. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	administracionService.js	Módulo con una única responsabilidad: cambiar el estado Activa o Desactivada de la cuenta. No mezcla la lógica de otros módulos.
TDD	RF45.estadoUsuario.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: cambio de estado, cancelación, cuenta inexistente y error al actualizar. Se evidenció primero en rojo y luego en verde.

RF46 — Gestionar el estado de las mascotas registradas

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	administracionControlador.js	Vista: gestionMascotas.jsx, que solo muestra datos y captura las acciones del usuario. administracionControlador.js actúa como controlador: recibe la acción y la delega en mascotaAdminServicio.js, sin que la vista contenga lógica de negocio.
DAO	mascotaDAO.js	Centraliza el acceso a Supabase para buscar mascotas y actualizar su estado. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.

SOLID — SRP	mascotaAdminServicio.js	Módulo con una única responsabilidad: activar o desactivar el registro de la mascota. No mezcla la lógica de otros módulos.
TDD	RF46.estadoMascota.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: cambio de estado, mascota no encontrada, búsqueda sin datos y error al actualizar. Se evidenció primero en rojo y luego en verde.

RF47 — Activar o desactivar el registro de un propietario

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	administracionControlador.js	Vista: gestionPropietarios.jsx, que solo muestra datos y captura las acciones del usuario. administracionControlador.js actúa como controlador: recibe la acción y la delega en propietarioAdminServicio.js, sin que la vista contenga lógica de negocio.
DAO	propietarioDAO.js	Centraliza el acceso a Supabase para buscar propietarios y actualizar su estado. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	propietarioAdminServicio.js	Módulo con una única responsabilidad: activar o desactivar el registro del propietario. No mezcla la lógica de otros módulos.
TDD	RF47.estadoPropietario.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: cambio de estado, propietario no encontrado, búsqueda sin datos y error al actualizar. Se evidenció primero en rojo y luego en verde.

RF48 — Consultar un resumen general de la información de la veterinaria

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	administracionControlador.js	Vista: administracion.jsx, que solo muestra datos y captura las acciones del usuario. administracionControlador.js actúa como controlador: recibe la acción y la delega en resumenGeneralServicio.js, sin que la vista contenga lógica de negocio.
DAO	indicadoresDAO.js	Centraliza el acceso a Supabase para contar registros por categoría y estado. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	resumenGeneralServicio.js	Módulo con una única responsabilidad: consolidar las cantidades de usuarios, mascotas, propietarios, atenciones y actividades. No mezcla la lógica de otros módulos.
TDD	RF48.resumenGeneral.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: resumen completo,

		categorías en cero, información incompleta y error de consulta. Se evidenció primero en rojo y luego en verde.

RF49 — Consultar indicadores históricos de las atenciones veterinarias

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	administracionControlador.js	Vista: indicadoresHistoricos.jsx, que solo muestra datos y captura las acciones del usuario. administracionControlador.js actúa como controlador: recibe la acción y la delega en indicadoresServicio.js, sin que la vista contenga lógica de negocio.
DAO	indicadoresDAO.js	Centraliza el acceso a Supabase para consultar atenciones por periodo y estado. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	indicadoresServicio.js	Módulo con una única responsabilidad: calcular los indicadores según periodo y estado. No mezcla la lógica de otros módulos.
TDD	RF49.indicadoresHistoricos.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: indicadores con filtros, periodo sin registros, filtros no válidos y error de carga. Se evidenció primero en rojo y luego en verde.

RF50 — Consultar los cambios realizados en la información registrada

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	auditoriaControlador.js	Vista: cambiosRealizados.jsx, que solo muestra datos y captura las acciones del usuario. auditoriaControlador.js actúa como controlador: recibe la acción y la delega en auditoriaServicio.js, sin que la vista contenga lógica de negocio.
DAO	registroCambiosDAO.js	Centraliza el acceso a Supabase para consultar cambios por usuario, tipo, acción y periodo. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	auditoriaServicio.js	Módulo con una única responsabilidad: filtrar y presentar el historial de cambios. No mezcla la lógica de otros módulos.
TDD	RF50.consultarCambios.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: consulta con filtros, sin resultados, filtros no válidos y error de carga. Se evidenció primero en rojo y luego en verde.

RF51 — Programar y cancelar citas de atención veterinaria

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	citasControlador.js	Vista: programarCita.jsx, que solo muestra datos y captura las acciones del usuario. citasControlador.js actúa como controlador: recibe la acción y la delega en citasServicio.js, sin que la vista contenga lógica de negocio.
DAO	citasDAO.js	Centraliza el acceso a Supabase para insertar citas y actualizar su estado a Programada o Cancelada. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	citasServicio.js	Módulo con una única responsabilidad: validar fecha, hora y duplicados, y cancelar citas. No mezcla la lógica de otros módulos.
TDD	RF51.programarCita.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: cita programada, campos incompletos, fecha pasada, cita duplicada, mascota inactiva, cita no cancelable y error al guardar. Se evidenció primero en rojo y luego en verde.

RF52 — Gestionar el catálogo de medicamentos

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	medicamentosControlador.js	Vista: gestionMedicamentos.jsx, que solo muestra datos y captura las acciones del usuario. medicamentosControlador.js actúa como controlador: recibe la acción y la delega en medicamentosServicio.js, sin que la vista contenga lógica de negocio.
DAO	medicamentosDAO.js	Centraliza el acceso a Supabase para insertar, actualizar y buscar medicamentos del catálogo. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	medicamentosServicio.js	Módulo con una única responsabilidad: validar y guardar medicamentos sin duplicados. No mezcla la lógica de otros módulos.
TDD	RF52.catalogoMedicamentos.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: medicamento guardado, campos incompletos, duplicado, sin resultados y error al guardar. Se evidenció primero en rojo y luego en verde.

RF53 — Registrar la posología de los medicamentos en una actividad

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: formularioActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como

		controlador: recibe la acción y la delega en posologiaServicio.js, sin que la vista contenga lógica de negocio.
DAO	actividadesDAO.js	Centraliza el acceso a Supabase para insertar los medicamentos y su posología asociados a la actividad. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	posologiaServicio.js	Módulo con una única responsabilidad: validar la posología de cada medicamento de la actividad. No mezcla la lógica de otros módulos.
TDD	RF53.posologia.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: posología registrada, medicamento no seleccionado, dosis vacía, duración inválida, medicamento repetido y error al registrar. Se evidenció primero en rojo y luego en verde.

RF54 — Registrar la dieta indicada a una mascota dentro de una atención

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: formularioActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en dietaServicio.js, sin que la vista contenga lógica de negocio.
DAO	dietasDAO.js	Centraliza el acceso a Supabase para consultar el catálogo de dietas y sus alimentos. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	dietaServicio.js	Módulo con una única responsabilidad: obtener la restricción y los alimentos permitidos del tipo de dieta. No mezcla la lógica de otros módulos.
TDD	RF54.dieta.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: dieta registrada, dieta no seleccionada, dieta sin alimentos, error de catálogo y error al registrar. Se evidenció primero en rojo y luego en verde.

RF55 — Registrar los hallazgos de una ecografía por órgano evaluado

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	actividadesControlador.js	Vista: formularioActividad.jsx, que solo muestra datos y captura las acciones del usuario. actividadesControlador.js actúa como controlador: recibe la acción y la delega en ecografiaServicio.js, sin que la vista contenga lógica de negocio.
DAO	ecografiaDAO.js	Centraliza el acceso a Supabase para insertar los hallazgos por órgano asociados a la

		actividad. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	ecografiaServicio.js	Módulo con una única responsabilidad: validar los hallazgos de cada órgano evaluado. No mezcla la lógica de otros módulos.
TDD	RF55.hallazgosEcografia.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: hallazgos registrados, sin órganos, hallazgos vacíos, órgano repetido, medición inválida y error al registrar. Se evidenció primero en rojo y luego en verde.

RF56 — Gestionar el catálogo de tipos de actividad

Principio aplicado	Clase o componente	Cómo se aplicó
MVC	administracionControlador.js	Vista: gestionTiposActividad.jsx, que solo muestra datos y captura las acciones del usuario. administracionControlador.js actúa como controlador: recibe la acción y la delega en tiposActividadServicio.js, sin que la vista contenga lógica de negocio.
DAO	tiposActividadDAO.js	Centraliza el acceso a Supabase para insertar, actualizar y cambiar la disponibilidad de tipos de actividad. Ninguna pantalla, controlador ni servicio ejecuta consultas directamente.
SOLID — SRP	tiposActividadServicio.js	Módulo con una única responsabilidad: validar, guardar y activar o desactivar tipos de actividad. No mezcla la lógica de otros módulos.
TDD	RF56.tiposActividad.test.js	Pruebas escritas antes de implementar el requerimiento. Cubren: tipo guardado, campos incompletos, nombre duplicado y error al guardar. Se evidenció primero en rojo y luego en verde.

1.3. Casos de prueba

CASOS DE PRUEBA — RF01: Iniciar sesión con correo electrónico y contraseña

#	Escenario	Precondición	Acción del actor	Resultado esperado	Resultado obtenido
CP01	Éxito — inicio de sesión válido	Pantalla “Inicio de sesión” visible; cuenta de prueba activa `asistente@veterinaria.com`; contraseña válida;	El usuario ingresa sus credenciales y	El sistema valida las credenciales, autentica al usuario y lo	Aprobado en la ejecución automatizada

		contador de intentos fallidos en 0; servicio de autenticación disponible; sin sesión activa.	selecciona “Iniciar sesión”.	dirige al panel principal correspondiente a su rol.	conjunta de RF01–RF04.
CP02	E1 — Campos incompletos	Pantalla “Inicio de sesión” visible; cuenta activa de prueba; correo o contraseña vacío; contador de intentos fallidos en 0; sin sesión activa.	El usuario deja un campo vacío y selecciona “Iniciar sesión”.	El sistema muestra “Complete los campos obligatorios.” y no inicia sesión. El contador permanece en 0.	Aprobado en la ejecución automatizada conjunta de RF01–RF04.
CP03	E2 — Formato de correo inválido	Pantalla “Inicio de sesión” visible; correo ingresado `asistente@veterinaria`; contraseña de prueba ingresada; contador de intentos fallidos en 0; sin sesión activa.	El usuario ingresa el correo incorrecto y selecciona “Iniciar sesión”.	El sistema muestra “Ingrese un formato de correo electrónico válido.” y solicita corregir el correo. No inicia sesión.	Aprobado en la ejecución automatizada conjunta de RF01–RF04.
CP04	E3 — Credenciales incorrectas	Pantalla “Inicio de sesión” visible; cuenta activa `asistente@veterinaria.com`; contraseña incorrecta `Incorrecta123`; contador de intentos fallidos en 0; servicio disponible; sin sesión activa.	El usuario ingresa el correo registrado y la contraseña incorrecta, y selecciona “Iniciar sesión”.	El sistema muestra “Error. Ingresar nuevamente las credenciales”, no inicia sesión y aumenta el contador de intentos fallidos de 0 a 1.	Aprobado en la ejecución automatizada conjunta de RF01–RF04.
CP05	E4 — Error de conexión	Pantalla “Inicio de sesión” visible; cuenta de prueba activa; credenciales válidas; servicio de autenticación no disponible; contador de intentos fallidos en 0; sin sesión activa.	El usuario ingresa sus credenciales y selecciona “Iniciar sesión”.	El sistema muestra “No se pudo conectar con el servicio. Intente nuevamente.” y no inicia sesión.	Aprobado en la ejecución automatizada conjunta de RF01–RF04.
CP06	E5 — Demasiados intentos fallidos	Pantalla “Inicio de sesión” visible; cuenta de prueba activa; límite configurado de 5 intentos; contador en 5; servicio disponible; sin sesión activa.	El usuario intenta iniciar sesión nuevamente con una	El sistema bloquea temporalmente los intentos y muestra	Aprobado en la ejecución automatizada conjunta

			contraseña incorrecta.	“Demasiados intentos. Intente nuevamente más tarde.”	de RF01–RF04.

Éxito — inicio de sesión válido

```
test('CP01 - autentica credenciales vÃ¡lidas y permite ir al panel principal', async () => {
  // ARRANGE
  const usuario = {
    id: 'usuario-001',
    email: correoValido,
  };

  autenticar.mockResolvedValue(usuario);
  verificarCuentaActiva.mockResolvedValue({
    autorizada: true,
    mensaje: '',
    perfil: {
      estado_cuenta: 'Activo',
    },
  });

  // ACT
  const resultado = await controlador.iniciarSesion(
    correoValido,
    contrasenaCorrecta
  );

  // ASSERT
  expect(autenticar).toHaveBeenCalledWith(
    correoValido,
    contrasenaCorrecta
  );
  expect(verificarCuentaActiva).toHaveBeenCalledWith(
    usuario.id
  );
```

E1 — Campos incompletos

E2 — Formato de correo inválido

E3 — Credenciales incorrectas

E4 — Error de conexión

E5 — Demasiados intentos fallidos

- Test general

CASOS DE PRUEBA — RF02: Identificar el rol del usuario y mostrar sus funciones

#	Escenario	Precondición	Acción del actor	Resultado esperado	Resultado obtenido
CP01	Éxito — Usuario Administrador	Cuenta de prueba `admin@veterinaria.com` activa; rol Administrador asignado; credenciales válidas; permisos del rol configurados; sin sesión activa.	El usuario inicia sesión con las credenciales del Administrador.	El sistema identifica el rol Administrador y muestra las funciones operativas y las opciones exclusivas de administración, incluida la gestión de usuarios y los indicadores.	Aprobado en la ejecución automatizada conjunta de RF01–RF04.
CP02	Éxito — Usuario Asistente	Cuenta de prueba `asistente@veterinaria.com` activa; rol Asistente asignado; credenciales válidas; permisos	El usuario inicia sesión con las credenciales	El sistema identifica el rol Asistente y muestra las funciones de	Aprobado en la ejecución automatizada

		configurados; sin sesión activa.	del Asistente.	atención, actividades y seguimiento. No muestra la opción Administración.	conjunta de RF01–RF04.
CP03	E1 — Rol no válido	Cuenta de prueba activa `sinrol@veterinaria.com`; sin rol asociado o con un rol no reconocido; credenciales válidas; sin sesión activa.	El usuario intenta iniciar sesión.	El sistema muestra "No se pudo verificar el rol. Intente nuevamente." y no muestra el panel principal.	Aprobado en la ejecución automatizada conjunta de RF01–RF04.
CP04	E2 — Cuenta inactiva	Cuenta de prueba `inactivo@veterinaria.com` desactivada; credenciales válidas; estado Inactivo registrado; sin sesión activa.	El usuario intenta iniciar sesión con sus credenciales.	El sistema impide el acceso y muestra "La cuenta se encuentra inactiva. Comuníquese con el administrador."	Aprobado en la ejecución automatizada conjunta de RF01–RF04.
CP05	E3 — Error al cargar funciones	Cuenta de prueba activa `asistente@veterinaria.com`; rol Asistente válido; credenciales válidas; servicio de carga de funciones no disponible; sin sesión activa.	El usuario intenta iniciar sesión.	El sistema no muestra un panel incompleto y presenta "No se pudo cargar el panel. Intente nuevamente."	Aprobado en la ejecución automatizada conjunta de RF01–RF04.

CP01	Éxito — Usuario Administrador

CP02	Éxito — Usuario Asistente

CP03	E1 — Rol no válido

CP04	E2 — Cuenta inactiva

CP05	E3 — Error al cargar funciones

- Test exitoso

CASOS DE PRUEBA — RF03: Recuperar la contraseña mediante el correo electrónico registrado

#	Escenario	Precondición	Acción del actor	Resultado esperado	Resultado obtenido
CP01	Éxito — Recuperación de contraseña	Pantalla “Recuperar contraseña” visible; cuenta activa asociada a `asistente@veterinaria.com`; contador de solicitudes en 0; servicio de recuperación disponible; nueva contraseña de prueba `Nueva@2026` válida.	El usuario ingresa el correo, selecciona “Enviar enlace”, abre el enlace vigente y establece la nueva contraseña.	El sistema envía el enlace y permite establecer la nueva contraseña.	Aprobado en la ejecución automatizada de RF03.
CP02	E1 — Correo no registrado	Pantalla “Recuperar contraseña” visible; correo `noexiste@correo.com` no asociado a una cuenta; contador de solicitudes en 0; servicio disponible.	El usuario ingresa el correo no registrado y solicita el enlace.	El sistema muestra “No encontramos una cuenta asociada a este correo electrónico” y no permite restablecer una contraseña de una	Aprobado en la ejecución automatizada de RF03.

				cuenta inexistente.	
CP03	E2 — Límite de solicitudes excedido	Pantalla “Recuperar contraseña” visible; cuenta asistente@veterinaria.com; límite de 3 solicitudes por periodo; contador en 3; servicio disponible.	El usuario solicita nuevamente el enlace de recuperación.	El sistema bloquea la nueva solicitud y muestra “Ha excedido el límite de solicitudes. Intente nuevamente más tarde.”	Aprobado en la ejecución automatizada de RF03.
CP04	E3 — Enlace expirado	Enlace generado para asistente@veterinaria.com; tiempo de vigencia finalizado; enlace no utilizado.	El usuario abre el enlace expirado.	El sistema impide continuar y muestra “El enlace de recuperación ha expirado. Solicite un nuevo enlace.”	Aprobado en la ejecución automatizada de RF03.
CP05	E4 — Formato de correo incorrecto	Pantalla “Recuperar contraseña” visible; correo ingresado asistente.veterinaria.com, sin arroba; contador de solicitudes en 0.	El usuario ingresa el correo inválido y solicita el enlace.	El sistema muestra “Ingrese un correo electrónico válido.” y no envía la solicitud.	Aprobado en la ejecución automatizada de RF03.
CP06	E5 — Error al enviar el enlace	Pantalla “Recuperar contraseña” visible; cuenta de prueba asistente@veterinaria.com; contador de solicitudes en 0; servicio de envío de correo simulado como no disponible.	El usuario ingresa el correo y solicita el enlace.	El sistema muestra “No se pudo enviar el enlace de recuperación. Intente nuevamente más tarde.”	Aprobado en la ejecución automatizada de RF03.

- CP01 Éxito — Recuperación de contraseña

- CP02 E1 — Correo no registrado

- CP03 E2 — Límite de solicitudes excedido
- CP04 E3 — Enlace expirado
- CP05 E4 — Formato de correo incorrecto
- CP06 E5 — Error al enviar el enlace

- Test exitoso

CASOS DE PRUEBA — RF04: Mantener activa la sesión mientras las credenciales sean válidas

#	Escenario	Precondición	Acción del actor	Resultado esperado	Resultado obtenido
CP01	Éxito — Sesión vigente	Usuario autenticado previamente `asistente@veterinaria.com`; cuenta activa; sesión guardada en el dispositivo; token válido y vigente; aplicación cerrada sin cerrar sesión.	El usuario abre la aplicación.	El sistema verifica la sesión, valida la cuenta, identifica el rol y muestra directamente el panel principal sin solicitar credenciales.	Aprobado en la ejecución automatizada de RF04.
CP02	E1 — Sesión no encontrada	No existe sesión guardada en el dispositivo por ser el primer acceso o haberse eliminado los datos; servicio de autenticación disponible.	El usuario abre la aplicación.	El sistema muestra la pantalla de inicio de sesión para ingresar las credenciales.	Aprobado en la ejecución automatizada de RF04.
CP03	E2 — Sesión expirada	Sesión guardada para `asistente@veterinaria.com`; tiempo de vigencia	El usuario abre la	El sistema invalida la sesión local y muestra la	Aprobado en la ejecución

		finalizado; renovación de la sesión no disponible o fallida.	aplicación.	pantalla de inicio de sesión.	automatizada de RF04.
CP04	E3 — Sesión inválida	Sesión guardada para `asistente@veterinaria.com`; tiempo de vigencia aún no finalizado; token alterado o revocado; validación del usuario no satisfactoria.	El usuario abre la aplicación.	El sistema invalida la sesión local, muestra la pantalla de inicio de sesión y solicita una nueva autenticación.	Aprobado en la ejecución automatizada de RF04.

Escenario

- CP01 Éxito — Sesión vigente
- CP02 E1 — Sesión no encontrada
- CP03 E2 — Sesión expirada
- CP04 E3 — Sesión inválida
- Test exitoso

CP01 Éxito — actualización de perfil

P02 E1 — Campos incompletos
CP03 E2 — Formato de correo inválido
CP04 E3 — Correo ya registrado
CP05 E4 — Formato de teléfono inválido

CP06 E5 — Confirmación del nuevo correo
CP07 E6 — Error al guardar

1.4. Casos de prueba

1.4. Diagramas de secuencia

DIAGRAMA DE SECUENCIAS — RF001-INICIAR SESIÓN | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
			
AS_Usuario	I_Login	C_Login	E_Usuario

BLOQUE 1 · El sistema muestra la pantalla de inicio de sesión (Pasos 1–2)

1		◄—— 1. mostrarPantallaInicio()	
		El control de autenticación ordena a la interfaz de inicio de sesión mostrar la pantalla de inicio de sesión.	
2		2. vistaDesplegada() - -►	
		La interfaz de inicio de sesión confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.	

BLOQUE 2 · El sistema muestra los campos “Correo electrónico” y “Contraseña”, junto con las opciones “Iniciar sesión” y “¿Olvidaste tu contraseña?” (Pasos 3–4)

3		◄—— 3. mostrarCamposCorreo(correoElectronico)	
		El control de autenticación ordena a la interfaz de inicio de sesión mostrar los campos “Correo electrónico” y “Contraseña”, junto con las opciones “Iniciar sesión” y “¿Olvidaste tu contraseña?”.	
4		4. vistaDesplegada() - -►	
		La interfaz de inicio de sesión confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.	

BLOQUE 3 · El sistema permite visualizar u ocultar la contraseña mediante el ícono correspondiente (Pasos 5–6)

5		◄—— 5. permitirVisualizarOcultar()	

El control de autenticación ordena a la interfaz de inicio de sesión habilitar la opción de visualizar u ocultar la contraseña mediante el ícono correspondiente.

6 ◄ - - 6. mostrarIniciarSesion()

La interfaz de inicio de sesión presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario ingresa su correo electrónico y contraseña (Pasos 7–8)

7 **7. ingresarCorreoElectronico() ——►**

El usuario ingresa su correo electrónico y contraseña en la interfaz de inicio de sesión.

8 ◄ - - 8. camposCompletados()

La interfaz de inicio de sesión refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 5 · El usuario selecciona “Iniciar sesión” (Pasos 9–10)

9 **9. seleccionarIniciarSesion(iniciarSesion) ——►**

El usuario selecciona “Iniciar sesión” en la interfaz de inicio de sesión, utilizando los elementos: Iniciar sesión.

10 **10. enviarIniciarSesion(iniciarSesion) ——►**

La interfaz de inicio de sesión recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema verifica que los campos requeridos estén completos y valida las credenciales mediante el servicio de autenticación configurado (Pasos 11–13)

11 **11. verificarCamposRequeridos(criterio) ——►**

El control de autenticación solicita a la entidad de usuarios (base de datos) la información necesaria para verificar que los campos requeridos estén completos y valida las credenciales mediante el servicio de autenticación configurado.

12 **◄ - - 12. datosIniciarSesion(resultado)**

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

13 **13. verificarCamposRequeridosResultado() ↺**

El control de autenticación procesa internamente los datos recibidos y verifica que los campos requeridos estén completos y valida las credenciales mediante el servicio de autenticación configurado, determinando el resultado de esta etapa.

BLOQUE 7 · Si las credenciales son válidas, el sistema autentica al usuario. Muestra mensaje #"Inicio de sesion exitoso" (Pasos 14–18)

14 **14. autenticarUsuarioMuestra(criterio) ──►**

El control de autenticación solicita a la entidad de usuarios (base de datos) la información necesaria para autenticar al usuario. Muestra mensaje #"Inicio de sesion exitoso" .

15 **◄ - - 15. datosIniciarSesion(resultado)**

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

16 **16. autenticarUsuarioMuestraResultado() ↺**

El control de autenticación procesa internamente los datos recibidos y autentica al usuario. Muestra mensaje #"Inicio de sesion exitoso" , determinando el resultado de esta etapa.

17 **◄── 17. mostrarResultadoIniciarSesion()**

El control de autenticación ordena a la interfaz de inicio de sesión mostrar el resultado de la operación realizada.

18

18. vistaDesplegada() - -►

La interfaz de inicio de sesión confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 8 · El sistema permite continuar hacia el panel principal para acceder a las funciones correspondientes a su rol (Pasos 19–20)

19

◄—— 19. permitirContinuarHacia()

El control de autenticación ordena a la interfaz de inicio de sesión habilitar la opción de continuar hacia el panel principal para acceder a las funciones correspondientes a su rol.

20

◄ - - 20. mostrarIniciarSesion()

La interfaz de inicio de sesión presenta la información en pantalla y la deja disponible para que el usuario continúe.

DIAGRAMA DE SECUENCIAS — RF002-GESTIONAR ACCESO SEGÚN ROL | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Inicio	C_Login	E_Usuario

BLOQUE 1 · El usuario inicia sesión correctamente (Pasos 1–2)

1 — **1. iniciarSesionCorrectamente() ——►** (AS_Usuario → I_Inicio)

El usuario inicia sesión correctamente en la interfaz del panel principal.

2 — **2. enviarGestionarAcceso(solicitud) ——►** (I_Inicio → C_Login)

La interfaz del panel principal recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema consulta el rol asociado a la cuenta autenticada (Pasos 3–5)

3 — **3. consultarRolAsociado(criterio) ——►** (C_Login → E_Usuario)

El control de autenticación solicita a la entidad de usuarios (base de datos) la información necesaria para consultar el rol asociado a la cuenta autenticada.

4 — **◄ - - 4. datosGestionarAcceso(resultado)** (E_Usuario → C_Login)

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

5 — **5. consultarRolAsociadoResultado() ↺** (C_Login)

El control de autenticación procesa internamente los datos recibidos y consulta el rol asociado a la cuenta autenticada, determinando el resultado de esta etapa.

BLOQUE 3 · El sistema identifica si el usuario tiene el rol Administrador o Asistente (Pasos 6–8)

6

6. identificarUsuarioTiene(criterio) ——►

El control de autenticación solicita a la entidad de usuarios (base de datos) la información necesaria para identificar si el usuario tiene el rol Administrador o Asistente.

7

◄ - - 7. datosGestionarAcceso(resultado)

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

8

8. identificarUsuarioTieneResultado() ↺

El control de autenticación procesa internamente los datos recibidos y identifica si el usuario tiene el rol Administrador o Asistente, determinando el resultado de esta etapa.

BLOQUE 4 · Si el rol es Administrador, el sistema habilita las funciones generales y las opciones exclusivas de administración (Pasos 9–10)

9

◄—— 9. habilitarFuncionesGenerales()

El control de autenticación ordena a la interfaz del panel principal habilitar las funciones generales y las opciones exclusivas de administración.

10

10. vistaDesplegada() - -►

La interfaz del panel principal confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 5 · Si el rol es Asistente, el sistema habilita únicamente las funciones relacionadas con atención, actividades y seguimiento de mascotas (Pasos 11–12)

11

◄—— 11. habilitarUnicamenteFunciones()

El control de autenticación ordena a la interfaz del panel principal habilitar únicamente las funciones relacionadas con atención, actividades y seguimiento de mascotas.

12 **12. vistaDesplegada() - -►**

La interfaz del panel principal confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 6 · El sistema muestra el panel principal con las opciones permitidas según el rol identificado (Pasos 13–14)

13 **◄—— 13. mostrarPanelPrincipal()**

El control de autenticación ordena a la interfaz del panel principal mostrar el panel principal con las opciones permitidas según el rol identificado.

14 **◄ - - 14. mostrarGestionarAcceso()**

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

DIAGRAMA DE SECUENCIAS — RF003-RECUPERAR CONTRASEÑA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_RecuperarContrasena	C_Login	E_Usuario

BLOQUE 1 · El usuario selecciona “¿Olvidaste tu contraseña?” en la pantalla de inicio de sesión (Pasos 1–2)

1 — **1. seleccionarOlvidasteTu(olvidasteTu) ——►** (AS_Usuario → I_RecuperarContrasena)

El usuario selecciona “¿Olvidaste tu contraseña?” en la pantalla de inicio de sesión en la interfaz de recuperación de contraseña, utilizando los elementos: ¿Olvidaste tu contraseña?.

2 — **2. enviarRecuperarContrasena(olvidasteTu) ——►** (I_RecuperarContrasena → C_Login)

La interfaz de recuperación de contraseña recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra el campo “Correo electrónico” y la opción “Enviar enlace de recuperación” (Pasos 3–4)

3 — **◄—— 3. mostrarCampoCorreo(correoElectronico)** (C_Login → I_RecuperarContrasena)

El control de autenticación ordena a la interfaz de recuperación de contraseña mostrar el campo “Correo electrónico” y la opción “Enviar enlace de recuperación”.

4 — **◄ - - 4. mostrarRecuperarContrasena()** (I_RecuperarContrasena → AS_Usuario)

La interfaz de recuperación de contraseña presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario ingresa su correo electrónico y selecciona “Enviar enlace de recuperación” (Pasos 5–7)

5 — **5. ingresarCorreoElectronico(enviarEnlace) ——►** (AS_Usuario → I_RecuperarContrasena)

El usuario ingresa su correo electrónico y selecciona “Enviar enlace de recuperación” en la interfaz de recuperación de contraseña.

6 **◄ - - 6. camposCompletados()**

La interfaz de recuperación de contraseña refleja en pantalla los datos ingresados para que el usuario pueda continuar.

7 **7. enviarRecuperarContrasena(solicitud) ——►**

La interfaz de recuperación de contraseña recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema valida el formato del correo electrónico (Pasos 8–10)

8 **8. validarFormatoCorreo(criterio) ——►**

El control de autenticación solicita a la entidad de usuarios (base de datos) la información necesaria para validar el formato del correo electrónico.

9 **◄ - - 9. datosRecuperarContrasena(resultado)**

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

10 **10. validarFormatoCorreoResultado() ↺**

El control de autenticación procesa internamente los datos recibidos y valida el formato del correo electrónico, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema procesa la solicitud y muestra un mensaje de confirmación (Pasos 11–14)

11 **11. procesarSolicitudMuestra(datos) ——►**

El control de autenticación solicita a la entidad de usuarios (base de datos) procesar la solicitud y muestra un mensaje de confirmación.

12

◄ - - 12. operacionConfirmada(estado)

La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

13

◄—— 13. mostrarResultadoRecuperarContrasena()

El control de autenticación ordena a la interfaz de recuperación de contraseña mostrar el resultado de la operación realizada.

14

14. vistaDesplegada() - -►

La interfaz de recuperación de contraseña confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 6 · Si corresponde a una cuenta registrada, el sistema envía un enlace de recuperación al correo asociado (Pasos 15–16)

15

15. enviarEnlaceRecuperacion(datos) ——►

El control de autenticación solicita a la entidad de usuarios (base de datos) enviar un enlace de recuperación al correo asociado.

16

◄ - - 16. operacionConfirmada(estado)

La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 7 · El usuario accede al enlace recibido y establece una nueva contraseña (Pasos 17–18)

17

17. accederEnlaceRecibido() ——►

El usuario accede al enlace recibido y establece una nueva contraseña en la interfaz de recuperación de contraseña.

18

18. enviarRecuperarContrasena(solicitud) ——►

La interfaz de recuperación de contraseña recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema muestra informacion de requisitos necesarios para cambair contraseña (Pasos 19–20)

19 **◄—— 19. mostrarInformacionRequisitos()**

El control de autenticación ordena a la interfaz de recuperación de contraseña mostrar informacion de requisitos necesarios para cambair contraseña.

20 **20. vistaDesplegada() - -►**

La interfaz de recuperación de contraseña confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 9 · El sistema actualiza la contraseña y permite utilizarla para iniciar sesión nuevamente (Pasos 21–24)

21 **21. actualizarContrasenaPermite(datos) ——►**

El control de autenticación solicita a la entidad de usuarios (base de datos) actualizar la contraseña y permite utilizarla para iniciar sesión nuevamente.

22 **◄ - - 22. operacionConfirmada(estado)**

La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

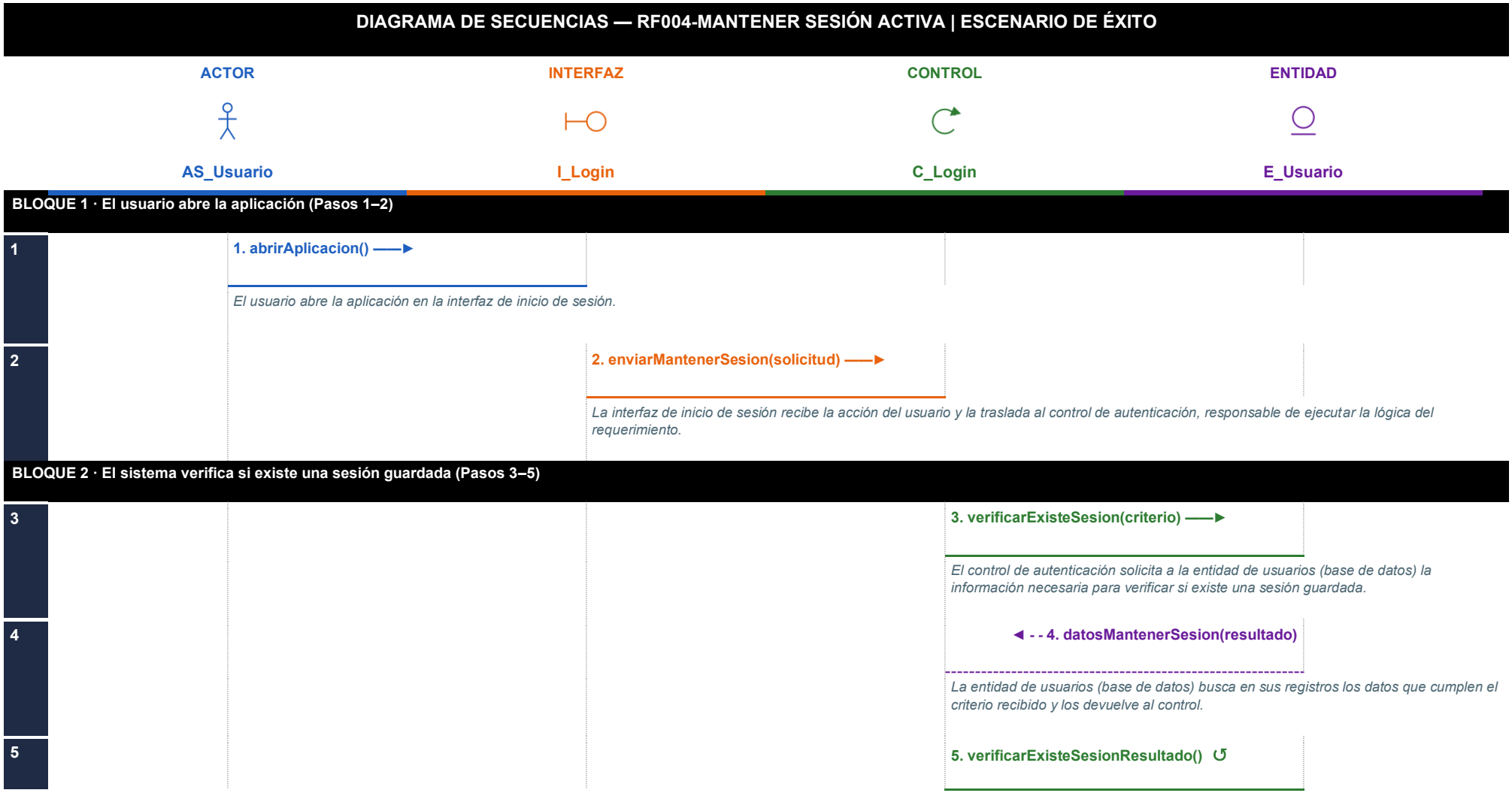
23 **◄—— 23. mostrarResultadoRecuperarContrasena()**

El control de autenticación ordena a la interfaz de recuperación de contraseña mostrar el resultado final de la operación.

24 **◄ - - 24. resultadoMostrado()**

La interfaz de recuperación de contraseña presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias



El control de autenticación procesa internamente los datos recibidos y verifica si existe una sesión guardada, determinando el resultado de esta etapa.

BLOQUE 3 · El sistema valida la vigencia de la sesión (Pasos 6–8)

6

6. validarVigenciaSesion(criterio) ——►

El control de autenticación solicita a la entidad de usuarios (base de datos) la información necesaria para validar la vigencia de la sesión.

7

◄ - - 7. datosMantenerSesion(resultado)

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

8

8. validarVigenciaSesionResultado() ↺

El control de autenticación procesa internamente los datos recibidos y valida la vigencia de la sesión, determinando el resultado de esta etapa.

BLOQUE 4 · Si la sesión es válida, el sistema identifica el rol del usuario (Pasos 9–11)

9

9. identificarRolUsuario(criterio) ——►

El control de autenticación solicita a la entidad de usuarios (base de datos) la información necesaria para identificar el rol del usuario.

10

◄ - - 10. datosMantenerSesion(resultado)

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

11

11. identificarRolUsuarioResultado() ↺

El control de autenticación procesa internamente los datos recibidos y identifica el rol del usuario, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema muestra directamente el panel principal correspondiente sin solicitar nuevamente las credenciales (Pasos 12–13)

12 ◄—— 12. mostrarDirectamentePanel()

El control de autenticación ordena a la interfaz de inicio de sesión mostrar directamente el panel principal correspondiente sin solicitar nuevamente las credenciales.

13 ◄ - - 13. mostrarMantenerSesion()

La interfaz de inicio de sesión presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · Si el usuario sale temporalmente de la aplicación y vuelve a abrirla, el sistema verifica nuevamente la sesión (Pasos 14–18)

14 14. siUsuarioSale() ——►

Si el usuario sale temporalmente de la aplicación y vuelve a abrirla en la interfaz de inicio de sesión.

15 15. enviarMantenerSesion(solicitud) ——►

La interfaz de inicio de sesión recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

16 16. verificarNuevamenteSesion(criterio) ——►

El control de autenticación solicita a la entidad de usuarios (base de datos) la información necesaria para verificar nuevamente la sesión.

17 ◄ - - 17. datosMantenerSesion(resultado)

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

18 18. verificarNuevamenteSesionResultado() ↺

El control de autenticación procesa internamente los datos recibidos y verifica nuevamente la sesión, determinando el resultado de esta etapa.

BLOQUE 7 · Si la sesión continúa vigente, el sistema mantiene el acceso al panel correspondiente (Pasos 19–22)

19

19. mantenerAccesoPanel(datos) ——►

El control de autenticación solicita a la entidad de usuarios (base de datos) mantener el acceso al panel correspondiente.

20

◄ - - 20. operacionConfirmada(estado)

La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

21

◄—— 21. mostrarResultadoMantenerSesion()

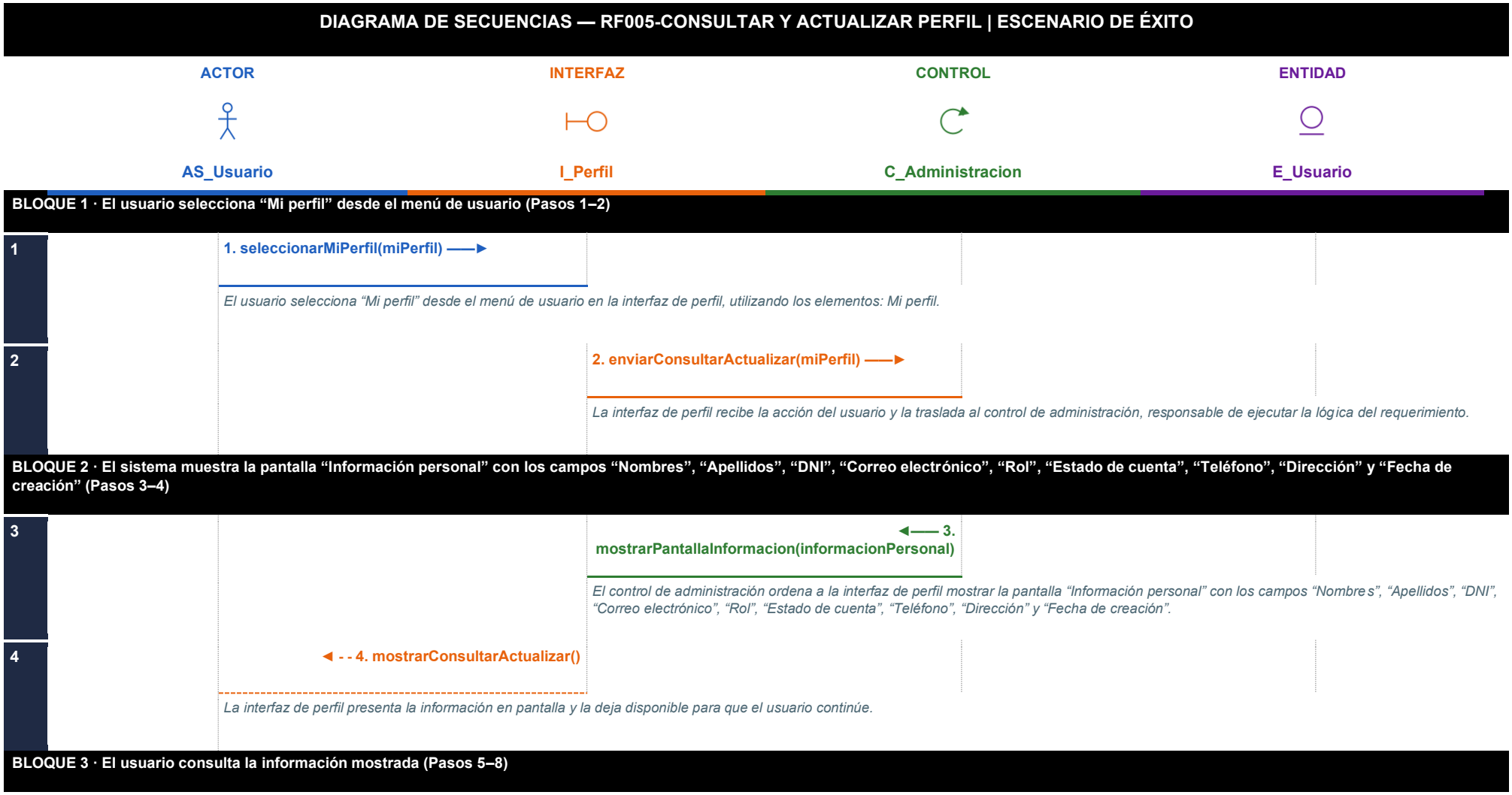
El control de autenticación ordena a la interfaz de inicio de sesión mostrar el resultado final de la operación.

22

◄ - - 22. resultadoMostrado()

La interfaz de inicio de sesión presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias



5

5. consultarInformacionMostrada() ——►

El usuario consulta la información mostrada en la interfaz de perfil.

6

6. enviarConsultarActualizar(solicitud) ——►

La interfaz de perfil recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

7

◄ - - 7. solicitudRecibida()

El control de administración registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

8

◄ - - 8. mostrarSiguientePaso()

La interfaz de perfil habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 4 · El usuario selecciona “Editar información” (Pasos 9–10)

9

9. seleccionarEditarInformacion(editarInformacion) ——►

El usuario selecciona “Editar información” en la interfaz de perfil, utilizando los elementos: Editar información.

10

10. enviarConsultarActualizar(editarInformacion) ——►

La interfaz de perfil recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema habilita los campos “Nombres”, “Apellidos”, “Correo electrónico”, “Teléfono” y “Dirección” para su modificación (Pasos 11–12)

11

◄—— 11. habilitarCamposNombres(nombres)

El control de administración ordena a la interfaz de perfil habilitar los campos “Nombres”, “Apellidos”, “Correo electrónico”, “Teléfono” y “Dirección” para su modificación.

12

12. vistaDesplegada() - -►

La interfaz de perfil confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 6 · El sistema mantiene “DNI”, “Rol”, “Estado de cuenta” y “Fecha de creación” como campos de solo lectura (Pasos 13–14)

13

13. mantenerDniRol(datos) ——►

El control de administración solicita a la entidad de usuarios (base de datos) mantener “DNI”, “Rol”, “Estado de cuenta” y “Fecha de creación” como campos de solo lectura.

14

◄ - - 14. operacionConfirmada(estado)

La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 7 · El usuario modifica los datos necesarios y selecciona “Guardar cambios” (Pasos 15–16)

15

15. modificarDatosNecesarios(guardarCambios) ——►

El usuario modifica los datos necesarios y selecciona “Guardar cambios” en la interfaz de perfil, utilizando los elementos: Guardar cambios.

16

16. enviarConsultarActualizar(guardarCambios) ——►

La interfaz de perfil recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema valida la información y muestra el mensaje: “¿Deseas guardar los cambios realizados en tu perfil?”, con las opciones “Confirmar” y “Cancelar” (Pasos 17–21)

17

17. validarInformacionMuestra(criterio) ——►

El control de administración solicita a la entidad de usuarios (base de datos) la información necesaria para validar la información y muestra el mensaje: "¿Deseas guardar los cambios realizados en tu perfil?", con las opciones "Confirmar" y "Cancelar".

18 **◄ - - 18. datosConsultarActualizar(resultado)**

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

19 **19. validarInformacionMuestraResultado() ↺**

El control de administración procesa internamente los datos recibidos y valida la información y muestra el mensaje: "¿Deseas guardar los cambios realizados en tu perfil?", con las opciones "Confirmar" y "Cancelar", determinando el resultado de esta etapa.

20 **◄—— 20. mostrarResultadoConsultarActualizar()**

El control de administración ordena a la interfaz de perfil mostrar el resultado de la operación realizada.

21 **◄ - - 21. mostrarConsultarActualizar()**

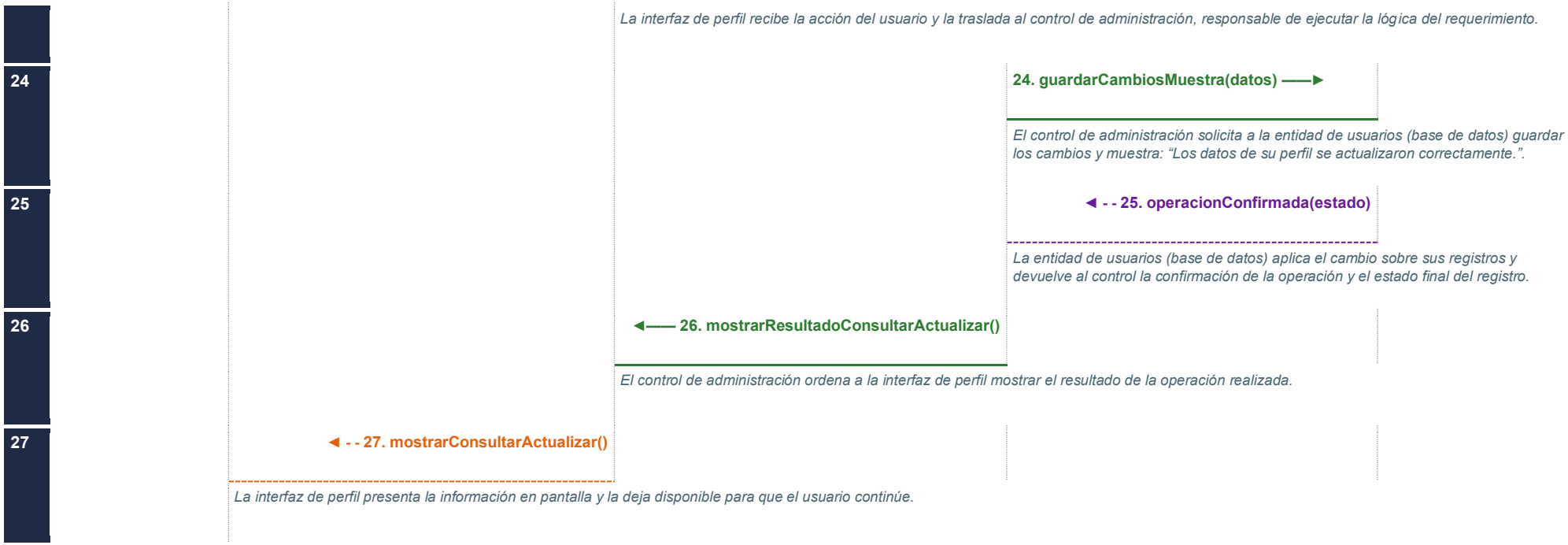
La interfaz de perfil presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 9 · Si el usuario confirma, el sistema guarda los cambios y muestra: "Los datos de su perfil se actualizaron correctamente." (Pasos 22–27)

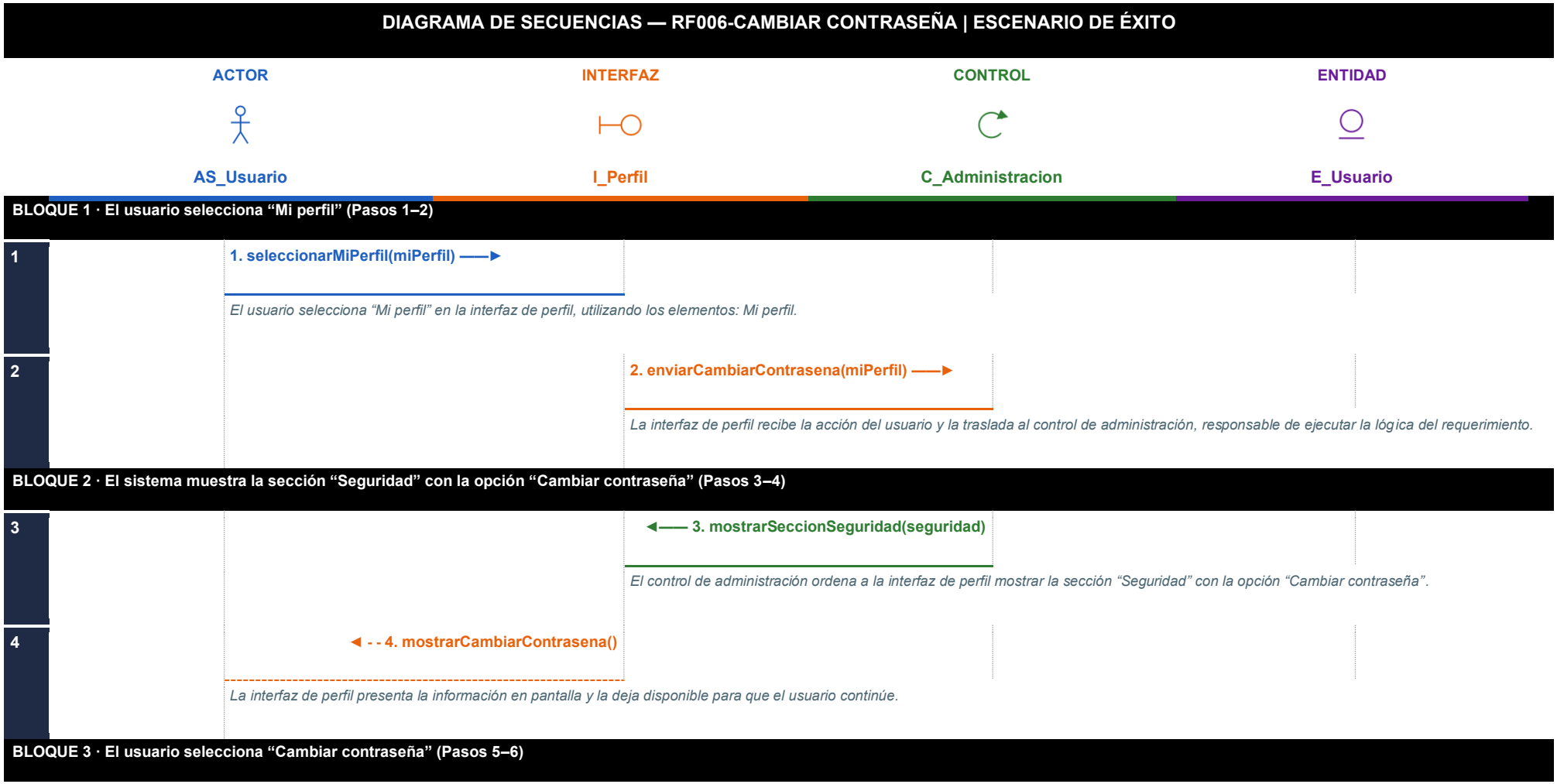
22 **22. confirmarOperacion() ——►**

Si el usuario confirma en la interfaz de perfil.

23 **23. enviarConsultarActualizar(solicitud) ——►**


La interfaz de perfil recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.
24
24. guardarCambiosMuestra(datos) ——►
El control de administración solicita a la entidad de usuarios (base de datos) guardar los cambios y muestra: “Los datos de su perfil se actualizaron correctamente.”.
25
◄ - - 25. operacionConfirmada(estado)
La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.
26
◄—— 26. mostrarResultadoConsultarActualizar()
El control de administración ordena a la interfaz de perfil mostrar el resultado de la operación realizada.
27
◄ - - 27. mostrarConsultarActualizar()
La interfaz de perfil presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias



5

5. seleccionarCambiarContrasena(cambiarContrasena) ——►

El usuario selecciona “Cambiar contraseña” en la interfaz de perfil, utilizando los elementos: Cambiar contraseña.

6

6. enviarCambiarContrasena(cambiarContrasena) ——►

La interfaz de perfil recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la pantalla “Cambiar contraseña” con los campos “Contraseña actual”, “Nueva contraseña” y “Confirmar nueva contraseña”, además de los botones “Cancelar” y “Guardar contraseña”. Muestra ademas los requisitos para cerar contraseña nueva (Pasos 7–8)

7

◄—— 7. mostrarPantallaCambiar(cambiarContrasena)

El control de administración ordena a la interfaz de perfil mostrar la pantalla “Cambiar contraseña” con los campos “Contraseña actual”, “Nueva contraseña” y “Confirmar nueva contraseña”, además de los botones “Cancelar” y “Guardar contraseña”. Muestra ademas los requisitos para cerar contraseña nueva.

8

◄ - - 8. mostrarCambiarContrasena()

La interfaz de perfil presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario ingresa su contraseña actual, una nueva contraseña y la confirmación de la nueva contraseña (Pasos 9–10)

9

9. ingresarContrasenaActual() ——►

El usuario ingresa su contraseña actual, una nueva contraseña y la confirmación de la nueva contraseña en la interfaz de perfil.

10

◄ - - 10. camposCompletados()

La interfaz de perfil refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 6 · El usuario selecciona “Guardar contraseña” (Pasos 11–12)

11

11. seleccionarGuardarContrasena(guardarContrasena) ——►

El usuario selecciona “Guardar contraseña” en la interfaz de perfil, utilizando los elementos: Guardar contraseña.

12

12. enviarCambiarContrasena(guardarContrasena) ——►

La interfaz de perfil recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema valida la contraseña actual, verifica los requisitos de la nueva contraseña y comprueba que coincida con su confirmación (Pasos 13–15)

13

13. validarContrasenaActual(criterio) ——►

El control de administración solicita a la entidad de usuarios (base de datos) la información necesaria para validar la contraseña actual, verifica los requisitos de la nueva contraseña y comprueba que coincida con su confirmación.

14

◄ - - 14. datosCambiarContrasena(resultado)

La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

15

15. validarContrasenaActualResultado() ↺

El control de administración procesa internamente los datos recibidos y valida la contraseña actual, verifica los requisitos de la nueva contraseña y comprueba que coincida con su confirmación, determinando el resultado de esta etapa.

BLOQUE 8 · Si los datos son válidos, el sistema actualiza la contraseña y muestra: “La contraseña se cambió correctamente.” (Pasos 16–19)

16

16. actualizarContrasenaMuestra(datos) ——►

El control de administración solicita a la entidad de usuarios (base de datos) actualizar la contraseña y muestra: “La contraseña se cambió correctamente.”.

17 **◄ - - 17. operacionConfirmada(estado)**

La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

18 **◄—— 18. mostrarResultadoCambiarContrasena()**

El control de administración ordena a la interfaz de perfil mostrar el resultado de la operación realizada.

19 **◄ - - 19. mostrarCambiarContrasena()**

La interfaz de perfil presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF007-CERRAR SESIÓN | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Inicio	C_Login	E_Usuario

BLOQUE 1 · El usuario se encuentra dentro de la aplicación con una sesión activa (Pasos 1–2)

1 — **1. seerEncuentraDentro() ——►** (AS_Usuario)

El usuario se encuentra dentro de la aplicación con una sesión activa en la interfaz del panel principal.

2 — **2. enviarCerrarSesion(solicitud) ——►** (I_Inicio)

La interfaz del panel principal recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra la opción “Cerrar sesión” en el menú de usuario (Pasos 3–4)

3 — **◄—— 3. mostrarOpcionCerrar(cerrarSesion)** (C_Login)

El control de autenticación ordena a la interfaz del panel principal mostrar la opción “Cerrar sesión” en el menú de usuario.

4 — **◄ - - 4. mostrarCerrarSesion()** (I_Inicio)

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona la opción “Cerrar sesión” (Pasos 5–6)

5 — **5. seleccionarOpcionCerrar(cerrarSesion) ——►** (AS_Usuario)

El usuario selecciona la opción “Cerrar sesión” en la interfaz del panel principal, utilizando los elementos: Cerrar sesión.

6

6. enviarCerrarSesion(cerrarSesion) ——►

La interfaz del panel principal recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra un mensaje de confirmación indicando “¿Desea cerrar sesión?”, junto con las opciones “Confirmar” y “Cancelar” (Pasos 7–8)

7

◄—— 7. mostrarMensajeConfirmacion(deseaCerrar)

El control de autenticación ordena a la interfaz del panel principal mostrar un mensaje de confirmación indicando “¿Desea cerrar sesión?”, junto con las opciones “Confirmar” y “Cancelar”.

8

◄ - - 8. mostrarCerrarSesion()

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona “Confirmar” (Pasos 9–10)

9

9. seleccionarConfirmar(confirmar) ——►

El usuario selecciona “Confirmar” en la interfaz del panel principal, utilizando los elementos: Confirmar.

10

10. enviarCerrarSesion(confirmar) ——►

La interfaz del panel principal recibe la acción del usuario y la traslada al control de autenticación, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema finaliza la sesión activa y restringe el acceso a las funciones protegidas (Pasos 11–12)

11

11. finalizarSesionActiva(datos) ——►

El control de autenticación solicita a la entidad de usuarios (base de datos) finalizar la sesión activa y restringe el acceso a las funciones protegidas.

12

◄ - - 12. operacionConfirmada(estado)

La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 7 · El sistema redirige al usuario a la pantalla de inicio de sesión (Pasos 13–14)

13

◄—— 13. redirigirUsuarioPantalla()

El control de autenticación ordena a la interfaz del panel principal redirigir al usuario a la pantalla de inicio de sesión.

14

◄ - - 14. mostrarCerrarSesion()

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF008-VISUALIZAR PANEL PRINCIPAL | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
			
AS_Usuario	I_Inicio	C_Seguimiento	E_Atencion

BLOQUE 1 · El usuario inicia sesión correctamente y carga la pantalla "Inicio" (Pasos 1–2)

1 — **1. iniciarSesionCorrectamente(inicio) ——►** (AS_Usuario → I_Inicio)

El usuario inicia sesión correctamente y carga la pantalla "Inicio" en la interfaz del panel principal.

2 — **2. enviarVisualizarPanel(inicio) ——►** (I_Inicio → C_Seguimiento)

La interfaz del panel principal recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema organiza la información para mostrar un resumen general (Pasos 3–7)

3 — **3. organizarInformacionMostrar(criterio) ——►** (C_Seguimiento → E_Atencion)

El control de seguimiento solicita a la entidad de atenciones (base de datos) la información necesaria para organizar la información para mostrar un resumen general.

4 — **◄ - - 4. datosVisualizarPanel(resultado)** (E_Atencion → C_Seguimiento)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

5 — **5. organizarInformacionMostrarResultado() ↺** (C_Seguimiento)

El control de seguimiento procesa internamente los datos recibidos y organiza la información para mostrar un resumen general, determinando el resultado de esta etapa.

6 **◄—— 6. mostrarResultadoVisualizarPanel()**

El control de seguimiento ordena a la interfaz del panel principal mostrar el resultado de la operación realizada.

7 **7. vistaDesplegada() - -►**

La interfaz del panel principal confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 3 · Se muestra el saludo, nombre y rol del usuario, junto con el resumen de Atenciones activas y Resumen de actividades (Pasos 8–9)

8 **◄—— 8. mostrarSaludoNombre()**

El control de seguimiento ordena a la interfaz del panel principal mostrar el saludo, nombre y rol del usuario, junto con el resumen de Atenciones activas y Resumen de actividades.

9 **9. vistaDesplegada() - -►**

La interfaz del panel principal confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 4 · El sistema muestra los indicadores de Pendientes, En proceso, Completadas y No realizadas (Pasos 10–11)

10 **◄—— 10. mostrarIndicadoresPendientes()**

El control de seguimiento ordena a la interfaz del panel principal mostrar los indicadores de Pendientes, En proceso, Completadas y No realizadas.

11 **11. vistaDesplegada() - -►**

La interfaz del panel principal confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 5 · El sistema muestra los avisos de actividades vencidas (Pasos 12–13)

12 ◄—— 12. mostrarAvisosActividades()

El control de seguimiento ordena a la interfaz del panel principal mostrar los avisos de actividades vencidas.

13 ◄ - - 13. mostrarVisualizarPanel()

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario puede seleccionar una atención, estado o aviso del panel (Pasos 14–15)

14 14. seleccionarAtencionEstado() ——►

El usuario puede seleccionar una atención, estado o aviso del panel en la interfaz del panel principal.

15 15. enviarVisualizarPanel(solicitud) ——►

La interfaz del panel principal recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

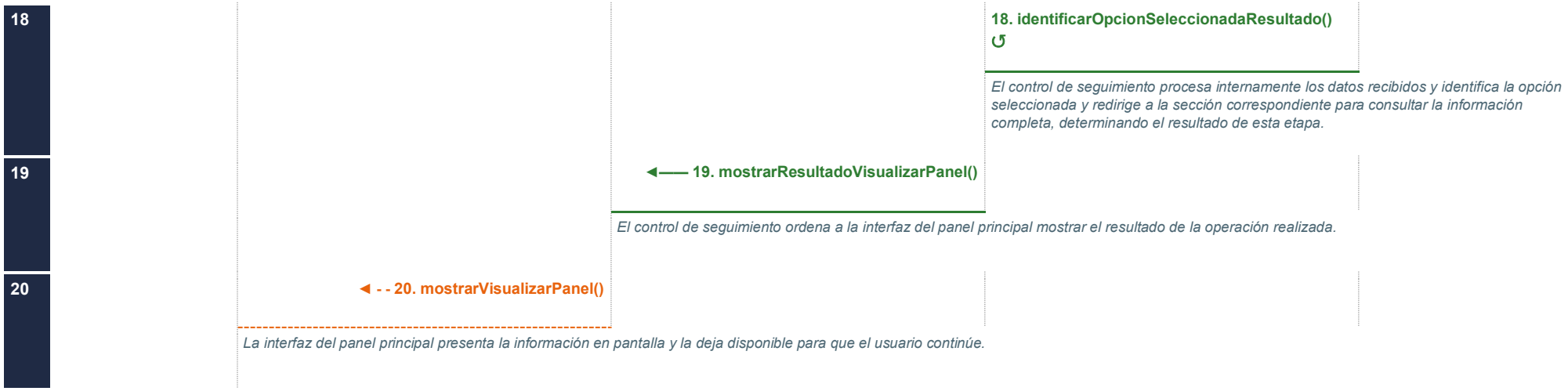
BLOQUE 7 · El sistema identifica la opción seleccionada y redirige a la sección correspondiente para consultar la información completa (Pasos 16–20)

16 16. identificarOpcionSeleccionada(criterio) ——►

El control de seguimiento solicita a la entidad de atenciones (base de datos) la información necesaria para identificar la opción seleccionada y redirige a la sección correspondiente para consultar la información completa.

17 ◄ - - 17. datosVisualizarPanel(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.


18
18. identificarOpcionSeleccionadaResultado()
El control de seguimiento procesa internamente los datos recibidos y identifica la opción seleccionada y redirige a la sección correspondiente para consultar la información completa, determinando el resultado de esta etapa.
19
19. mostrarResultadoVisualizarPanel()
El control de seguimiento ordena a la interfaz del panel principal mostrar el resultado de la operación realizada.
20
20. mostrarVisualizarPanel()
La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF009-VISUALIZAR ATENCIONES ACTIVAS | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Inicio	C_Atencion	E_Atencion

BLOQUE 1 · El usuario accede a la pantalla "Inicio" (Pasos 1–2)

1. **1. accederPantallaInicio(inicio) ——►** (AS_Usuario → I_Inicio)

   El usuario accede a la pantalla "Inicio" en la interfaz del panel principal, utilizando los elementos: Inicio.

2. **2. enviarVisualizarAtenciones(inicio) ——►** (I_Inicio → C_Atencion)

   La interfaz del panel principal recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema consulta las atenciones veterinarias registradas que se encuentran en estado "Activa" (Pasos 3–5)

3. **3. consultarAtencionesVeterinarias(criterio) ——►** (C_Atencion → E_Atencion)

   El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para consultar las atenciones veterinarias registradas que se encuentran en estado "Activa".

4. **◄ - - 4. datosVisualizarAtenciones(resultado)** (E_Atencion → C_Atencion)

   La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

5. **5. consultarAtencionesVeterinariasResultado() ↺** (C_Atencion)

El control de atenciones procesa internamente los datos recibidos y consulta las atenciones veterinarias registradas que se encuentran en estado "Activa", determinando el resultado de esta etapa.

BLOQUE 3 · El sistema muestra en el panel principal el indicador "Atenciones activas" junto con la cantidad correspondiente (Pasos 6–7)

6 **◄—— 6. mostrarPanelPrincipal(atencionesActivas)**

El control de atenciones ordena a la interfaz del panel principal mostrar en el panel principal el indicador "Atenciones activas" junto con la cantidad correspondiente.

7 **◄ - - 7. mostrarVisualizarAtenciones()**

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario visualiza el indicador y la cantidad de atenciones activas disponibles en el panel principal (Pasos 8–11)

8 **8. visualizarIndicadorCantidad() ——►**

El usuario visualiza el indicador y la cantidad de atenciones activas disponibles en el panel principal en la interfaz del panel principal.

9 **9. enviarVisualizarAtenciones(solicitud) ——►**

La interfaz del panel principal recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

10 **◄ - - 10. solicitudRecibida()**

El control de atenciones registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

11 **◄ - - 11. mostrarSiguientePaso()**

La interfaz del panel principal habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 5 · El usuario selecciona el indicador “Atenciones activas” (Pasos 12–13)

12

12. seleccionarIndicadorAtenciones(atencionesActivas) ——►

El usuario selecciona el indicador “Atenciones activas” en la interfaz del panel principal, utilizando los elementos: Atenciones activas.

13

13. enviarVisualizarAtenciones(atencionesActivas) ——►

La interfaz del panel principal recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema redirige al usuario a la sección “Atenciones” (Pasos 14–15)

14

◄—— 14. redirigirUsuarioSeccion(atenciones)

El control de atenciones ordena a la interfaz del panel principal redirigir al usuario a la sección “Atenciones”.

15

◄ - - 15. mostrarVisualizarAtenciones()

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario puede consultar desde esta sección la información registrada de las atenciones activas (Pasos 16–19)

16

16. consultarSeccionInformacion() ——►

El usuario puede consultar desde esta sección la información registrada de las atenciones activas en la interfaz del panel principal.

17

17. enviarVisualizarAtenciones(solicitud) ——►

La interfaz del panel principal recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

18

◄—— 18. mostrarResultadoVisualizarAtenciones()

El control de atenciones ordena a la interfaz del panel principal mostrar el resultado final de la operación.

19

◄ - - 19. resultadoMostrado()

La interfaz del panel principal presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF010-VISUALIZAR RESUMEN DE ACTIVIDADES | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Inicio	C_Seguimiento	E_Actividad

BLOQUE 1 · El usuario ingresa a "Inicio" (Pasos 1–2)

1

1. ingresarInicio(inicio) ——► (AS_Usuario → I_Inicio)

El usuario ingresa a "Inicio" en la interfaz del panel principal, utilizando los elementos: Inicio.

2

2. enviarVisualizarResumen(inicio) ——► (I_Inicio → C_Seguimiento)

La interfaz del panel principal recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema consulta y agrupa las actividades según sus estados: Pendiente, Completada y No realizada (Pasos 3–5)

3

3. consultarAgrupaActividades(criterio) ——► (C_Seguimiento → E_Actividad)

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para consultar y agrupa las actividades según sus estados: Pendiente, Completada y No realizada.

4

◄ - - 4. datosVisualizarResumen(resultado) (E_Actividad → C_Seguimiento)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

5

5. consultarAgrupaActividadesResultado() ↺ (C_Seguimiento)

El control de seguimiento procesa internamente los datos recibidos y consulta y agrupa las actividades según sus estados: Pendiente, Completada y No realizada, determinando el resultado de esta etapa.

BLOQUE 3 · El la pantalla se muestra el bloque “Resumen de actividades” con la cantidad correspondiente a cada estado (Pasos 6–8)

6

6. elPantallaMuestra() ↺

El control de seguimiento realiza internamente la lógica necesaria para el la pantalla se muestra el bloque “Resumen de actividades” con la cantidad correspondiente a cada estado.

7

◄—— 7. mostrarResultadoVisualizarResumen()

El control de seguimiento ordena a la interfaz del panel principal mostrar el resultado de la operación realizada.

8

◄ - - 8. mostrarVisualizarResumen()

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona uno de los indicadores (Pasos 9–10)

9

9. seleccionarUnoIndicadores() ——►

El usuario selecciona uno de los indicadores en la interfaz del panel principal.

10

10. enviarVisualizarResumen(solicitud) ——►

La interfaz del panel principal recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema identifica el estado seleccionado y redirige a “Actividades” (Pasos 11–15)

11

11. identificarEstadoSeleccionado(criterio) —— ►

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para identificar el estado seleccionado y redirige a “Actividades”.

12 **◄ - - 12. datosVisualizarResumen(resultado)**

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

13 **13. identificarEstadoSeleccionadoResultado()** ↺

El control de seguimiento procesa internamente los datos recibidos y identifica el estado seleccionado y redirige a “Actividades”, determinando el resultado de esta etapa.

14 **◄—— 14. mostrarResultadoVisualizarResumen()**

El control de seguimiento ordena a la interfaz del panel principal mostrar el resultado de la operación realizada.

15 **15. vistaDesplegada() - -►**

La interfaz del panel principal confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

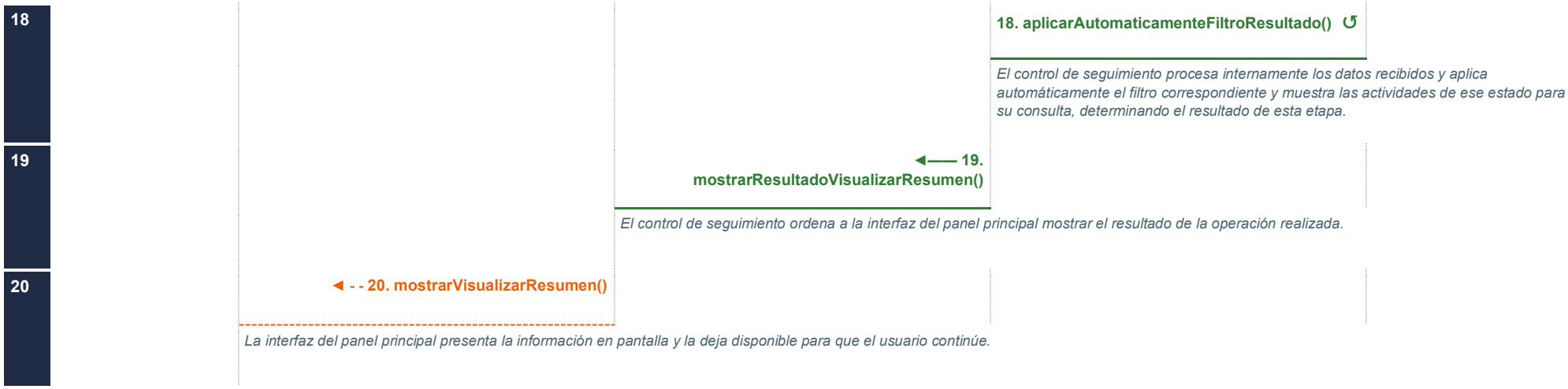
BLOQUE 6 · El sistema aplica automáticamente el filtro correspondiente y muestra las actividades de ese estado para su consulta (Pasos 16–20)

16 **16. aplicarAutomaticamenteFiltro(criterio) ——►**

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para aplicar automáticamente el filtro correspondiente y muestra las actividades de ese estado para su consulta.

17 **◄ - - 17. datosVisualizarResumen(resultado)**

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.


18
18. aplicarAutomaticamenteFiltroResultado()
El control de seguimiento procesa internamente los datos recibidos y aplica automáticamente el filtro correspondiente y muestra las actividades de ese estado para su consulta, determinando el resultado de esta etapa.
19
◄—— 19. mostrarResultadoVisualizarResumen()
El control de seguimiento ordena a la interfaz del panel principal mostrar el resultado de la operación realizada.
20
◄ - - 20. mostrarVisualizarResumen()
La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF011-MOSTRAR AVISOS DE ACTIVIDADES PRÓXIMAS A VENCER | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
			
AS_Usuario	I_Inicio	C_Seguimiento	E_Actividad

BLOQUE 1 · El usuario ingresa a la pantalla "Inicio" (Pasos 1–2)

1 — **1. ingresarPantallaInicio(inicio) ——►** (AS_Usuario → I_Inicio)

El usuario ingresa a la pantalla "Inicio" en la interfaz del panel principal, utilizando los elementos: Inicio.

2 — **2. enviarMostrarAvisos(inicio) ——►** (I_Inicio → C_Seguimiento)

La interfaz del panel principal recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema consulta las actividades que se encuentran en estado Pendiente (Pasos 3–5)

3 — **3. consultarActividadesEncuentran(criterio) ——►** (C_Seguimiento → E_Actividad)

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para consultar las actividades que se encuentran en estado Pendiente.

4 — **◄ - - 4. datosMostrarAvisos(resultado)** (E_Actividad → C_Seguimiento)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

5 — **5. consultarActividadesEncuentranResultado() ↺** (C_Seguimiento)

El control de seguimiento procesa internamente los datos recibidos y consulta las actividades que se encuentran en estado Pendiente, determinando el resultado de esta etapa.

BLOQUE 3 · El sistema verifica la fecha prevista de cada actividad (Pasos 6–8)

6

6. verificarFechaPrevista(criterio) ——►

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para verificar la fecha prevista de cada actividad.

7

◄ - - 7. datosMostrarAvisos(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

8

8. verificarFechaPrevistaResultado() ↺

El control de seguimiento procesa internamente los datos recibidos y verifica la fecha prevista de cada actividad, determinando el resultado de esta etapa.

BLOQUE 4 · El sistema identifica las actividades cuya fecha prevista se encuentra dentro de las próximas 24 horas (Pasos 9–11)

9

9. identificarActividadesCuya(criterio) ——►

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para identificar las actividades cuya fecha prevista se encuentra dentro de las próximas 24 horas.

10

◄ - - 10. datosMostrarAvisos(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

11

11. identificarActividadesCuyaResultado() ↺

El control de seguimiento procesa internamente los datos recibidos y identifica las actividades cuya fecha prevista se encuentra dentro de las próximas 24 horas, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema cambia el color de la tarjeta de la mascota y muestra en la parte superior un mensaje indicando que existe una actividad próxima a vencer (Pasos 12–15)

12

12. cambiarColorTarjeta(datos) ——►

El control de seguimiento solicita a la entidad de actividades (base de datos) cambiar el color de la tarjeta de la mascota y muestra en la parte superior un mensaje indicando que existe una actividad próxima a vencer.

13

◄ - - 13. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

14

◄—— 14. mostrarResultadoMostrarAvisos()

El control de seguimiento ordena a la interfaz del panel principal mostrar el resultado de la operación realizada.

15

◄ - - 15. mostrarMostrarAvisos()

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario selecciona el aviso o la opción “Ver actividad” (Pasos 16–17)

16

16. seleccionarAvisoOpcion(verActividad) —— ►

El usuario selecciona el aviso o la opción “Ver actividad” en la interfaz del panel principal, utilizando los elementos: Ver actividad.

17

17. enviarMostrarAvisos(verActividad) ——►

La interfaz del panel principal recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema redirige a “Atenciones” y muestra la actividad correspondiente para consultar y gestionar su estado (Pasos 18–19)

18

◄—— 18. redirigirAtencionesMuestra(atenciones)

El control de seguimiento ordena a la interfaz del panel principal redirigir a “Atenciones” y muestra la actividad correspondiente para consultar y gestionar su estado.

19

◄ - - 19. mostrarMostrarAvisos()

La interfaz del panel principal presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF012-REGISTRAR PROPIETARIO | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Propietarios	C_Propietario	E_Propietario

BLOQUE 1 · El usuario selecciona la sección “Propietarios” (Pasos 1–2)

1	1. seleccionarSeccionPropietarios(propietarios) ——►			
	El usuario selecciona la sección “Propietarios” en la interfaz de propietarios, utilizando los elementos: Propietarios.			
2		2. enviarRegistrarPropietario(propietarios) ——►		
		La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.		

BLOQUE 2 · El sistema muestra la lista de propietarios registrados y la opción “Nuevo” (Pasos 3–4)

3		◄—— 3. mostrarListaPropietarios(nuevo)		
		El control de propietarios ordena a la interfaz de propietarios mostrar la lista de propietarios registrados y la opción “Nuevo”.		
4	◄ - - 4. mostrarRegistrarPropietario()			
	La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.			

BLOQUE 3 · El usuario selecciona “Nuevo” (Pasos 5–6)

5	5. seleccionarNuevo(nuevo) ——►			

El usuario selecciona “Nuevo” en la interfaz de propietarios, utilizando los elementos: Nuevo.

6

6. enviarRegistrarPropietario(nuevo) ——►

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra el formulario de registro (Pasos 7–8)

7

◄—— 7. mostrarFormularioRegistro()

El control de propietarios ordena a la interfaz de propietarios mostrar el formulario de registro.

8

◄ - - 8. mostrarRegistrarPropietario()

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario ingresa nombres, dni, apellidos, teléfono, correo electrónico, dirección y observaciones (Pasos 9–10)

9

9. ingresarNombresDni() ——►

El usuario ingresa nombres, dni, apellidos, teléfono, correo electrónico, dirección y observaciones en la interfaz de propietarios.

10

◄ - - 10. camposCompletados()

La interfaz de propietarios refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 6 · El usuario selecciona “Registrar propietario” (Pasos 11–12)

11

11. seleccionarRegistrarPropietario(registrarPropietario) ——►

El usuario selecciona “Registrar propietario” en la interfaz de propietarios, utilizando los elementos: Registrar propietario.

12

12. enviarRegistrarPropietario(registrarPropietario) ⟶

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema valida los datos ingresados y muestra la una pantalla con todos los datos de propietario y la opción "Cancelar y "Confirmar registro" (Pasos 13–17)

13

13. validarDatosIngresados(criterio) ⟶

El control de propietarios solicita a la entidad de propietarios (base de datos) la información necesaria para validar los datos ingresados y muestra la una pantalla con todos los datos de propietario y la opción "Cancelar y "Confirmar registro".

14

⟵ - - 14. datosRegistrarPropietario(resultado)

La entidad de propietarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

15

15. validarDatosIngresadosResultado() ↺

El control de propietarios procesa internamente los datos recibidos y valida los datos ingresados y muestra la una pantalla con todos los datos de propietario y la opción "Cancelar y "Confirmar registro", determinando el resultado de esta etapa.

16

⟵ 16. mostrarResultadoRegistrarPropietario()

El control de propietarios ordena a la interfaz de propietarios mostrar el resultado de la operación realizada.

17

⟵ - - 17. mostrarRegistrarPropietario()

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · El usuario confirma el registro (Pasos 18–19)

18 **18. confirmarOperacion() ——►**

El usuario confirma el registro en la interfaz de propietarios.

19 **19. enviarRegistrarPropietario(solicitud) ——►**

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 9 · El sistema registra al propietario y muestra un mensaje de confirmación (Pasos 20–23)

20 **20. registrarPropietarioMuestra(datos) ——►**

El control de propietarios solicita a la entidad de propietarios (base de datos) registrar al propietario y muestra un mensaje de confirmación.

21 **◄ - - 21. operacionConfirmada(estado)**

La entidad de propietarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

22 **◄—— 22. mostrarResultadoRegistrarPropietario()**

El control de propietarios ordena a la interfaz de propietarios mostrar el resultado de la operación realizada.

23 **◄ - - 23. mostrarRegistrarPropietario()**

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF013-CONSULTAR PROPIETARIO | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Propietarios	C_Propietario	E_Propietario

BLOQUE 1 · El usuario selecciona la sección “Propietarios” (Pasos 1–2)

1 — AS_Usuario → I_Propietarios: **1. seleccionarSeccionPropietarios(propietarios)**

El usuario selecciona la sección “Propietarios” en la interfaz de propietarios, utilizando los elementos: Propietarios.

2 — I_Propietarios → C_Propietario: **2. enviarConsultarPropietario(propietarios)**

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra la lista de propietarios registrados (Pasos 3–4)

3 — C_Propietario → I_Propietarios: **3. mostrarListaPropietarios()**

El control de propietarios ordena a la interfaz de propietarios mostrar la lista de propietarios registrados.

4 — I_Propietarios → AS_Usuario: **4. mostrarConsultarPropietario()**

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona un propietario (Pasos 5–6)

5 — AS_Usuario → I_Propietarios: **5. seleccionarPropietario()**

El usuario selecciona un propietario en la interfaz de propietarios.

6

6. enviarConsultarPropietario(solicitud) ——►

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema obtiene la información registrada (Pasos 7–9)

7

7. obtenerInformacionRegistrada(criterio) ——►

El control de propietarios solicita a la entidad de propietarios (base de datos) la información necesaria para obtener la información registrada.

8

◄ - - 8. datosConsultarPropietario(resultado)

La entidad de propietarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

9

9. obtenerInformacionRegistradaResultado() ↺

El control de propietarios procesa internamente los datos recibidos y obtiene la información registrada, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema muestra los datos del propietario, como nombres, apellidos, dni, teléfono, correo electrónico, dirección y fecha de registro (Pasos 10–11)

10

◄—— 10. mostrarDatosPropietario()

El control de propietarios ordena a la interfaz de propietarios mostrar los datos del propietario, como nombres, apellidos, dni, teléfono, correo electrónico, dirección y fecha de registro.

11

11. vistaDesplegada() - -►

La interfaz de propietarios confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 6 · El sistema muestra las mascotas asociadas al propietario (Pasos 12–13)

12 ◄—— 12. mostrarMascotasAsociadas()

El control de propietarios ordena a la interfaz de propietarios mostrar las mascotas asociadas al propietario.

13 ◄ - - 13. mostrarConsultarPropietario()

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario puede seleccionar una mascota para consultar su información (Pasos 14–17)

14 14. seleccionarMascotaConsultar() ——►

El usuario puede seleccionar una mascota para consultar su información en la interfaz de propietarios.

15 15. enviarConsultarPropietario(solicitud) ——►

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

16 ◄—— 16. mostrarResultadoConsultarPropietario()

El control de propietarios ordena a la interfaz de propietarios mostrar el resultado final de la operación.

17 ◄ - - 17. resultadoMostrado()

La interfaz de propietarios presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF014-CONSULTAR MASCOTAS DE PROPIETARIO | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Propietarios	C_Propietario	E_Mascota

BLOQUE 1 · El usuario selecciona la sección "Propietarios" (Pasos 1–2)

1. **1. seleccionarSeccionPropietarios(propietarios)** ——►
   El usuario selecciona la sección "Propietarios" en la interfaz de propietarios, utilizando los elementos: Propietarios.
2. **2. enviarConsultarMascotas(propietarios)** ——►
   La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra los propietarios registrados (Pasos 3–4)

3. ◄—— **3. mostrarPropietariosRegistrados()**
   El control de propietarios ordena a la interfaz de propietarios mostrar los propietarios registrados.
4. ◄ - - **4. mostrarConsultarMascotas()**
   La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona un propietario (Pasos 5–6)

5. **5. seleccionarPropietario()** ——►

El usuario selecciona un propietario en la interfaz de propietarios.

6 **6. enviarConsultarMascotas(solicitud) ——►**

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema consulta las mascotas asociadas (Pasos 7–9)

7 **7. consultarMascotasAsociadas(criterio) ——►**

El control de propietarios solicita a la entidad de mascotas (base de datos) la información necesaria para consultar las mascotas asociadas.

8 **◄ - - 8. datosConsultarMascotas(resultado)**

La entidad de mascotas (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

9 **9. consultarMascotasAsociadasResultado() ↺**

El control de propietarios procesa internamente los datos recibidos y consulta las mascotas asociadas, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema muestra las mascotas asociadas con sus datos principales (Pasos 10–11)

10 **◄—— 10. mostrarMascotasAsociadas()**

El control de propietarios ordena a la interfaz de propietarios mostrar las mascotas asociadas con sus datos principales.

11 **◄ - - 11. mostrarConsultarMascotas()**

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario selecciona “Ver mascota” (Pasos 12–13)

12 **12. seleccionarVerMascota(verMascota) ——►**

El usuario selecciona “Ver mascota” en la interfaz de propietarios, utilizando los elementos: Ver mascota.

13 **13. enviarConsultarMascotas(verMascota) —— ►**

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema muestra la información de la mascota seleccionada (Pasos 14–15)

14 **◄—— 14. mostrarInformacionMascota()**

El control de propietarios ordena a la interfaz de propietarios mostrar la información de la mascota seleccionada.

15 **◄ - - 15. mostrarConsultarMascotas()**

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF015-MODIFICAR PROPIETARIO | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Propietarios	C_Propietario	E_Propietario

BLOQUE 1 · El usuario selecciona la sección "Propietarios" (Pasos 1–2)

1 — **1. seleccionarSeccionPropietarios(propietarios) ——►**

El usuario selecciona la sección "Propietarios" en la interfaz de propietarios, utilizando los elementos: Propietarios.

2 — **2. enviarModificarPropietario(propietarios) ——►**

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra los propietarios registrados (Pasos 3–4)

3 — **◄—— 3. mostrarPropietariosRegistrados()**

El control de propietarios ordena a la interfaz de propietarios mostrar los propietarios registrados.

4 — **◄ - - 4. mostrarModificarPropietario()**

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona el propietario que desea modificar (Pasos 5–6)

5 — **5. seleccionarPropietarioDesea() ——►**

El usuario selecciona el propietario que desea modificar en la interfaz de propietarios.

6

6. enviarModificarPropietario(solicitud) ——►

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la información registrada del propietario (Pasos 7–8)

7

◄—— 7. mostrarInformacionRegistrada()

El control de propietarios ordena a la interfaz de propietarios mostrar la información registrada del propietario.

8

◄ - - 8. mostrarModificarPropietario()

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona “Editar” (Pasos 9–10)

9

9. seleccionarEditar(editar) ——►

El usuario selecciona “Editar” en la interfaz de propietarios, utilizando los elementos: Editar.

10

10. enviarModificarPropietario(editar) ——►

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema muestra el formulario con los datos actuales (Pasos 11–12)

11

◄—— 11. mostrarFormularioDatos()

El control de propietarios ordena a la interfaz de propietarios mostrar el formulario con los datos actuales.

12 ◄ - - 12. mostrarModificarPropietario()

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario modifica los datos y selecciona “Guardar cambios” (Pasos 13–14)

13 **13. modificarDatosSelecciona(guardarCambios)** ——►

El usuario modifica los datos y selecciona “Guardar cambios” en la interfaz de propietarios, utilizando los elementos: Guardar cambios.

14 **14. enviarModificarPropietario(guardarCambios)** ——►

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema valida la información y muestra la confirmación de los cambios (Pasos 15–19)

15 **15. validarInformacionMuestra(criterio)** ——►

El control de propietarios solicita a la entidad de propietarios (base de datos) la información necesaria para validar la información y muestra la confirmación de los cambios.

16 ◄ - - **16. datosModificarPropietario(resultado)**

La entidad de propietarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

17 **17. validarInformacionMuestraResultado()** ↺

El control de propietarios procesa internamente los datos recibidos y valida la información y muestra la confirmación de los cambios, determinando el resultado de esta etapa.

18

◄—— 18. mostrarResultadoModificarPropietario()

El control de propietarios ordena a la interfaz de propietarios mostrar el resultado de la operación realizada.

19

◄ - - 19. mostrarModificarPropietario()

La interfaz de propietarios presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 9 · El usuario confirma y el sistema actualiza los datos del propietario (Pasos 20–25)

20

20. confirmarOperacion() ——►

El usuario confirma en la interfaz de propietarios.

21

21. enviarModificarPropietario(solicitud) ——►

La interfaz de propietarios recibe la acción del usuario y la traslada al control de propietarios, responsable de ejecutar la lógica del requerimiento.

22

22. actualizarDatosPropietario(datos) ——►

El control de propietarios solicita a la entidad de propietarios (base de datos) actualizar los datos del propietario.

23

◄ - - 23. operacionConfirmada(estado)

La entidad de propietarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

24

◄—— 24. mostrarResultadoModificarPropietario()

El control de propietarios ordena a la interfaz de propietarios mostrar el resultado final de la operación.

◄ - - 25. resultadoMostrado()

La interfaz de propietarios presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF016-BUSCAR MASCOTA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Mascotas	C_Mascota	E_Mascota

BLOQUE 1 · El usuario ingresa a la sección "Mascotas" (Pasos 1–2)

1 — AS_Usuario → I_Mascotas: **1. ingresarSeccionMascotas(mascotas) ——►**

El usuario ingresa a la sección "Mascotas" en la interfaz de mascotas, utilizando los elementos: Mascotas.

2 — I_Mascotas → C_Mascota: **2. enviarBuscarMascota(mascotas) ——►**

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las mascotas registradas y la barra de búsqueda (Pasos 3–4)

3 — C_Mascota → I_Mascotas: **◄—— 3. mostrarMascotasRegistradas()**

El control de mascotas ordena a la interfaz de mascotas mostrar las mascotas registradas y la barra de búsqueda.

4 — I_Mascotas → AS_Usuario: **◄ - - 4. mostrarBuscarMascota()**

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario ingresa un dato, como nombre, especie, raza o código de identificación (Pasos 5–6)

5 — AS_Usuario → I_Mascotas: **5. ingresarDatoNombre() ——►**

El usuario ingresa un dato, como nombre, especie, raza o código de identificación en la interfaz de mascotas.

6 **◄ - - 6. camposCompletados()**

La interfaz de mascotas refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 4 · Selecciona “Buscar” (Pasos 7–8)

7 **7. seleccionarBuscar(buscar) ——►**

El usuario selecciona “Buscar” en la interfaz de mascotas, utilizando los elementos: Buscar.

8 **8. enviarBuscarMascota(buscar) ——►**

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema compara el dato con los registros y muestra las mascotas coincidentes (Pasos 9–13)

9 **9. compararDatoRegistros(criterio) ——►**

El control de mascotas solicita a la entidad de mascotas (base de datos) la información necesaria para comparar el dato con los registros y muestra las mascotas coincidentes.

10 **◄ - - 10. datosBuscarMascota(resultado)**

La entidad de mascotas (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

11 **11. compararDatoRegistrosResultado() ↺**

El control de mascotas procesa internamente los datos recibidos y compara el dato con los registros y muestra las mascotas coincidentes, determinando el resultado de esta etapa.

12 ◄—— 12. mostrarResultadoBuscarMascota()

El control de mascotas ordena a la interfaz de mascotas mostrar el resultado de la operación realizada.

13 ◄ - - 13. mostrarBuscarMascota()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario selecciona una mascota (Pasos 14–15)

14 **14. seleccionarMascota() ——►**

El usuario selecciona una mascota en la interfaz de mascotas.

15 **15. enviarBuscarMascota(solicitud) ——►**

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema muestra la información registrada de la mascota (Pasos 16–17)

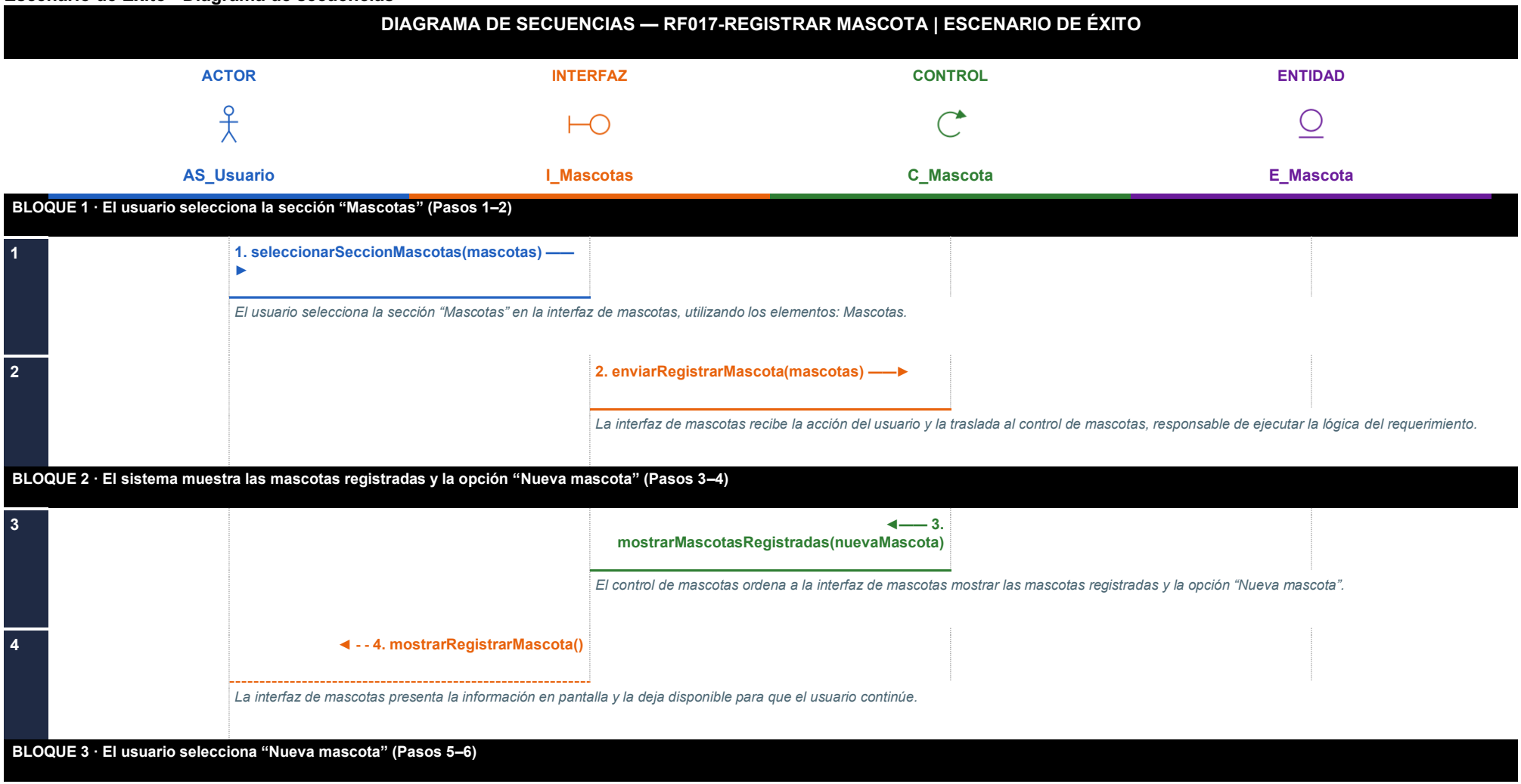
16 ◄—— 16. mostrarInformacionRegistrada()

El control de mascotas ordena a la interfaz de mascotas mostrar la información registrada de la mascota.

17 ◄ - - 17. mostrarBuscarMascota()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias



5

5. seleccionarNuevaMascota(nuevaMascota) ——►

El usuario selecciona “Nueva mascota” en la interfaz de mascotas, utilizando los elementos: Nueva mascota.

6

6. enviarRegistrarMascota(nuevaMascota) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra el formulario para registrar la mascota, con los campos nombre, especie, raza, sexo, fecha de nacimiento, color, estado_esterilización, propietario y observaciones (Pasos 7–8)

7

◄—— 7. mostrarFormularioRegistrar()

El control de mascotas ordena a la interfaz de mascotas mostrar el formulario para registrar la mascota, con los campos nombre, especie, raza, sexo, fecha de nacimiento, color, estado_esterilización, propietario y observaciones.

8

◄ - - 8. mostrarRegistrarMascota()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario ingresa los datos de la mascota y selecciona un propietario registrado de la lista disponible (Pasos 9–11)

9

9. ingresarDatosMascota() ——►

El usuario ingresa los datos de la mascota y selecciona un propietario registrado de la lista disponible en la interfaz de mascotas.

10

◄ - - 10. camposCompletados()

La interfaz de mascotas refleja en pantalla los datos ingresados para que el usuario pueda continuar.

11

11. enviarRegistrarMascota(solicitud) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El usuario selecciona “Registrar mascota” (Pasos 12–13)

12

12. seleccionarRegistrarMascota(registrarMascota) ——►

El usuario selecciona “Registrar mascota” en la interfaz de mascotas, utilizando los elementos: Registrar mascota.

13

13. enviarRegistrarMascota(registrarMascota) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema valida que los datos ingresados sean correctos y que se haya seleccionado un propietario (Pasos 14–16)

14

14. validarDatosIngresados(criterio) ——►

El control de mascotas solicita a la entidad de mascotas (base de datos) la información necesaria para validar que los datos ingresados sean correctos y que se haya seleccionado un propietario.

15

◄ - - 15. datosRegistrarMascota(resultado)

La entidad de mascotas (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

16

16. validarDatosIngresadosResultado() ↺

El control de mascotas procesa internamente los datos recibidos y valida que los datos ingresados sean correctos y que se haya seleccionado un propietario, determinando el resultado de esta etapa.

BLOQUE 8 · El sistema muestra una pantalla de confirmación con los datos ingresados de la mascota y el propietario seleccionado (Pasos 17–18)

17

◄—— 17. mostrarPantallaConfirmacion()

El control de mascotas ordena a la interfaz de mascotas mostrar una pantalla de confirmación con los datos ingresados de la mascota y el propietario seleccionado.

18

◄ - - 18. mostrarRegistrarMascota()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 9 · El usuario revisa la información y selecciona “Sí, registrar” (Pasos 19–20)

19

19. revisarInformacionSelecciona(registrar) —— ►

El usuario revisa la información y selecciona “Sí, registrar” en la interfaz de mascotas, utilizando los elementos: Sí, registrar.

20

20. enviarRegistrarMascota(registrar) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 10 · El sistema muestra mensaje de "Confirmar nuevo registro" y registra la mascota con los datos ingresados, la asocia al propietario seleccionado y muestra la ficha de la mascota. 9. El sistema muestra la información registrada, incluyendo el número de historia clínica asignado (Pasos 21–22)

21

◄—— 21. mostrarMensajeConfirmar(confirmarNuevo)

El control de mascotas ordena a la interfaz de mascotas mostrar mensaje de "Confirmar nuevo registro" y registra la mascota con los datos ingresados, la asocia al propietario seleccionado y muestra la ficha de la mascota. 9. El sistema muestra la información registrada, incluyendo el número de historia clínica asignado.

22

◄ - - 22. mostrarRegistrarMascota()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF018-MODIFICAR MASCOTA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
			
AS_Usuario	I_Mascotas	C_Mascota	E_Mascota

BLOQUE 1 · El usuario ingresa a la sección “Mascotas” (Pasos 1–4)

1 — **1. ingresarSeccionMascotas(mascotas) ——►** (ACTOR → INTERFAZ)

El usuario ingresa a la sección “Mascotas” en la interfaz de mascotas, utilizando los elementos: Mascotas.

2 — **2. enviarModificarMascota(mascotas) ——►** (INTERFAZ → CONTROL)

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

3 — **◄ - - 3. solicitudRecibida()** (CONTROL → INTERFAZ)

El control de mascotas registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

4 — **◄ - - 4. mostrarSiguientePaso()** (INTERFAZ → ACTOR)

La interfaz de mascotas habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 2 · El usuario selecciona una mascota (Pasos 5–6)

5 — **5. seleccionarMascota() ——►** (ACTOR → INTERFAZ)

El usuario selecciona una mascota en la interfaz de mascotas.

6

6. enviarModificarMascota(solicitud) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra la ficha de la mascota (Pasos 7–8)

7

◄—— 7. mostrarFichaMascota()

El control de mascotas ordena a la interfaz de mascotas mostrar la ficha de la mascota.

8

◄ - - 8. mostrarModificarMascota()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona “Editar” (Pasos 9–10)

9

9. seleccionarEditar(editar) ——►

El usuario selecciona “Editar” en la interfaz de mascotas, utilizando los elementos: Editar.

10

10. enviarModificarMascota(editar) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema muestra el formulario con la información registrada (Pasos 11–12)

11

◄—— 11. mostrarFormularioInformacion()

El control de mascotas ordena a la interfaz de mascotas mostrar el formulario con la información registrada.

12

◄ - - 12. mostrarModificarMascota()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario modifica los datos necesarios (Pasos 13–14)

13 **13. modificarDatosNecesarios() ——►**

El usuario modifica los datos necesarios en la interfaz de mascotas.

14 **◄ - - 14. camposCompletados()**

La interfaz de mascotas refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 7 · El usuario selecciona “Guardar cambios” (Pasos 15–16)

15 **15. seleccionarGuardarCambios(guardarCambios) ——►**

El usuario selecciona “Guardar cambios” en la interfaz de mascotas, utilizando los elementos: Guardar cambios.

16 **16. enviarModificarMascota(guardarCambios) ——►**

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema valida la información y muestra una pantalla de confirmación (Pasos 17–21)

17 **17. validarInformacionMuestra(criterio) ——►**

El control de mascotas solicita a la entidad de mascotas (base de datos) la información necesaria para validar la información y muestra una pantalla de confirmación.

18 **◄ - - 18. datosModificarMascota(resultado)**

La entidad de mascotas (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

19 **19. validarInformacionMuestraResultado() ↺**

El control de mascotas procesa internamente los datos recibidos y valida la información y muestra una pantalla de confirmación, determinando el resultado de esta etapa.

20 **◄—— 20. mostrarResultadoModificarMascota()**

El control de mascotas ordena a la interfaz de mascotas mostrar el resultado de la operación realizada.

21 **◄ - - 21. mostrarModificarMascota()**

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 9 · El usuario confirma los cambio y el sistema actualiza la información de la mascota (Pasos 22–27)

22 **22. confirmarOperacion() ——►**

El usuario confirma los cambio en la interfaz de mascotas.

23 **23. enviarModificarMascota(solicitud) ——►**

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

24 **24. actualizarInformacionMascota(datos) ——►**

El control de mascotas solicita a la entidad de mascotas (base de datos) actualizar la información de la mascota.

25 **◄ - - 25. operacionConfirmada(estado)**

La entidad de mascotas (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

26 **◄—— 26. mostrarResultadoModificarMascota()**

El control de mascotas ordena a la interfaz de mascotas mostrar el resultado final de la operación.

27 **◄ - - 27. resultadoMostrado()**

La interfaz de mascotas presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF019-CONSULTAR INFORMACIÓN DE MASCOTA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL
AS_Usuario	I_Mascotas	C_Mascota

BLOQUE 1 · El sistema muestra las mascotas registradas (Pasos 1–2)

1			◄—— 1. mostrarMascotasRegistradas()
			El control de mascotas ordena a la interfaz de mascotas mostrar las mascotas registradas.
2		◄ - - 2. mostrarConsultarInformacion()	
		La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.	

BLOQUE 2 · El usuario busca y selecciona la mascota que desea consultar (Pasos 3–4)

3	3. buscarSeleccionaMascota() ——►		
	El usuario busca y selecciona la mascota que desea consultar en la interfaz de mascotas.		
4			4. enviarConsultarInformacion(solicitud) ——►
			La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra la información registrada de la mascota (Pasos 5–6)

5			◄—— 5. mostrarInformacionRegistrada()

		El control de mascotas ordena a la interfaz de mascotas mostrar la información registrada de la mascota.
6		**6. vistaDesplegada() - -►**
		La interfaz de mascotas confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.
BLOQUE 4 · El sistema muestra sus datos básicos como número de historia clínica, nombre, especie, raza, sexo, fecha de nacimiento, color, estado_esterilización, propietario y observaciones (Pasos 7–8)		
7		**◄—— 7. mostrarDatosBasicos()**
		El control de mascotas ordena a la interfaz de mascotas mostrar sus datos básicos como número de historia clínica, nombre, especie, raza, sexo, fecha de nacimiento, color, estado_esterilización, propietario y observaciones.
8		**8. vistaDesplegada() - -►**
		La interfaz de mascotas confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.
BLOQUE 5 · El sistema muestra el propietario asociado a la mascota (Pasos 9–10)		
9		**◄—— 9. mostrarPropietarioAsociado()**
		El control de mascotas ordena a la interfaz de mascotas mostrar el propietario asociado a la mascota.
10		**10. vistaDesplegada() - -►**
		La interfaz de mascotas confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.
BLOQUE 6 · El sistema muestra las opciones Editar, Desactivar e Historial de atención (Pasos 11–12)		
11		**◄—— 11. mostrarOpcionesEditar()**

El control de mascotas ordena a la interfaz de mascotas mostrar las opciones Editar, Desactivar e Historial de atención.

12 ◄ - - 12. mostrarConsultarInformacion()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario selecciona una de las opciones disponibles (Pasos 13–14)

13 **13. seleccionarOpcionesDisponibles() ——►**

El usuario selecciona una de las opciones disponibles en la interfaz de mascotas.

14 **14. enviarConsultarInformacion(solicitud) ——►**

La interfaz de mascotas recibe la acción del usuario y la traslada al control de mascotas, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema dirige al usuario a la función seleccionada (Pasos 15–16)

15 **◄—— 15. dirigirUsuarioFuncion()**

El control de mascotas ordena a la interfaz de mascotas dirigir al usuario a la función seleccionada.

16 ◄ - - 16. mostrarConsultarInformacion()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF020-CONSULTAR HISTORIAL DE ATENCIÓN DE UNA MASCOTA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
			
AS_Usuario	I_Mascotas	C_Atencion	E_Atencion

BLOQUE 1 · El usuario ingresa a la sección Mascotas (Pasos 1–4)

1 — **1. ingresarSeccionMascotas() ——►**

El usuario ingresa a la sección Mascotas en la interfaz de mascotas.

2 — **2. enviarConsultarHistorial(solicitud) ——►**

La interfaz de mascotas recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

3 — **◄ - - 3. solicitudRecibida()**

El control de atenciones registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

4 — **◄ - - 4. mostrarSiguientePaso()**

La interfaz de mascotas habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 2 · El usuario busca y selecciona la mascota que desea consultar (Pasos 5–6)

5 — **5. buscarSeleccionaMascota() ——►**

El usuario busca y selecciona la mascota que desea consultar en la interfaz de mascotas.

6

6. enviarConsultarHistorial(solicitud) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra la información general de la mascota, incluyendo su número de historia clínica (Pasos 7–8)

7

◄—— 7. mostrarInformacionGeneral()

El control de atenciones ordena a la interfaz de mascotas mostrar la información general de la mascota, incluyendo su número de historia clínica.

8

◄ - - 8. mostrarConsultarHistorial()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona la opción “Línea de tiempo” (Pasos 9–10)

9

9. seleccionarOpcionLinea(lineaTiempo) ——►

El usuario selecciona la opción “Línea de tiempo” en la interfaz de mascotas, utilizando los elementos: Línea de tiempo.

10

10. enviarConsultarHistorial(lineaTiempo) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema consulta las atenciones registradas para la mascota (Pasos 11–13)

11

11. consultarAtencionesRegistradas(criterio) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para consultar las atenciones registradas para la mascota.

12

◄ - - 12. datosConsultarHistorial(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

13

13. consultarAtencionesRegistradasResultado() ↺

El control de atenciones procesa internamente los datos recibidos y consulta las atenciones registradas para la mascota, determinando el resultado de esta etapa.

BLOQUE 6 · El sistema muestra las atenciones ordenadas cronológicamente, indicando información como nombre de actividad, fech y estado de cada atención (Pasos 14–15)

14

◄—— 14. mostrarAtencionesOrdenadas()

El control de atenciones ordena a la interfaz de mascotas mostrar las atenciones ordenadas cronológicamente, indicando información como nombre de actividad, fech y estado de cada atención.

15

◄ - - 15. mostrarConsultarHistorial()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario selecciona una atención del historial (Pasos 16–17)

16

16. seleccionarAtencionHistorial() ——►

El usuario selecciona una atención del historial en la interfaz de mascotas.

17

17. enviarConsultarHistorial(solicitud) ——►

La interfaz de mascotas recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema muestra el detalle de la atención y las actividades realizadas durante ella (Pasos 18–19)

18

◄—— 18. mostrarDetalleAtencion()

El control de atenciones ordena a la interfaz de mascotas mostrar el detalle de la atención y las actividades realizadas durante ella.

19

◄ - - 19. mostrarConsultarHistorial()

La interfaz de mascotas presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF021-FILTRAR ATENCIONES | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Atenciones	C_Atencion	E_Atencion

BLOQUE 1 · El usuario se encuentra en la sección “Atenciones” (Pasos 1–2)

1 — **1. seerEncuentraSeccion(atenciones) ——►** (AS_Usuario → I_Atenciones)

El usuario se encuentra en la sección “Atenciones” en la interfaz de atenciones.

2 — **2. enviarFiltrarAtenciones(atenciones) ——►** (I_Atenciones → C_Atencion)

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra el listado de atenciones y el filtro “Estado” (Pasos 3–4)

3 — **◄—— 3. mostrarListadoAtenciones(estado)** (C_Atencion → I_Atenciones)

El control de atenciones ordena a la interfaz de atenciones mostrar el listado de atenciones y el filtro “Estado”.

4 — **4. vistaDesplegada() - -►** (I_Atenciones → C_Atencion)

La interfaz de atenciones confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 3 · El sistema muestra las opciones “Todos”, “Activas” y “Finalizadas” (Pasos 5–6)

5 — **◄—— 5. mostrarOpcionesActivas()** (C_Atencion → I_Atenciones)

El control de atenciones ordena a la interfaz de atenciones mostrar las opciones “Todos”, “Activas” y “Finalizadas”.

6 ◄ - - 6. mostrarFiltrarAtenciones()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona el estado que desea consultar (Pasos 7–8)

7 **7. seleccionarEstadoDesea() ——►**

El usuario selecciona el estado que desea consultar en la interfaz de atenciones.

8 **8. enviarFiltrarAtenciones(solicitud) ——►**

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema aplica el filtro seleccionado (Pasos 9–11)

9 **9. aplicarFiltroSeleccionado(criterio) ——►**

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para aplicar el filtro seleccionado.

10 ◄ - - 10. datosFiltrarAtenciones(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

11 **11. aplicarFiltroSeleccionadoResultado() ↺**

El control de atenciones procesa internamente los datos recibidos y aplica el filtro seleccionado, determinando el resultado de esta etapa.

BLOQUE 6 · El sistema muestra únicamente las atenciones que corresponden al estado seleccionado (Pasos 12–13)

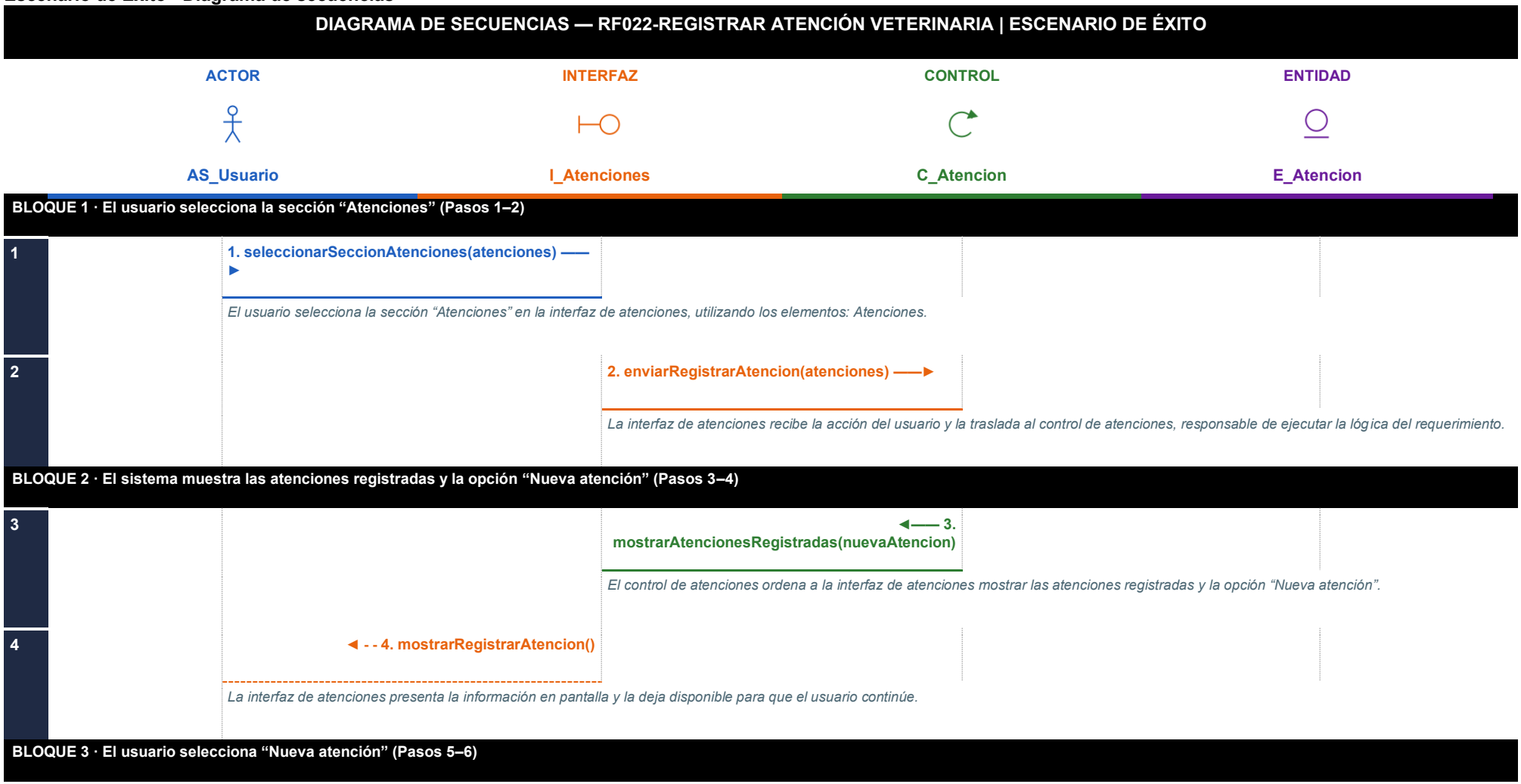
12 ◄—— **12. mostrarUnicamenteAtenciones()**

El control de atenciones ordena a la interfaz de atenciones mostrar únicamente las atenciones que corresponden al estado seleccionado.

13 ◄ - - **13. mostrarFiltrarAtenciones()**

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias



5

5. seleccionarNuevaAtencion(nuevaAtencion) ——►

El usuario selecciona “Nueva atención” en la interfaz de atenciones, utilizando los elementos: Nueva atención.

6

6. enviarRegistrarAtencion(nuevaAtencion) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra una barra de busqueda y permite seleccionar la mascota asociada (Pasos 7–8)

7

◄—— 7. mostrarBarraBusqueda()

El control de atenciones ordena a la interfaz de atenciones mostrar una barra de busqueda y permite seleccionar la mascota asociada.

8

◄ - - 8. mostrarRegistrarAtencion()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona la mascota y selecciona “Confirmar atención” (Pasos 9–10)

9

9. seleccionarMascotaSelecciona(confirmarAtencion) ——►

El usuario selecciona la mascota y selecciona “Confirmar atención” en la interfaz de atenciones, utilizando los elementos: Confirmar atención.

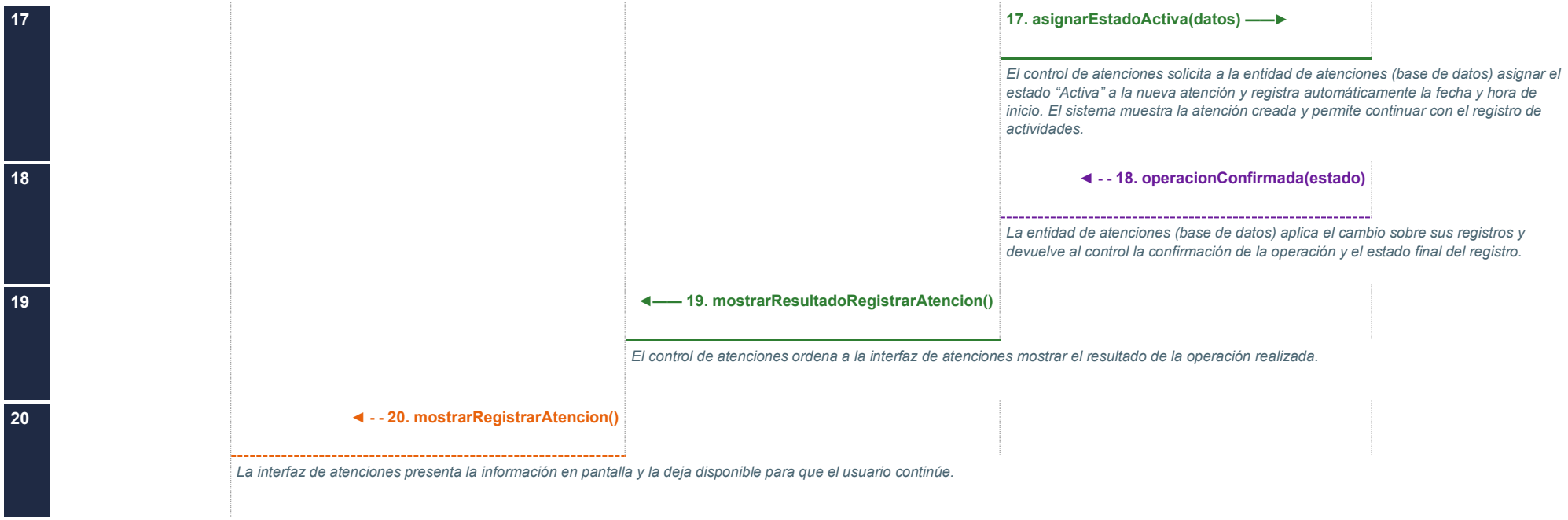
10

10. enviarRegistrarAtencion(confirmarAtencion) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

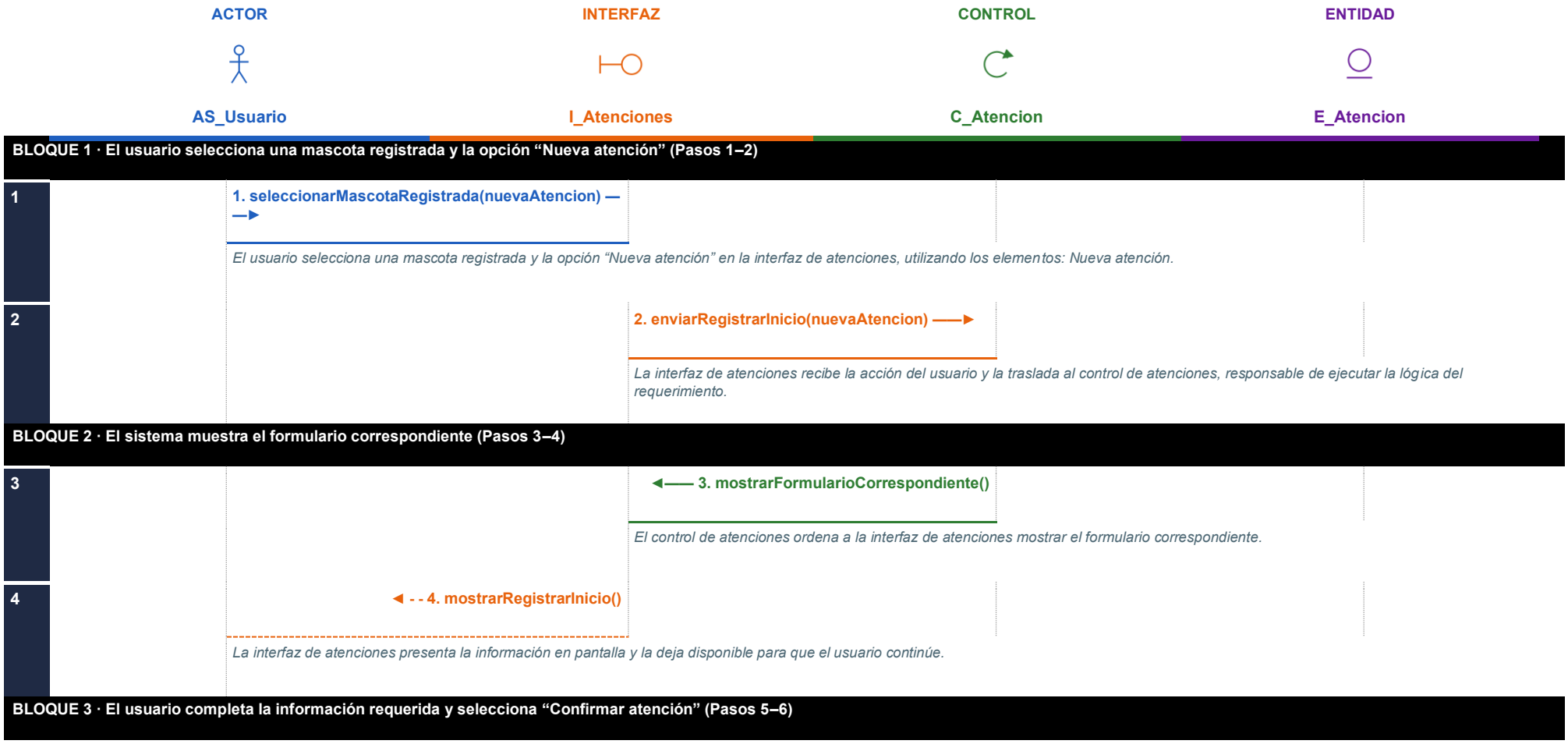
BLOQUE 6 · El sistema valida la información ingresada (Pasos 11–13)

11			**11. validarInformacionIngresada(criterio) ——►** *El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para validar la información ingresada.*
12			**◄ - - 12. datosRegistrarAtencion(resultado)** *La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.*
13			**13. validarInformacionIngresadaResultado() ↺** *El control de atenciones procesa internamente los datos recibidos y valida la información ingresada, determinando el resultado de esta etapa.*
BLOQUE 7 · El sistema valida la selección y registra la nueva atención asociada a la mascota (Pasos 14–16)			
14			**14. validarSeleccionRegistra(criterio) ——►** *El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para validar la selección y registra la nueva atención asociada a la mascota.*
15			**◄ - - 15. datosRegistrarAtencion(resultado)** *La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.*
16			**16. validarSeleccionRegistraResultado() ↺** *El control de atenciones procesa internamente los datos recibidos y valida la selección y registra la nueva atención asociada a la mascota, determinando el resultado de esta etapa.*
BLOQUE 8 · El sistema asigna el estado “Activa” a la nueva atención y registra automáticamente la fecha y hora de inicio. El sistema muestra la atención creada y permite continuar con el registro de actividades (Pasos 17–20)			


17
17. asignarEstadoActiva(datos) ——►
El control de atenciones solicita a la entidad de atenciones (base de datos) asignar el estado “Activa” a la nueva atención y registra automáticamente la fecha y hora de inicio. El sistema muestra la atención creada y permite continuar con el registro de actividades.
18
◄ - - 18. operacionConfirmada(estado)
La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.
19
◄—— 19. mostrarResultadoRegistrarAtencion()
El control de atenciones ordena a la interfaz de atenciones mostrar el resultado de la operación realizada.
20
◄ - - 20. mostrarRegistrarAtencion()
La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF023-REGISTRAR INICIO DE ATENCIÓN | ESCENARIO DE ÉXITO



5

5. completarInformacionRequerida(confirmarAtencion) ——►

El usuario completa la información requerida y selecciona "Confirmar atención" en la interfaz de atenciones, utilizando los elementos: Confirmar atención.

6

6. enviarRegistrarInicio(confirmarAtencion) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema valida la información ingresada. 5.El sistema registra automáticamente la fecha y hora de inicio (Pasos 7–9)

7

7. validarInformacionIngresada(criterio) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para validar la información ingresada. 5.El sistema registra automáticamente la fecha y hora de inicio.

8

◄ - - 8. datosRegistrarInicio(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

9

9. validarInformacionIngresadaResultado() ↺

El control de atenciones procesa internamente los datos recibidos y valida la información ingresada. 5.El sistema registra automáticamente la fecha y hora de inicio, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema muestra la atención creada junto con su fecha y hora de inicio (Pasos 10–11)

10

◄—— 10. mostrarAtencionCreada()

El control de atenciones ordena a la interfaz de atenciones mostrar la atención creada junto con su fecha y hora de inicio.

11

◄ - - 11. mostrarRegistrarInicio()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF024-CONSULTAR DETALLE DE ATENCIÓN VETERINARIA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Atenciones	C_Atencion	E_Atencion

BLOQUE 1 · El usuario ingresa a la sección "Atenciones" (Pasos 1–2)

1 — AS_Usuario → I_Atenciones: **1. ingresarSeccionAtenciones(atenciones) ——►**

El usuario ingresa a la sección "Atenciones" en la interfaz de atenciones, utilizando los elementos: Atenciones.

2 — I_Atenciones → C_Atencion: **2. enviarConsultarDetalle(atenciones) ——►**

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las atenciones registradas (Pasos 3–4)

3 — C_Atencion → I_Atenciones: **◄—— 3. mostrarAtencionesRegistradas()**

El control de atenciones ordena a la interfaz de atenciones mostrar las atenciones registradas.

4 — I_Atenciones → AS_Usuario: **◄ - - 4. mostrarConsultarDetalle()**

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona una atención (Pasos 5–6)

5 — AS_Usuario → I_Atenciones: **5. seleccionarAtencion() ——►**

El usuario selecciona una atención en la interfaz de atenciones.

6

6. enviarConsultarDetalle(solicitud) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema consulta la información correspondiente (Pasos 7–9)

7

7. consultarInformacionCorrespondiente(criterio) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para consultar la información correspondiente.

8

◄ - - 8. datosConsultarDetalle(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

9

9. consultarInformacionCorrespondienteResultado() ↺

El control de atenciones procesa internamente los datos recibidos y consulta la información correspondiente, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema muestra los datos de la mascota asociada (Pasos 10–11)

10

◄—— 10. mostrarDatosMascota()

El control de atenciones ordena a la interfaz de atenciones mostrar los datos de la mascota asociada.

11

11. vistaDesplegada() - -►

La interfaz de atenciones confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 6 · El sistema muestra el estado de la atención, fecha y hora de inicio y, cuando corresponda, fecha y hora de finalización (Pasos 12–13)

12 **◄—— 12. mostrarEstadoAtencion()**

El control de atenciones ordena a la interfaz de atenciones mostrar el estado de la atención, fecha y hora de inicio y, cuando corresponda, fecha y hora de finalización.

13 **13. vistaDesplegada() - -►**

La interfaz de atenciones confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 7 · El sistema muestra el progreso y el listado de actividades asociadas (Pasos 14–15)

14 **◄—— 14. mostrarProgresoListado()**

El control de atenciones ordena a la interfaz de atenciones mostrar el progreso y el listado de actividades asociadas.

15 **◄ - - 15. mostrarConsultarDetalle()**

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · El usuario puede consultar la información mostrada (Pasos 16–19)

16 **16. consultarInformacionMostrada() ——►**

El usuario puede consultar la información mostrada en la interfaz de atenciones.

17 **17. enviarConsultarDetalle(solicitud) ——►**

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

18 ◄—— 18. mostrarResultadoConsultarDetalle()

El control de atenciones ordena a la interfaz de atenciones mostrar el resultado final de la operación.

19 ◄ - - 19. resultadoMostrado()

La interfaz de atenciones presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF025-IDENTIFICAR ESTADO DE ATENCIÓN | ESCENARIO DE ÉXITO



El usuario identifica la atención que desea consultar en la interfaz de atenciones.

6 **6. enviarIdentificarEstado(solicitud) ——►**

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

7 **◄ - - 7. solicitudRecibida()**

El control de atenciones registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

8 **◄ - - 8. mostrarSiguientePaso()**

La interfaz de atenciones habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 4 · El usuario selecciona la atención correspondiente (Pasos 9–10)

9 **9. seleccionarAtencionCorrespondiente() ——►**

El usuario selecciona la atención correspondiente en la interfaz de atenciones.

10 **10. enviarIdentificarEstado(solicitud) ——►**

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema consulta la información registrada de la atención seleccionada (Pasos 11–13)

11 **11. consultarInformacionRegistrada(criterio) ——►**

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para consultar la información registrada de la atención seleccionada.

12

◄ - - 12. datosIdentificarEstado(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

13

13. consultarInformacionRegistradaResultado() ↺

El control de atenciones procesa internamente los datos recibidos y consulta la información registrada de la atención seleccionada, determinando el resultado de esta etapa.

BLOQUE 6 · El sistema verifica el estado actual de la atención (Pasos 14–16)

14

14. verificarEstadoActual(criterio) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para verificar el estado actual de la atención.

15

◄ - - 15. datosIdentificarEstado(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

16

16. verificarEstadoActualResultado() ↺

El control de atenciones procesa internamente los datos recibidos y verifica el estado actual de la atención, determinando el resultado de esta etapa.

BLOQUE 7 · El sistema identifica si la atención se encuentra en estado “Activa” o “Finalizada” (Pasos 17–19)

17

17. identificarAtencionEncuentra(criterio) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para identificar si la atención se encuentra en estado “Activa” o “Finalizada”.

18

◄ - - 18. datosIdentificarEstado(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

19

19. identificarAtencionEncuentraResultado() ↺

El control de atenciones procesa internamente los datos recibidos y identifica si la atención se encuentra en estado “Activa” o “Finalizada”, determinando el resultado de esta etapa.

BLOQUE 8 · El sistema muestra el estado de la atención de forma visible en el detalle de la atención (Pasos 20–21)

20

◄—— 20. mostrarEstadoAtencion()

El control de atenciones ordena a la interfaz de atenciones mostrar el estado de la atención de forma visible en el detalle de la atención.

21

◄ - - 21. mostrarIdentificarEstado()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF026-IDENTIFICAR ATENCIÓN LISTA PARA FINALIZAR | ESCENARIO DE ÉXITO



BLOQUE 1 · El usuario consulta una atención activa (Pasos 1–2)

1	1. consultarAtencionActiva() ——►			
	El usuario consulta una atención activa en la interfaz de atenciones.			
2		2. enviarIdentificarAtencion(solicitud) ——►		
		La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.		

BLOQUE 2 · El sistema consulta los estados de las actividades asociadas (Pasos 3–5)

3			3. consultarEstadosActividades(criterio) ——►	
			El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para consultar los estados de las actividades asociadas.	
4			◄ - - 4. datosIdentificarAtencion(resultado)	
			La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.	
5			5. consultarEstadosActividadesResultado() ↺	

El control de atenciones procesa internamente los datos recibidos y consulta los estados de las actividades asociadas, determinando el resultado de esta etapa.

BLOQUE 3 · El sistema verifica si existen actividades en estado “Pendiente” (Pasos 6–8)

6

6. verificarExistenActividades(criterio) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para verificar si existen actividades en estado “Pendiente”.

7

◄ - - 7. datosIdentificarAtencion(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

8

8. verificarExistenActividadesResultado() ↺

El control de atenciones procesa internamente los datos recibidos y verifica si existen actividades en estado “Pendiente”, determinando el resultado de esta etapa.

BLOQUE 4 · Si no existen actividades pendientes, el sistema identifica que todas están “Completadas” o “No realizadas” (Pasos 9–11)

9

9. identificarEstanCompletadas(criterio) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para identificar que todas están “Completadas” o “No realizadas”.

10

◄ - - 10. datosIdentificarAtencion(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

11

11. identificarEstanCompletadasResultado() ↺

El control de atenciones procesa internamente los datos recibidos y identifica que todas están “Completadas” o “No realizadas”, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema muestra un aviso indicando “La atención está lista para finalizar” (Pasos 12–13)

12 ◄—— 12. mostrarAvisoIndicando(atencionEsta)

El control de atenciones ordena a la interfaz de atenciones mostrar un aviso indicando “La atención está lista para finalizar”.

13 ◄ - - 13. mostrarIdentificarAtencion()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario puede continuar con la finalización de la atención (Pasos 14–17)

14 14. continuarFinalizacionAtencion() ——►

El usuario puede continuar con la finalización de la atención en la interfaz de atenciones.

15 15. enviarIdentificarAtencion(solicitud) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

16 ◄ - - 16. solicitudRecibida()

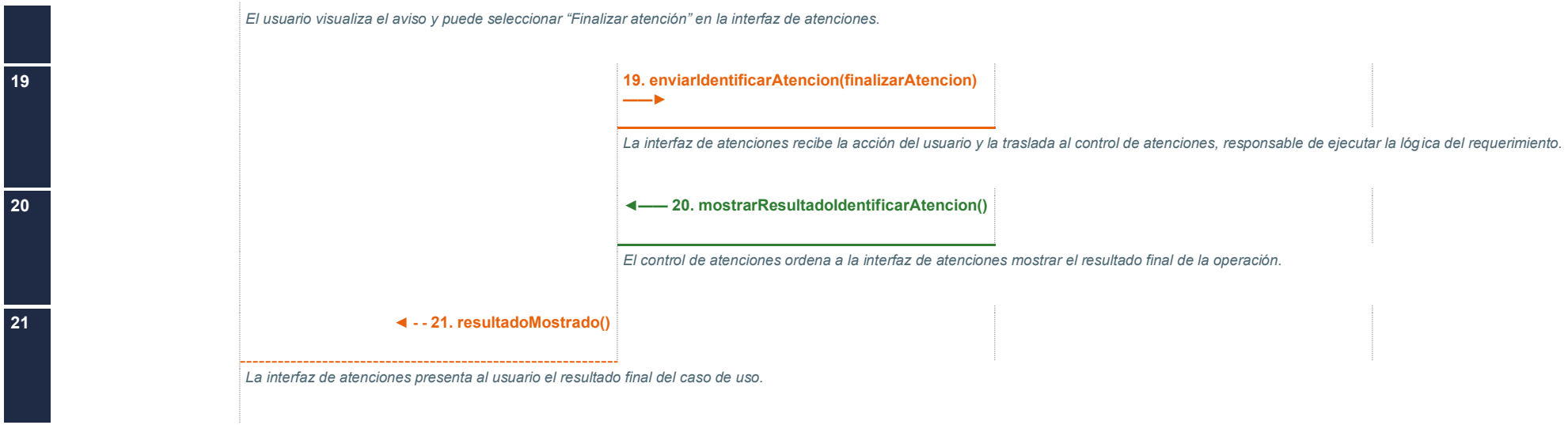
El control de atenciones registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

17 ◄ - - 17. mostrarSiguientePaso()

La interfaz de atenciones habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 7 · El usuario visualiza el aviso y puede seleccionar “Finalizar atención” (Pasos 18–21)

18 18. visualizarAvisoPuede(finalizarAtencion) ——►


El usuario visualiza el aviso y puede seleccionar “Finalizar atención” en la interfaz de atenciones.
19
19. enviarIdentificarAtencion(finalizarAtencion)
La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.
20
20. mostrarResultadoIdentificarAtencion()
El control de atenciones ordena a la interfaz de atenciones mostrar el resultado final de la operación.
21
21. resultadoMostrado()
La interfaz de atenciones presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF027-FINALIZAR ATENCIÓN VETERINARIA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Atenciones	C_Atencion	E_Atencion

BLOQUE 1 · El usuario ingresa al detalle de una atención activa (Pasos 1–2)

1 — **1. ingresarDetalleAtencion() ——►** (AS_Usuario → I_Atenciones)

El usuario ingresa al detalle de una atención activa en la interfaz de atenciones.

2 — **2. enviarFinalizarAtencion(solicitud) ——►** (I_Atenciones → C_Atencion)

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema verifica que todas las actividades se encuentren en estado “Completada” o “No realizada” (Pasos 3–5)

3 — **3. verificarActividadesEncuentren(criterio) ——►** (C_Atencion → E_Atencion)

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para verificar que todas las actividades se encuentren en estado “Completada” o “No realizada”.

4 — **◄ - - 4. datosFinalizarAtencion(resultado)** (E_Atencion → C_Atencion)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

5 — **5. verificarActividadesEncuentrenResultado() ↺** (C_Atencion)

El control de atenciones procesa internamente los datos recibidos y verifica que todas las actividades se encuentren en estado “Completada” o “No realizada”, determinando el resultado de esta etapa.

BLOQUE 3 · El sistema muestra la opción “Finalizar atención” (Pasos 6–7)

6

◄—— 6. mostrarOpcionFinalizar(finalizarAtencion)

El control de atenciones ordena a la interfaz de atenciones mostrar la opción “Finalizar atención”.

7

◄ - - 7. mostrarFinalizarAtencion()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona “Finalizar atención” (Pasos 8–9)

8

8. seleccionarFinalizarAtencion(finalizarAtencion) ——►

El usuario selecciona “Finalizar atención” en la interfaz de atenciones, utilizando los elementos: Finalizar atención.

9

9. enviarFinalizarAtencion(finalizarAtencion) —— ►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema solicita confirmación mediante “¿Está seguro de que desea finalizar esta atención?” (Pasos 10–11)

10

◄—— 10. solicitarConfirmacionEsta(estaSeguro)

El control de atenciones ordena a la interfaz de atenciones solicitar confirmación mediante “¿Está seguro de que desea finalizar esta atención?”.

11 ◄ - - 11. mostrarFinalizarAtencion()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario confirma la finalización (Pasos 12–13)

12 **12. confirmarOperacion() ——►**

El usuario confirma la finalización en la interfaz de atenciones.

13 **13. enviarFinalizarAtencion(solicitud) ——►**

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema cambia el estado de la atención a "Finalizada" (Pasos 14–15)

14 **14. cambiarEstadoAtencion(datos) ——►**

El control de atenciones solicita a la entidad de atenciones (base de datos) cambiar el estado de la atención a "Finalizada".

15 ◄ - - 15. operacionConfirmada(estado)

La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 8 · El sistema muestra "La atención veterinaria fue finalizada correctamente." (Pasos 16–17)

16 ◄—— 16. mostrarAtencionVeterinaria(atencionVeterinaria)

El control de atenciones ordena a la interfaz de atenciones mostrar "La atención veterinaria fue finalizada correctamente.".

17 17. vistaDesplegada() - -►

La interfaz de atenciones confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 9 · El sistema registra automáticamente la fecha y hora de finalización (Pasos 18–21)

18 18. registrarAutomaticamenteFecha(datos) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) registrar automáticamente la fecha y hora de finalización.

19 ◄ - - 19. operacionConfirmada(estado)

La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

20 ◄—— 20. mostrarResultadoFinalizarAtencion()

El control de atenciones ordena a la interfaz de atenciones mostrar el resultado final de la operación.

21 ◄ - - 21. resultadoMostrado()

La interfaz de atenciones presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF028-REGISTRAR FINALIZACIÓN DE ATENCIÓN | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Atenciones	C_Atencion	E_Atencion

BLOQUE 1 · El usuario confirma la finalización de la atención (Pasos 1–2)

1 — **1. confirmarOperacion() ——►** (AS_Usuario → I_Atenciones)

El usuario confirma la finalización de la atención en la interfaz de atenciones.

2 — **2. enviarRegistrarFinalizacion(solicitud) ——►** (I_Atenciones → C_Atencion)

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema valida que la atención pueda ser finalizada (Pasos 3–5)

3 — **3. validarAtencionPueda(criterio) ——►** (C_Atencion → E_Atencion)

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para validar que la atención pueda ser finalizada.

4 — **◄ - - 4. datosRegistrarFinalizacion(resultado)** (E_Atencion → C_Atencion)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

5 — **5. validarAtencionPuedaResultado() ↺** (C_Atencion)

			El control de atenciones procesa internamente los datos recibidos y valida que la atención pueda ser finalizada, determinando el resultado de esta etapa.
BLOQUE 3 · El sistema cambia el estado de la atención a “Finalizada” (Pasos 6–7)			
6			**6. cambiarEstadoAtencion(datos) ——►**
			El control de atenciones solicita a la entidad de atenciones (base de datos) cambiar el estado de la atención a “Finalizada”.
7			**◄ - - 7. operacionConfirmada(estado)**
			La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.
BLOQUE 4 · El sistema registra automáticamente la fecha y hora de finalización (Pasos 8–9)			
8			**8. registrarAutomaticamenteFecha(datos) ——►**
			El control de atenciones solicita a la entidad de atenciones (base de datos) registrar automáticamente la fecha y hora de finalización.
9			**◄ - - 9. operacionConfirmada(estado)**
			La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.
BLOQUE 5 · El sistema guarda la información actualizada (Pasos 10–11)			
10			**10. guardarInformacionActualizada(datos) —— ►**
			El control de atenciones solicita a la entidad de atenciones (base de datos) guardar la información actualizada.
11			**◄ - - 11. operacionConfirmada(estado)**

La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 6 · El sistema muestra la atención con su estado “Finalizada” (Pasos 12–13)

12 ◄—— **12. mostrarAtencionEstado(finalizada)**

El control de atenciones ordena a la interfaz de atenciones mostrar la atención con su estado “Finalizada”.

13 ◄ - - **13. mostrarRegistrarFinalizacion()**

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF029-REGISTRAR ACTIVIDAD VETERINARIA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_NuevaActividad	C_Actividad	E_Actividad

BLOQUE 1 · El usuario ingresa al módulo "Atenciones" (Pasos 1–2)

1 — **1. ingresarModuloAtenciones(atenciones) ——►** (AS_Usuario → I_NuevaActividad)

El usuario ingresa al módulo "Atenciones" en la interfaz de nueva actividad, utilizando los elementos: Atenciones.

2 — **2. enviarRegistrarActividad(atenciones) ——►** (I_NuevaActividad → C_Actividad)

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las atenciones veterinarias registradas y permite identificar las que se encuentran Activas (Pasos 3–4)

3 — **◄—— 3. mostrarAtencionesVeterinarias()** (C_Actividad → I_NuevaActividad)

El control de actividades ordena a la interfaz de nueva actividad mostrar las atenciones veterinarias registradas y permite identificar las que se encuentran Activas.

4 — **◄ - - 4. mostrarRegistrarActividad()** (I_NuevaActividad → AS_Usuario)

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona una atención veterinaria activa (Pasos 5–6)

5 — **5. seleccionarAtencionVeterinaria() ——►** (AS_Usuario → I_NuevaActividad)

El usuario selecciona una atención veterinaria activa en la interfaz de nueva actividad.

6 **6. enviarRegistrarActividad(solicitud) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra el detalle de la atención seleccionada, incluyendo la información de la mascota, estado de la atención, progreso y actividades registradas, junto con la opción “Nueva actividad” (Pasos 7–8)

7 **◄—— 7. mostrarDetalleAtencion(nuevaActividad)**

El control de actividades ordena a la interfaz de nueva actividad mostrar el detalle de la atención seleccionada, incluyendo la información de la mascota, estado de la atención, progreso y actividades registradas, junto con la opción “Nueva actividad”.

8 **◄ - - 8. mostrarRegistrarActividad()**

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona “Nueva actividad” (Pasos 9–10)

9 **9. seleccionarNuevaActividad(nuevaActividad) ——►**

El usuario selecciona “Nueva actividad” en la interfaz de nueva actividad, utilizando los elementos: Nueva actividad.

10 **10. enviarRegistrarActividad(nuevaActividad) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema muestra el formulario correspondiente al tipo de actividad seleccionado (Pasos 11–12)

11 **◄—— 11. mostrarFormularioCorrespondiente()**

El control de actividades ordena a la interfaz de nueva actividad mostrar el formulario correspondiente al tipo de actividad seleccionado.

12 ◄ - - 12. mostrarRegistrarActividad()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario completa la información requerida (Pasos 13–14)

13 **13. completarInformacionRequerida() ——►**

El usuario completa la información requerida en la interfaz de nueva actividad.

14 ◄ - - 14. camposCompletados()

La interfaz de nueva actividad refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 8 · El usuario selecciona “Registrar actividad”. El sistema valida los datos y registra la actividad asociada a la atención con estado inicial “Pendiente” (Pasos 15–21)

15 **15. seleccionarRegistrarActividad(registrarActividad) ——►**

El usuario selecciona “Registrar actividad” en la interfaz de nueva actividad, utilizando los elementos: Registrar actividad.

16 **16. enviarRegistrarActividad(registrarActividad) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

17 **17. validarDatosRegistra(criterio) ——►**

El control de actividades solicita a la entidad de actividades (base de datos) la información necesaria para validar los datos y registra la actividad asociada a la atención con estado inicial “Pendiente”.

18 ◄ - - 18. datosRegistrarActividad(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

19 **19. validarDatosRegistraResultado() ↺**

El control de actividades procesa internamente los datos recibidos y valida los datos y registra la actividad asociada a la atención con estado inicial “Pendiente”, determinando el resultado de esta etapa.

20 **◄—— 20. mostrarResultadoRegistrarActividad()**

El control de actividades ordena a la interfaz de nueva actividad mostrar el resultado final de la operación.

21 **◄ - - 21. resultadoMostrado()**

La interfaz de nueva actividad presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF030-SELECCIONAR TIPO DE ACTIVIDAD | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_NuevaActividad	C_Actividad	E_Actividad

BLOQUE 1 · El usuario ingresa al módulo Atenciones (Pasos 1–2)

1 — **1. ingresarModuloAtenciones() ——►** (AS_Usuario → I_NuevaActividad)

El usuario ingresa al módulo Atenciones en la interfaz de nueva actividad.

2 — **2. enviarSeleccionarTipo(solicitud) ——►** (I_NuevaActividad → C_Actividad)

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las atenciones veterinarias registradas y permite identificar las que se encuentran Activas (Pasos 3–4)

3 — **◄—— 3. mostrarAtencionesVeterinarias()** (C_Actividad → I_NuevaActividad)

El control de actividades ordena a la interfaz de nueva actividad mostrar las atenciones veterinarias registradas y permite identificar las que se encuentran Activas.

4 — **◄ - - 4. mostrarSeleccionarTipo()** (I_NuevaActividad → AS_Usuario)

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona una atención veterinaria activa (Pasos 5–6)

5 — **5. seleccionarAtencionVeterinaria() ——►** (AS_Usuario → I_NuevaActividad)

El usuario selecciona una atención veterinaria activa en la interfaz de nueva actividad.

6

6. enviarSeleccionarTipo(solicitud) ——►

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra el detalle de la atención seleccionada, incluyendo la información de la mascota, estado de la atención, progreso y actividades registradas, junto con la opción “Nueva actividad” (Pasos 7–8)

7

◄—— 7. mostrarDetalleAtencion(nuevaActividad)

El control de actividades ordena a la interfaz de nueva actividad mostrar el detalle de la atención seleccionada, incluyendo la información de la mascota, estado de la atención, progreso y actividades registradas, junto con la opción “Nueva actividad”.

8

◄ - - 8. mostrarSeleccionarTipo()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona “Nueva actividad” (Pasos 9–10)

9

9. seleccionarNuevaActividad(nuevaActividad) —— ►

El usuario selecciona “Nueva actividad” en la interfaz de nueva actividad, utilizando los elementos: Nueva actividad.

10

10. enviarSeleccionarTipo(nuevaActividad) —— ►

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema muestra las opciones disponibles para seleccionar el tipo de actividad (Pasos 11–12)

11

◄—— 11. mostrarOpcionesDisponibles()

El control de actividades ordena a la interfaz de nueva actividad mostrar las opciones disponibles para seleccionar el tipo de actividad.

12 ◄ - - 12. mostrarSeleccionarTipo()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario selecciona el tipo de actividad que desea registrar y el sistema muestra los campos correspondientes: - Consulta: motivo de consulta, síntomas, peso, temperatura, diagnóstico, indicaciones/tratamiento, recomendaciones, fecha prevista y observaciones. - Medicación: medicamento, dosis, frecuencia, vía de administración, fecha prevista y observaciones. - Vacunación: nombre de vacuna, dosis, fecha prevista y observaciones. - Ecografía: área a evaluar, hallazgos, fecha prevista y observaciones. - Esterilización: tipo de actividad, notas preoperatorias, fecha prevista y observaciones. - Otro procedimiento: descripción del procedimiento, fecha prevista y observaciones (Pasos 13–16)

13 **13. seleccionarTipoActividad() ——►**

El usuario selecciona el tipo de actividad que desea registrar en la interfaz de nueva actividad.

14 **14. enviarSeleccionarTipo(solicitud) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

15 **◄—— 15. mostrarCamposCorrespondientes()**

El control de actividades ordena a la interfaz de nueva actividad mostrar los campos correspondientes: - Consulta: motivo de consulta, síntomas, peso, temperatura, diagnóstico, indicaciones/tratamiento, recomendaciones, fecha prevista y observaciones. - Medicación: medicamento, dosis, frecuencia, vía de administración, fecha prevista y observaciones. - Vacunación: nombre de vacuna, dosis, fecha prevista y observaciones. - Ecografía: área a evaluar, hallazgos, fecha prevista y observaciones. - Esterilización: tipo de actividad, notas preoperatorias, fecha prevista y observaciones. - Otro procedimiento: descripción del procedimiento, fecha prevista y observaciones.

16 ◄ - - 16. mostrarSeleccionarTipo()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · El usuario completa la información solicitada y selecciona “Registrar actividad”. El sistema valida los datos y, si son correctos, registra la actividad asociada a la atención con estado inicial “Pendiente” (Pasos 17–23)

17

17. completarInformacionSolicitada(registrarActividad) ——►

El usuario completa la información solicitada y selecciona “Registrar actividad” en la interfaz de nueva actividad, utilizando los elementos: Registrar actividad.

18

18. enviarSeleccionarTipo(registrarActividad) ——►

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

19

19. validarDatosCorrectos(criterio) ——►

El control de actividades solicita a la entidad de actividades (base de datos) la información necesaria para validar los datos y, si son correctos, registra la actividad asociada a la atención con estado inicial “Pendiente”.

20

◄ - - 20. datosSeleccionarTipo(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

21

21. validarDatosCorrectosResultado() ↺

El control de actividades procesa internamente los datos recibidos y valida los datos y, si son correctos, registra la actividad asociada a la atención con estado inicial “Pendiente”, determinando el resultado de esta etapa.

22

◄—— 22. mostrarResultadoSeleccionarTipo()

El control de actividades ordena a la interfaz de nueva actividad mostrar el resultado final de la operación.

23

◄ - - 23. resultadoMostrado()

La interfaz de nueva actividad presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF031-CONSULTAR ACTIVIDADS DE UNA ATENCIÓN | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL
AS_Usuario	I_Actividades	C_Actividad

BLOQUE 1 · El usuario selecciona “Atenciones” desde el menú de usuario (Pasos 1–2)

1 — **1. seleccionarAtencionesMenu(atenciones) ——►** (AS_Usuario → I_Actividades)

El usuario selecciona “Atenciones” desde el menú de usuario en la interfaz de actividades, utilizando los elementos: Atenciones.

2 — **2. enviarConsultarActividads(atenciones) ——►** (I_Actividades → C_Actividad)

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las atenciones veterinarias registradas (Pasos 3–4)

3 — **◄—— 3. mostrarAtencionesVeterinarias()** (C_Actividad → I_Actividades)

El control de actividades ordena a la interfaz de actividades mostrar las atenciones veterinarias registradas.

4 — **◄ - - 4. mostrarConsultarActividads()** (I_Actividades → AS_Usuario)

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona una atención veterinaria (Pasos 5–6)

5 — **5. seleccionarAtencionVeterinaria() ——►** (AS_Usuario → I_Actividades)

El usuario selecciona una atención veterinaria en la interfaz de actividades.

6 **6. enviarConsultarActividads(solicitud) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra el detalle de la atención seleccionada y las actividades asociadas (Pasos 7–8)

7 **◄—— 7. mostrarDetalleAtencion()**

El control de actividades ordena a la interfaz de actividades mostrar el detalle de la atención seleccionada y las actividades asociadas.

8 **◄ - - 8. mostrarConsultarActividads()**

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona una actividad para consultar su información (Pasos 9–10)

9 **9. seleccionarActividadConsultar() ——►**

El usuario selecciona una actividad para consultar su información en la interfaz de actividades.

10 **10. enviarConsultarActividads(solicitud) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema muestra los datos registrados de la actividad según el tipo correspondiente, incluyendo su estado y fecha prevista cuando corresponda (Pasos 11–12)

11 **◄—— 11. mostrarDatosRegistrados()**

El control de actividades ordena a la interfaz de actividades mostrar los datos registrados de la actividad según el tipo correspondiente, incluyendo su estado y fecha prevista cuando corresponda.

12 **◄ - - 12. mostrarConsultarActividads()**

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario consulta la información de la actividad seleccionada (Pasos 13–14)

13 **13. consultarInformacionActividad() ——►**

El usuario consulta la información de la actividad seleccionada en la interfaz de actividades.

14 **14. enviarConsultarActividads(solicitud) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema permite regresar al detalle de la atención manteniendo visible la lista de actividades registradas (Pasos 15–16)

15 **◄—— 15. permitirRegresarDetalle()**

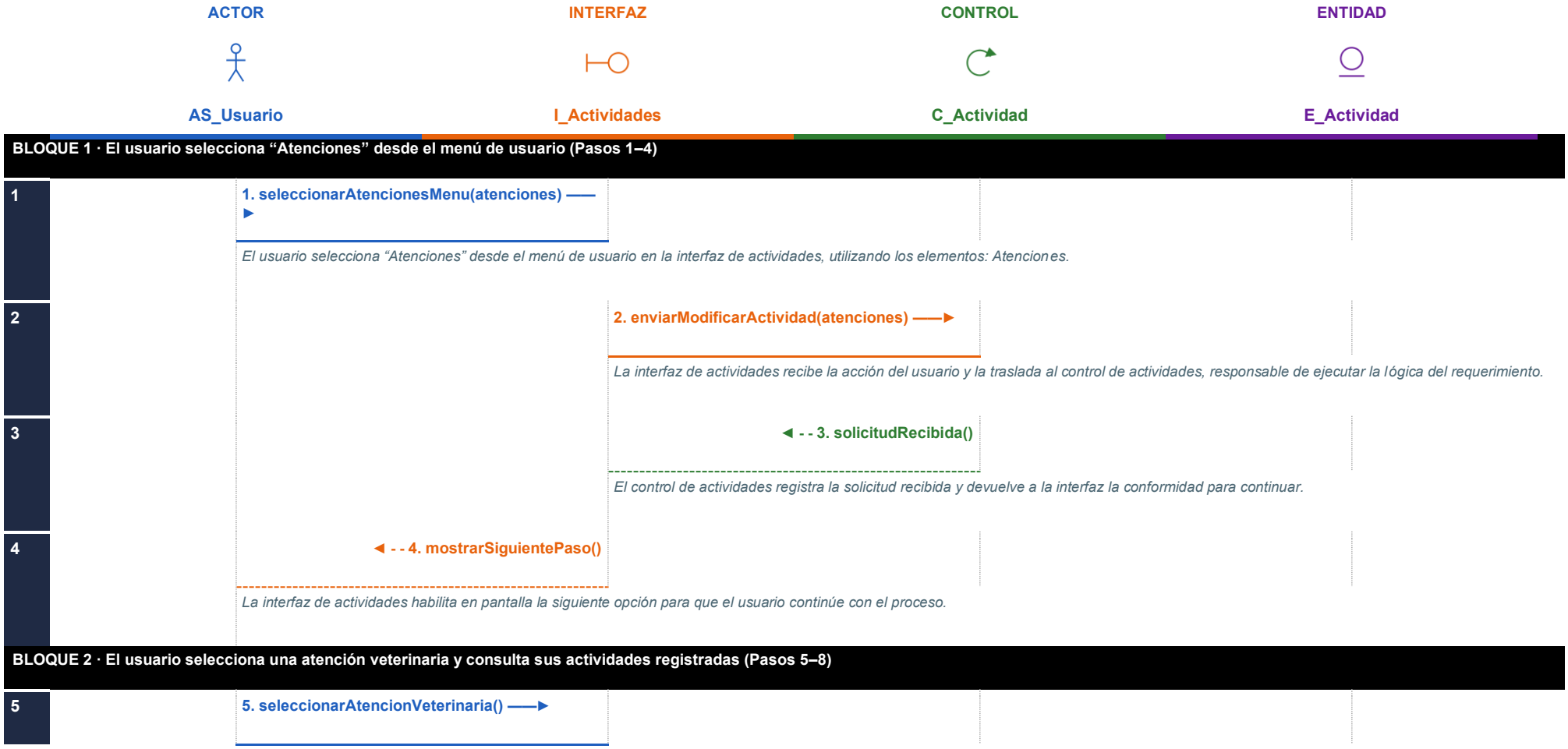
El control de actividades ordena a la interfaz de actividades habilitar la opción de regresar al detalle de la atención manteniendo visible la lista de actividades registradas.

16 **◄ - - 16. mostrarConsultarActividads()**

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF032-MODIFICAR ACTIVIDAD VETERINARIA | ESCENARIO DE ÉXITO



El usuario selecciona una atención veterinaria y consulta sus actividades registradas en la interfaz de actividades.

6 **6. enviarModificarActividad(solicitud) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

7 **◄ - - 7. solicitudRecibida()**

El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

8 **◄ - - 8. mostrarSiguientePaso()**

La interfaz de actividades habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 3 · El usuario selecciona la actividad que desea modificar (Pasos 9–10)

9 **9. seleccionarActividadDesea() ——►**

El usuario selecciona la actividad que desea modificar en la interfaz de actividades.

10 **10. enviarModificarActividad(solicitud) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la información registrada de la actividad y habilita la opción “Editar” (Pasos 11–12)

11 **◄—— 11. mostrarInformacionRegistrada(editar)**

El control de actividades ordena a la interfaz de actividades mostrar la información registrada de la actividad y habilita la opción “Editar”.

12

◄ - - 12. mostrarModificarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona “Editar” y modifica los datos permitidos de acuerdo con el tipo de actividad (Pasos 13–16)

13

13. seleccionarEditarModifica(editar) ——►

El usuario selecciona “Editar” y modifica los datos permitidos de acuerdo con el tipo de actividad en la interfaz de actividades, utilizando los elementos: Editar.

14

14. enviarModificarActividad(editar) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

15

◄ - - 15. solicitudRecibida()

El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

16

◄ - - 16. mostrarSiguientePaso()

La interfaz de actividades habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 6 · El usuario selecciona “Guardar cambios” (Pasos 17–18)

17

17. seleccionarGuardarCambios(guardarCambios) ——►

El usuario selecciona “Guardar cambios” en la interfaz de actividades, utilizando los elementos: Guardar cambios.

18

18. enviarModificarActividad(guardarCambios) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema valida la información ingresada y muestra el mensaje: "¿Deseas guardar los cambios realizados en la actividad?", con las opciones "Confirmar" y "Cancelar" (Pasos 19–23)

19 **19. validarInformacionIngresada(criterio) ——►**

El control de actividades solicita a la entidad de actividades (base de datos) la información necesaria para validar la información ingresada y muestra el mensaje: "¿Deseas guardar los cambios realizados en la actividad?", con las opciones "Confirmar" y "Cancelar".

20 **◄ - - 20. datosModificarActividad(resultado)**

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

21 **21. validarInformacionIngresadaResultado() ↺**

El control de actividades procesa internamente los datos recibidos y valida la información ingresada y muestra el mensaje: "¿Deseas guardar los cambios realizados en la actividad?", con las opciones "Confirmar" y "Cancelar", determinando el resultado de esta etapa.

22 **◄—— 22. mostrarResultadoModificarActividad()**

El control de actividades ordena a la interfaz de actividades mostrar el resultado de la operación realizada.

23 **◄ - - 23. mostrarModificarActividad()**

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · Si el usuario confirma, el sistema guarda los cambios y muestra: "La actividad se actualizó correctamente." (Pasos 24–29)

24 **24. confirmarOperacion() ——►**

Si el usuario confirma en la interfaz de actividades.

25

25. enviarModificarActividad(solicitud) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

26

26. guardarCambiosMuestra(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) guardar los cambios y muestra: “La actividad se actualizó correctamente.”.

27

◄ - - 27. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

28

◄—— 28. mostrarResultadoModificarActividad()

El control de actividades ordena a la interfaz de actividades mostrar el resultado de la operación realizada.

29

◄ - - 29. mostrarModificarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF033-ELIMINAR ACTIVIDAD | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Actividades	C_Actividad	E_Actividad

BLOQUE 1 · El usuario selecciona “Atenciones” desde el menú de usuario (Pasos 1–4)

1 — **1. seleccionarAtencionesMenu(atenciones) ——►** (AS_Usuario → I_Actividades)

El usuario selecciona “Atenciones” desde el menú de usuario en la interfaz de actividades, utilizando los elementos: Atenciones.

2 — **2. enviarEliminarActividad(atenciones) ——►** (I_Actividades → C_Actividad)

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

3 — **◄ - - 3. solicitudRecibida()** (C_Actividad → I_Actividades)

El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

4 — **◄ - - 4. mostrarSiguientePaso()** (I_Actividades → AS_Usuario)

La interfaz de actividades habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 2 · El usuario selecciona una atención veterinaria y consulta sus actividades (Pasos 5–8)

5 — **5. seleccionarAtencionVeterinaria() ——►** (AS_Usuario → I_Actividades)

El usuario selecciona una atención veterinaria y consulta sus actividades en la interfaz de actividades.

6 **6. enviarEliminarActividad(solicitud) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

7 **◄ - - 7. solicitudRecibida()**

El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

8 **◄ - - 8. mostrarSiguientePaso()**

La interfaz de actividades habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 3 · El usuario selecciona la actividad que desea eliminar (Pasos 9–10)

9 **9. seleccionarActividadDesea() ——►**

El usuario selecciona la actividad que desea eliminar en la interfaz de actividades.

10 **10. enviarEliminarActividad(solicitud) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la información de la actividad y la opción “Eliminar” (Pasos 11–12)

11 **◄—— 11. mostrarInformacionActividad(eliminar)**

El control de actividades ordena a la interfaz de actividades mostrar la información de la actividad y la opción “Eliminar”.

12 ◄ - - 12. mostrarEliminarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona “Eliminar” (Pasos 13–14)

13 **13. seleccionarEliminar(eliminar) ——►**

El usuario selecciona “Eliminar” en la interfaz de actividades, utilizando los elementos: Eliminar.

14 **14. enviarEliminarActividad(eliminar) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema muestra el mensaje: “¿Deseas eliminar esta actividad?”, con las opciones “Confirmar” y “Cancelar” (Pasos 15–16)

15 **◄—— 15. mostrarMensajeDeseas(deseasEliminar)**

El control de actividades ordena a la interfaz de actividades mostrar el mensaje: “¿Deseas eliminar esta actividad?”, con las opciones “Confirmar” y “Cancelar”.

16 ◄ - - 16. mostrarEliminarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario selecciona “Confirmar” (Pasos 17–18)

17 **17. seleccionarConfirmar(confirmar) ——►**

El usuario selecciona “Confirmar” en la interfaz de actividades, utilizando los elementos: Confirmar.

18

18. enviarEliminarActividad(confirmar) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema elimina la actividad y muestra: “La actividad se eliminó correctamente (Pasos 19–22)

19

19. eliminarActividadMuestra(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) eliminar la actividad y muestra: “La actividad se eliminó correctamente.

20

◄ - - 20. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

21

◄—— 21. mostrarResultadoEliminarActividad()

El control de actividades ordena a la interfaz de actividades mostrar el resultado de la operación realizada.

22

◄ - - 22. mostrarEliminarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF034-ASIGNAR ESTADO INICIAL A UNA ACTIVIDAD | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Actividades	C_Actividad	E_Actividad

BLOQUE 1 · El usuario registra una nueva actividad dentro de una atención veterinaria activa (Pasos 1–2)

1 — **1. registrarNuevaActividad() ——►** (AS_Usuario → I_Actividades)

El usuario registra una nueva actividad dentro de una atención veterinaria activa en la interfaz de actividades.

2 — **2. enviarAsignarEstado(solicitud) ——►** (I_Actividades → C_Actividad)

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema valida la información ingresada para el registro de la actividad (Pasos 3–5)

3 — **3. validarInformacionIngresada(criterio) ——►** (C_Actividad → E_Actividad)

El control de actividades solicita a la entidad de actividades (base de datos) la información necesaria para validar la información ingresada para el registro de la actividad.

4 — **◄ - - 4. datosAsignarEstado(resultado)** (E_Actividad → C_Actividad)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

5 — **5. validarInformacionIngresadaResultado() ↺** (C_Actividad)

El control de actividades procesa internamente los datos recibidos y valida la información ingresada para el registro de la actividad, determinando el resultado de esta etapa.

BLOQUE 3 · El sistema registra la nueva actividad asociada a la atención correspondiente (Pasos 6–7)

6

6. registrarNuevaActividad(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) registrar la nueva actividad asociada a la atención correspondiente.

7

◄ - - 7. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 4 · El sistema asigna automáticamente el estado “Pendiente” a la actividad (Pasos 8–9)

8

8. asignarAutomaticamenteEstado(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) asignar automáticamente el estado “Pendiente” a la actividad.

9

◄ - - 9. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 5 · El sistema guarda el estado asignado junto con la información de la actividad (Pasos 10–11)

10

10. guardarEstadoAsignado(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) guardar el estado asignado junto con la información de la actividad.

11

◄ - - 11. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 6 · El sistema muestra la actividad registrada con el estado “Pendiente” dentro del detalle de la atención (Pasos 12–13)

12 ◄—— **12. mostrarActividadRegistrada(pendiente)**

El control de actividades ordena a la interfaz de actividades mostrar la actividad registrada con el estado “Pendiente” dentro del detalle de la atención.

13 ◄ - - **13. mostrarAsignarEstado()**

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario consulta la actividad registrada y verifica su estado inicial (Pasos 14–15)

14 **14. consultarActividadRegistrada()** ——►

El usuario consulta la actividad registrada y verifica su estado inicial en la interfaz de actividades.

15 **15. enviarAsignarEstado(solicitud)** ——►

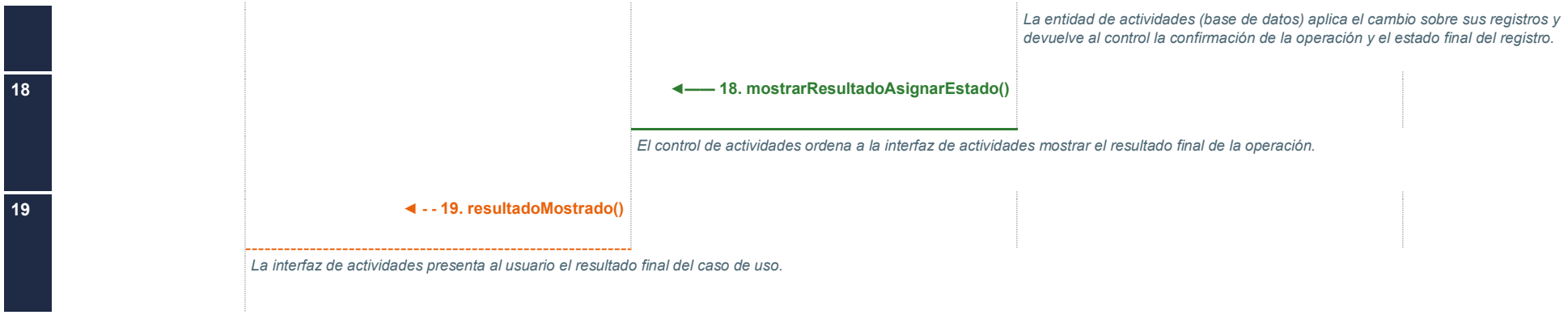
La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema mantiene la actividad en estado “Pendiente” hasta que el usuario realice una acción permitida para modificar su estado (Pasos 16–19)

16 **16. mantenerActividadEstado(datos)** ——►

El control de actividades solicita a la entidad de actividades (base de datos) mantener la actividad en estado “Pendiente” hasta que el usuario realice una acción permitida para modificar su estado.

17 ◄ - - **17. operacionConfirmada(estado)**


La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.
18
◄—— 18. mostrarResultadoAsignarEstado()
El control de actividades ordena a la interfaz de actividades mostrar el resultado final de la operación.
19
◄ - - 19. resultadoMostrado()
La interfaz de actividades presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF035-MARCAR ACTIVIDAD COMO COMPLETADA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Actividades	C_Actividad	E_Actividad

BLOQUE 1 · El usuario selecciona “Atenciones” desde el menú de usuario (Pasos 1–4)

1 — **1. seleccionarAtencionesMenu(atenciones) ——►** (AS_Usuario → I_Actividades)

El usuario selecciona “Atenciones” desde el menú de usuario en la interfaz de actividades, utilizando los elementos: Atenciones.

2 — **2. enviarMarcarActividad(atenciones) ——►** (I_Actividades → C_Actividad)

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

3 — **◄ - - 3. solicitudRecibida()** (C_Actividad → I_Actividades)

El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

4 — **◄ - - 4. mostrarSiguientePaso()** (I_Actividades → AS_Usuario)

La interfaz de actividades habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 2 · El usuario selecciona una atención veterinaria activa (Pasos 5–6)

5 — **5. seleccionarAtencionVeterinaria() ——►** (AS_Usuario → I_Actividades)

El usuario selecciona una atención veterinaria activa en la interfaz de actividades.

6

6. enviarMarcarActividad(solicitud) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra el detalle de la atención y las actividades registradas (Pasos 7–8)

7

◄—— 7. mostrarDetalleAtencion()

El control de actividades ordena a la interfaz de actividades mostrar el detalle de la atención y las actividades registradas.

8

◄ - - 8. mostrarMarcarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona una actividad que se encuentra en estado “Pendiente” (Pasos 9–10)

9

9. seleccionarActividadEncuentra(pendiente) —— ►

El usuario selecciona una actividad que se encuentra en estado “Pendiente” en la interfaz de actividades, utilizando los elementos: Pendiente.

10

10. enviarMarcarActividad(pendiente) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema muestra la información de la actividad y la opción “Marcar como completada” (Pasos 11–12)

11

◄—— 11. mostrarInformacionActividad(marcarCompletada)

El control de actividades ordena a la interfaz de actividades mostrar la información de la actividad y la opción “Marcar como completada”.

12 ◄ - - 12. mostrarMarcarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario selecciona “Marcar como completada” (Pasos 13–14)

13 **13. seleccionarMarcarCompletada(marcarCompletada) ——►**

El usuario selecciona “Marcar como completada” en la interfaz de actividades, utilizando los elementos: Marcar como completada.

14 **14. enviarMarcarActividad(marcarCompletada) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema muestra el mensaje: “¿Deseas marcar esta actividad como completada?”, con las opciones “Confirmar” y “Cancelar” (Pasos 15–16)

15 **◄—— 15. mostrarMensajeDeseas(deseasMarcar)**

El control de actividades ordena a la interfaz de actividades mostrar el mensaje: “¿Deseas marcar esta actividad como completada?”, con las opciones “Confirmar” y “Cancelar”.

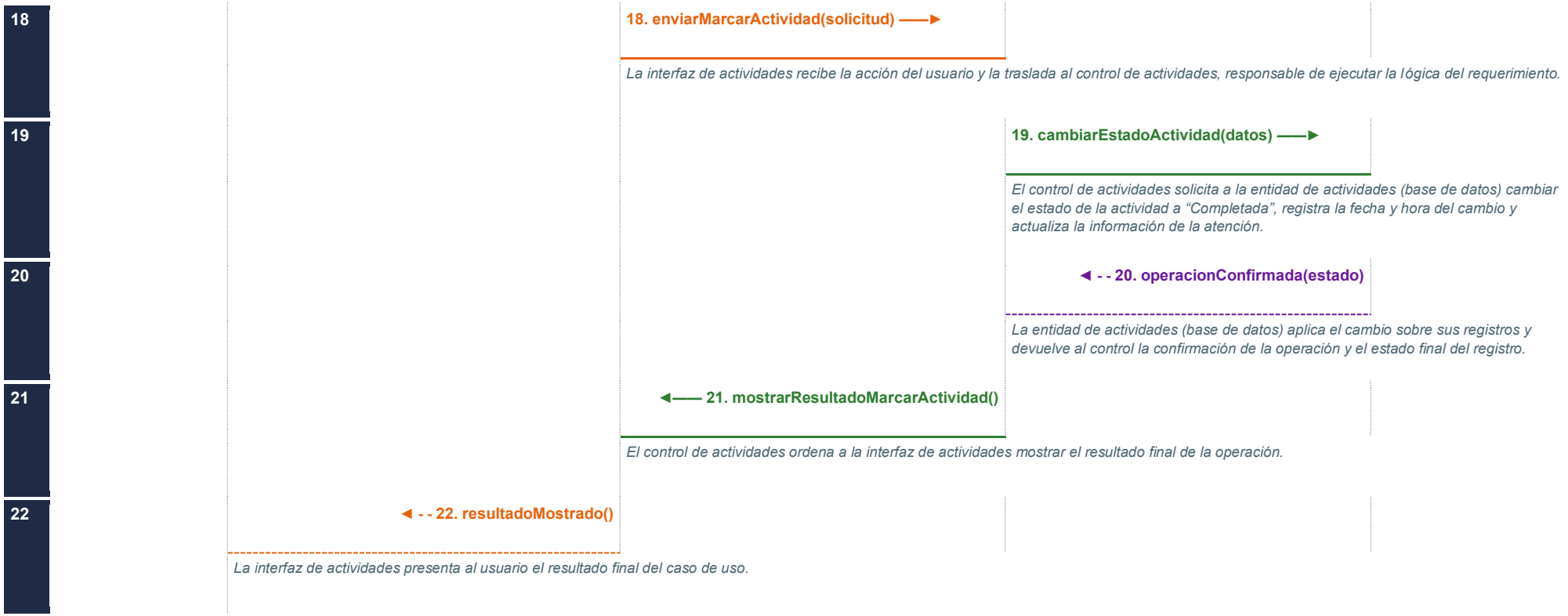
16 **◄ - - 16. mostrarMarcarActividad()**

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · Si el usuario confirma, el sistema cambia el estado de la actividad a “Completada”, registra la fecha y hora del cambio y actualiza la información de la atención (Pasos 17–22)

17 **17. confirmarOperacion() ——►**

Si el usuario confirma en la interfaz de actividades.


18
18. enviarMarcarActividad(solicitud) ——►
La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.
19
19. cambiarEstadoActividad(datos) ——►
El control de actividades solicita a la entidad de actividades (base de datos) cambiar el estado de la actividad a “Completada”, registra la fecha y hora del cambio y actualiza la información de la atención.
20
◄ - - 20. operacionConfirmada(estado)
La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.
21
◄—— 21. mostrarResultadoMarcarActividad()
El control de actividades ordena a la interfaz de actividades mostrar el resultado final de la operación.
22
◄ - - 22. resultadoMostrado()
La interfaz de actividades presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF036-MARCAR ACTIVIDAD COMO NO REALIZADA | ESCENARIO DE ÉXITO



El usuario selecciona una atención veterinaria activa en la interfaz de actividades.

6

6. enviarMarcarActividad(solicitud) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra el detalle de la atención y las actividades registradas (Pasos 7–8)

7

◄—— 7. mostrarDetalleAtencion()

El control de actividades ordena a la interfaz de actividades mostrar el detalle de la atención y las actividades registradas.

8

◄ - - 8. mostrarMarcarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona una actividad que se encuentra en estado “Pendiente” (Pasos 9–10)

9

9. seleccionarActividadEncuentra(pendiente) — —►

El usuario selecciona una actividad que se encuentra en estado “Pendiente” en la interfaz de actividades, utilizando los elementos: Pendiente.

10

10. enviarMarcarActividad(pendiente) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema muestra la información de la actividad y la opción “Marcar como no realizada” (Pasos 11–12)

11

◄—— 11. mostrarInformacionActividad(marcarRealizada)

El control de actividades ordena a la interfaz de actividades mostrar la información de la actividad y la opción “Marcar como no realizada”.

12 ◄ - - 12. mostrarMarcarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario selecciona “Marcar como no realizada” (Pasos 13–14)

13 **13. seleccionarMarcarRealizada(marcarRealizada) ——►**

El usuario selecciona “Marcar como no realizada” en la interfaz de actividades, utilizando los elementos: Marcar como no realizada.

14 **14. enviarMarcarActividad(marcarRealizada) — —►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema solicita ingresar el motivo por el cual la actividad no fue realizada y muestra las opciones “Confirmar” y “Cancelar” (Pasos 15–16)

15 **◄—— 15. solicitarIngresarMotivo(confirmar)**

El control de actividades ordena a la interfaz de actividades solicitar ingresar el motivo por el cual la actividad no fue realizada y muestra las opciones “Confirmar” y “Cancelar”.

16 ◄ - - 16. mostrarMarcarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · El usuario ingresa el motivo y confirma. El sistema cambia el estado de la actividad a “No realizada”, registra la fecha y hora del cambio y actualiza la información de la atención (Pasos 17–22)

17 **17. confirmarOperacion() ——►**

El usuario ingresa el motivo y confirma en la interfaz de actividades.

18

18. enviarMarcarActividad(solicitud) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

19

19. cambiarEstadoActividad(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) cambiar el estado de la actividad a “No realizada”, registra la fecha y hora del cambio y actualiza la información de la atención.

20

◄ - - 20. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

21

◄—— 21. mostrarResultadoMarcarActividad()

El control de actividades ordena a la interfaz de actividades mostrar el resultado final de la operación.

22

◄ - - 22. resultadoMostrado()

La interfaz de actividades presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF037-REACTIVAR ACTIVIDAD NO REALIZADA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
			
AS_Usuario	I_Actividades	C_Actividad	E_Actividad

BLOQUE 1 · El usuario selecciona “Atenciones” desde el menú de usuario (Pasos 1–4)

1 — **1. seleccionarAtencionesMenu(atenciones) ——►** (ACTOR → INTERFAZ)

El usuario selecciona “Atenciones” desde el menú de usuario en la interfaz de actividades, utilizando los elementos: Atenciones.

2 — **2. enviarReactivarActividad(atenciones) ——►** (INTERFAZ → CONTROL)

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

3 — **◄ - - 3. solicitudRecibida()** (CONTROL → INTERFAZ)

El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

4 — **◄ - - 4. mostrarSiguientePaso()** (INTERFAZ → ACTOR)

La interfaz de actividades habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 2 · El usuario selecciona la atención veterinaria que contiene la actividad marcada como “No realizada” (Pasos 5–6)

5 — **5. seleccionarAtencionVeterinaria(realizada) ——►** (ACTOR → INTERFAZ)

El usuario selecciona la atención veterinaria que contiene la actividad marcada como “No realizada” en la interfaz de actividades, utilizando los elementos: No realizada.

6

6. enviarReactivarActividad(realizada) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra el detalle de la atención y las actividades registradas (Pasos 7–8)

7

◄—— 7. mostrarDetalleAtencion()

El control de actividades ordena a la interfaz de actividades mostrar el detalle de la atención y las actividades registradas.

8

◄ - - 8. mostrarReactivarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona la actividad que se encuentra en estado “No realizada” (Pasos 9–10)

9

9. seleccionarActividadEncuentra(realizada) —— ►

El usuario selecciona la actividad que se encuentra en estado “No realizada” en la interfaz de actividades, utilizando los elementos: No realizada.

10

10. enviarReactivarActividad(realizada) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema muestra la información de la actividad y la opción “Reactivar actividad” (Pasos 11–12)

11

◄—— 11. mostrarInformacionActividad(reactivarActividad)

El control de actividades ordena a la interfaz de actividades mostrar la información de la actividad y la opción “Reactivar actividad”.

12 ◄ - - 12. mostrarReactivarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario selecciona “Reactivar actividad” (Pasos 13–14)

13 **13. seleccionarReactivarActividad(reactivarActividad) ——►**

El usuario selecciona “Reactivar actividad” en la interfaz de actividades, utilizando los elementos: Reactivar actividad.

14 **14. enviarReactivarActividad(reactivarActividad) ——►**

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema muestra el mensaje: “¿Deseas reactivar esta actividad?”, con las opciones “Confirmar” y “Cancelar” (Pasos 15–16)

15 **◄—— 15. mostrarMensajeDeseas(deseasReactivar)**

El control de actividades ordena a la interfaz de actividades mostrar el mensaje: “¿Deseas reactivar esta actividad?”, con las opciones “Confirmar” y “Cancelar”.

16 ◄ - - 16. mostrarReactivarActividad()

La interfaz de actividades presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · Si el usuario confirma, el sistema cambia el estado de la actividad a “Pendiente”, registra la fecha y hora del cambio y actualiza la información de la atención (Pasos 17–22)

17 **17. confirmarOperacion() ——►**

Si el usuario confirma en la interfaz de actividades.

18

18. enviarReactivarActividad(solicitud) ——►

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

19

19. cambiarEstadoActividad(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) cambiar el estado de la actividad a "Pendiente", registra la fecha y hora del cambio y actualiza la información de la atención.

20

◄ - - 20. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

21

◄—— 21. mostrarResultadoReactivarActividad()

El control de actividades ordena a la interfaz de actividades mostrar el resultado final de la operación.

22

◄ - - 22. resultadoMostrado()

La interfaz de actividades presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF038-CALCULAR AVANCE DE ATENCIÓN | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Seguimiento	C_Seguimiento	E_Actividad

BLOQUE 1 · El usuario selecciona “Seguimientos” desde el menú de usuario (Pasos 1–2)

1 — **1. seleccionarSeguimientosMenu(seguimientos)** ——►

El usuario selecciona “Seguimientos” desde el menú de usuario en la interfaz de seguimiento, utilizando los elementos: Seguimientos.

2 — **2. enviarCalcularAvance(seguimientos)** ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las atenciones veterinarias disponibles para seguimiento, mostrando mascota, especie, raza, código de atención y estado de la atención (Pasos 3–4)

3 — ◄—— **3. mostrarAtencionesVeterinarias()**

El control de seguimiento ordena a la interfaz de seguimiento mostrar las atenciones veterinarias disponibles para seguimiento, mostrando mascota, especie, raza, código de atención y estado de la atención.

4 — **4. vistaDesplegada()** - -►

La interfaz de seguimiento confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 3 · El sistema consulta las actividades asociadas a cada atención (Pasos 5–7)

5 — **5. consultarActividadesAsociadas(criterio)** ——►

			El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para consultar las actividades asociadas a cada atención.
6			**◄ - - 6. datosCalcularAvance(resultado)**
			La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.
7			**7. consultarActividadesAsociadasResultado() ↺**
			El control de seguimiento procesa internamente los datos recibidos y consulta las actividades asociadas a cada atención, determinando el resultado de esta etapa.
BLOQUE 4 · El sistema identifica la cantidad total de actividades registradas (Pasos 8–10)			
8			**8. identificarCantidadTotal(criterio) ——►**
			El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para identificar la cantidad total de actividades registradas.
9			**◄ - - 9. datosCalcularAvance(resultado)**
			La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.
10			**10. identificarCantidadTotalResultado() ↺**
			El control de seguimiento procesa internamente los datos recibidos y identifica la cantidad total de actividades registradas, determinando el resultado de esta etapa.
BLOQUE 5 · El sistema identifica la cantidad de actividades en estado Completada (Pasos 11–13)			
11			**11. identificarCantidadActividades(criterio) —— ►**

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para identificar la cantidad de actividades en estado Completada.

12 **◄ - - 12. datosCalcularAvance(resultado)**

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

13 **13. identificarCantidadActividadesResultado() ↺**

El control de seguimiento procesa internamente los datos recibidos y identifica la cantidad de actividades en estado Completada, determinando el resultado de esta etapa.

BLOQUE 6 · El sistema calcula el porcentaje de avance (Pasos 14–16)

14 **14. calcularPorcentajeAvance(criterio) ——►**

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para calcular el porcentaje de avance.

15 **◄ - - 15. datosCalcularAvance(resultado)**

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

16 **16. calcularPorcentajeAvanceResultado() ↺**

El control de seguimiento procesa internamente los datos recibidos y calcula el porcentaje de avance, determinando el resultado de esta etapa.

BLOQUE 7 · El sistema muestra el porcentaje y la cantidad de actividades completadas respecto del total (Pasos 17–18)

17 **◄—— 17. mostrarPorcentajeCantidad()**

El control de seguimiento ordena a la interfaz de seguimiento mostrar el porcentaje y la cantidad de actividades completadas respecto del total.

18 ◄ - - 18. mostrarCalcularAvance()

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · El usuario puede seleccionar “Ver seguimiento” para consultar el detalle de una atención (Pasos 19–22)

19 **19. seleccionarVerSeguimiento(verSeguimiento) ——►**

El usuario puede seleccionar “Ver seguimiento” para consultar el detalle de una atención en la interfaz de seguimiento.

20 **20. enviarCalcularAvance(verSeguimiento) —— ►**

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

21 **◄—— 21. mostrarResultadoCalcularAvance()**

El control de seguimiento ordena a la interfaz de seguimiento mostrar el resultado final de la operación.

22 **◄ - - 22. resultadoMostrado()**

La interfaz de seguimiento presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF039-VISUALIZAR PROGRESO DE ATENCIÓN | ESCENARIO DE ÉXITO



El usuario selecciona una atención en la interfaz de seguimiento.

6

6. enviarVisualizarProgreso(solicitud) ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema consulta el porcentaje de avance correspondiente (Pasos 7–9)

7

7. consultarPorcentajeAvance(criterio) ——►

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para consultar el porcentaje de avance correspondiente.

8

◄ - - 8. datosVisualizarProgreso(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

9

9. consultarPorcentajeAvanceResultado() ↺

El control de seguimiento procesa internamente los datos recibidos y consulta el porcentaje de avance correspondiente, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema genera la representación visual del porcentaje mediante una barra de progreso (Pasos 10–12)

10

10. generarRepresentacionVisual(criterio) ——►

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para generar la representación visual del porcentaje mediante una barra de progreso.

11

◄ - - 11. datosVisualizarProgreso(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

12

12. generarRepresentacionVisualResultado() ↺

El control de seguimiento procesa internamente los datos recibidos y genera la representación visual del porcentaje mediante una barra de progreso, determinando el resultado de esta etapa.

BLOQUE 6 · El sistema muestra la barra junto con el porcentaje de avance (Pasos 13–14)

13

◄—— 13. mostrarBarraJunto()

El control de seguimiento ordena a la interfaz de seguimiento mostrar la barra junto con el porcentaje de avance.

14

◄ - - 14. mostrarVisualizarProgreso()

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario visualiza el progreso de la atención (Pasos 15–18)

15

15. visualizarProgresoAtencion() ——►

El usuario visualiza el progreso de la atención en la interfaz de seguimiento.

16

16. enviarVisualizarProgreso(solicitud) ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

17

◄ - - 17. solicitudRecibida()

El control de seguimiento registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

18

◄ - - 18. mostrarSiguientePaso()

La interfaz de seguimiento habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 8 · El usuario visualiza la barra de progreso y el porcentaje de avance de la atención (Pasos 19–22)

19

19. visualizarBarraProgreso() ——►

El usuario visualiza la barra de progreso y el porcentaje de avance de la atención en la interfaz de seguimiento.

20

20. enviarVisualizarProgreso(solicitud) ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

21

◄—— 21. mostrarResultadoVisualizarProgreso()

El control de seguimiento ordena a la interfaz de seguimiento mostrar el resultado final de la operación.

22

◄ - - 22. resultadoMostrado()

La interfaz de seguimiento presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF040-CONSULTAR ACTIVIDADES POR ESTADO | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL
AS_Usuario	I_Seguimiento	C_Seguimiento

BLOQUE 1 · El usuario selecciona “Seguimientos” desde el menú de usuario (Pasos 1–2)

1. **1. seleccionarSeguimientosMenu(seguimientos) ——►**

El usuario selecciona “Seguimientos” desde el menú de usuario en la interfaz de seguimiento, utilizando los elementos: Seguimientos.

2. **2. enviarConsultarActividades(seguimientos) ——►**

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las atenciones disponibles (Pasos 3–4)

3. **◄—— 3. mostrarAtencionesDisponibles()**

El control de seguimiento ordena a la interfaz de seguimiento mostrar las atenciones disponibles.

4. **◄ - - 4. mostrarConsultarActividades()**

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona una atención (Pasos 5–6)

5. **5. seleccionarAtencion() ——►**

El usuario selecciona una atención en la interfaz de seguimiento.

6 **6. enviarConsultarActividades(solicitud) ——►**

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra el detalle del seguimiento y la opción “Ver actividades” (Pasos 7–8)

7 **◄—— 7. mostrarDetalleSeguimiento(verActividades)**

El control de seguimiento ordena a la interfaz de seguimiento mostrar el detalle del seguimiento y la opción “Ver actividades”.

8 **◄ - - 8. mostrarConsultarActividades()**

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona “Ver actividades” (Pasos 9–10)

9 **9. seleccionarVerActividades(verActividades) ——►**

El usuario selecciona “Ver actividades” en la interfaz de seguimiento, utilizando los elementos: Ver actividades.

10 **10. enviarConsultarActividades(verActividades) ——►**

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema muestra las actividades clasificadas por Pendiente, Completada y No realizada, junto con las opciones de filtro Todas, Pendientes, Completadas y No realizadas (Pasos 11–12)

11 **◄—— 11. mostrarActividadesClasificadas()**

El control de seguimiento ordena a la interfaz de seguimiento mostrar las actividades clasificadas por Pendiente, Completada y No realizada, junto con las opciones de filtro Todas, Pendientes, Completadas y No realizadas.

12 ◄ - - 12. mostrarConsultarActividades()

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El usuario selecciona un filtro (Pasos 13–14)

13 13. seleccionarFiltro() ——►

El usuario selecciona un filtro en la interfaz de seguimiento.

14 14. enviarConsultarActividades(solicitud) ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema muestra las actividades correspondientes al filtro seleccionado (Pasos 15–16)

15 ◄—— 15. mostrarActividadesCorrespondientes()

El control de seguimiento ordena a la interfaz de seguimiento mostrar las actividades correspondientes al filtro seleccionado.

16 ◄ - - 16. mostrarConsultarActividades()

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF041-VISUALIZAR LÍNEA DE TIEMPO DE ACTIVIDADES | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Seguimiento	C_Seguimiento	E_Actividad

BLOQUE 1 · El usuario selecciona “Seguimientos” desde el menú de usuario (Pasos 1–2)

1 — **1. seleccionarSeguimientosMenu(seguimientos)** ——►

El usuario selecciona “Seguimientos” desde el menú de usuario en la interfaz de seguimiento, utilizando los elementos: Seguimientos.

2 — **2. enviarVisualizarLinea(seguimientos)** ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las atenciones veterinarias disponibles para seguimiento (Pasos 3–4)

3 — ◄—— **3. mostrarAtencionesVeterinarias()**

El control de seguimiento ordena a la interfaz de seguimiento mostrar las atenciones veterinarias disponibles para seguimiento.

4 — ◄ - - **4. mostrarVisualizarLinea()**

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona una atención veterinaria (Pasos 5–6)

5 — **5. seleccionarAtencionVeterinaria()** ——►

El usuario selecciona una atención veterinaria en la interfaz de seguimiento.

6

6. enviarVisualizarLinea(solicitud) ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra el detalle del seguimiento y la opción “Ver línea de tiempo” (Pasos 7–8)

7

◄—— 7. mostrarDetalleSeguimiento(verLinea)

El control de seguimiento ordena a la interfaz de seguimiento mostrar el detalle del seguimiento y la opción “Ver línea de tiempo”.

8

◄ - - 8. mostrarVisualizarLinea()

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona “Ver línea de tiempo” (Pasos 9–10)

9

9. seleccionarVerLinea(verLinea) ——►

El usuario selecciona “Ver línea de tiempo” en la interfaz de seguimiento, utilizando los elementos: Ver línea de tiempo.

10

10. enviarVisualizarLinea(verLinea) ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema consulta los eventos registrados de la atención y los organiza cronológicamente (Pasos 11–13)

11

11. consultarEventosRegistrados(criterio) ——►

El control de seguimiento solicita a la entidad de actividades (base de datos) la información necesaria para consultar los eventos registrados de la atención y los organiza cronológicamente.

12 ◄ - - 12. datosVisualizarLinea(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

13 13. consultarEventosRegistradosResultado() ↺

El control de seguimiento procesa internamente los datos recibidos y consulta los eventos registrados de la atención y los organiza cronológicamente, determinando el resultado de esta etapa.

BLOQUE 7 · El sistema muestra los eventos correspondientes, incluyendo el inicio de la atención, registro de actividades, cambios de estado, reactivaciones y finalización de la atención, junto con su fecha y hora (Pasos 14–15)

14 ◄—— 14. mostrarEventosCorrespondientes()

El control de seguimiento ordena a la interfaz de seguimiento mostrar los eventos correspondientes, incluyendo el inicio de la atención, registro de actividades, cambios de estado, reactivaciones y finalización de la atención, junto con su fecha y hora.

15 ◄ - - 15. mostrarVisualizarLinea()

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 8 · El usuario consulta la línea de tiempo de la atención seleccionada (Pasos 16–19)

16 16. consultarLineaTiempo() ——►

El usuario consulta la línea de tiempo de la atención seleccionada en la interfaz de seguimiento.

17 17. enviarVisualizarLinea(solicitud) ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

18 ◄—— 18. mostrarResultadoVisualizarLinea()

El control de seguimiento ordena a la interfaz de seguimiento mostrar el resultado final de la operación.

19 ◄ - - 19. resultadoMostrado()

La interfaz de seguimiento presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF042-REGISTRAR CAMBIOS REALIZADOS | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Actividades	C_Actividad	E_HistorialCambio

BLOQUE 1 · El usuario realiza una acción de registro, modificación, eliminación, cambio de estado o reactivación sobre la información permitida (Pasos 1–2)

1 — **1. realizarAccionRegistro() ——►** (AS_Usuario → I_Actividades)

El usuario realiza una acción de registro, modificación, eliminación, cambio de estado o reactivación sobre la información permitida en la interfaz de actividades.

2 — **2. enviarRegistrarCambios(solicitud) ——►** (I_Actividades → C_Actividad)

La interfaz de actividades recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema procesa la acción realizada (Pasos 3–4)

3 — **3. procesarAccionRealizada(datos) ——►** (C_Actividad → E_HistorialCambio)

El control de actividades solicita a la entidad de historial de cambios (base de datos) procesar la acción realizada.

4 — **◄ - - 4. operacionConfirmada(estado)** (E_HistorialCambio → C_Actividad)

La entidad de historial de cambios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 3 · El sistema identifica al usuario que ejecutó la acción (Pasos 5–7)

5 — **5. identificarUsuarioEjecuto(criterio) ——►** (C_Actividad → E_HistorialCambio)

El control de actividades solicita a la entidad de historial de cambios (base de datos) la información necesaria para identificar al usuario que ejecutó la acción.

6 **◄ - - 6. datosRegistrarCambios(resultado)**

La entidad de historial de cambios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

7 **7. identificarUsuarioEjecutoResultado() ↺**

El control de actividades procesa internamente los datos recibidos y identifica al usuario que ejecutó la acción, determinando el resultado de esta etapa.

BLOQUE 4 · El sistema identifica el tipo de cambio realizado y la información afectada (Pasos 8–10)

8 **8. identificarTipoCambio(criterio) ——►**

El control de actividades solicita a la entidad de historial de cambios (base de datos) la información necesaria para identificar el tipo de cambio realizado y la información afectada.

9 **◄ - - 9. datosRegistrarCambios(resultado)**

La entidad de historial de cambios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

10 **10. identificarTipoCambioResultado() ↺**

El control de actividades procesa internamente los datos recibidos y identifica el tipo de cambio realizado y la información afectada, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema registra la fecha y hora del cambio (Pasos 11–12)

11 **11. registrarFechaHora(datos) ——►**

El control de actividades solicita a la entidad de historial de cambios (base de datos) registrar la fecha y hora del cambio.

12 **◄ - - 12. operacionConfirmada(estado)**

La entidad de historial de cambios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 6 · El sistema almacena el registro de la acción realizada (Pasos 13–14)

13 **13. almacenarRegistroAccion(datos) ——►**

El control de actividades solicita a la entidad de historial de cambios (base de datos) almacenar el registro de la acción realizada.

14 **◄ - - 14. operacionConfirmada(estado)**

La entidad de historial de cambios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 7 · El sistema confirma la operación solicitada al usuario (Pasos 15–17)

15 **15. confirmarOperacionSolicitada() ↺**

El control de actividades realiza internamente la lógica necesaria para confirmar la operación solicitada al usuario.

16 **◄—— 16. mostrarResultadoRegistrarCambios()**

El control de actividades ordena a la interfaz de actividades mostrar el resultado final de la operación.

17 **◄ - - 17. resultadoMostrado()**

La interfaz de actividades presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF043-ACTUALIZAR INFORMACIÓN EN TIEMPO REAL | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_Seguimiento	C_Seguimiento	E_Atencion

BLOQUE 1 · El usuario selecciona “Seguimientos” desde el menú de usuario (Pasos 1–2)

1. **1. seleccionarSeguimientosMenu(seguimientos)** ——▶ (AS_Usuario → I_Seguimiento)

   El usuario selecciona “Seguimientos” desde el menú de usuario en la interfaz de seguimiento, utilizando los elementos: Seguimientos.

2. **2. enviarActualizarInformacion(seguimientos)** ——▶ (I_Seguimiento → C_Seguimiento)

   La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las atenciones veterinarias disponibles para seguimiento (Pasos 3–4)

3. **◀—— 3. mostrarAtencionesVeterinarias()** (C_Seguimiento → I_Seguimiento)

   El control de seguimiento ordena a la interfaz de seguimiento mostrar las atenciones veterinarias disponibles para seguimiento.

4. **◀ - - 4. mostrarActualizarInformacion()** (I_Seguimiento → AS_Usuario)

   La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El usuario selecciona una atención veterinaria (Pasos 5–6)

5. **5. seleccionarAtencionVeterinaria()** ——▶ (AS_Usuario → I_Seguimiento)

El usuario selecciona una atención veterinaria en la interfaz de seguimiento.

6

6. enviarActualizarInformacion(solicitud) ——►

La interfaz de seguimiento recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la información actual del seguimiento (Pasos 7–8)

7

◄—— 7. mostrarInformacionActual()

El control de seguimiento ordena a la interfaz de seguimiento mostrar la información actual del seguimiento.

8

8. vistaDesplegada() - -►

La interfaz de seguimiento confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 5 · Se produce un cambio en una actividad de la atención, como pasar de Pendiente a Completada o No realizada (Paso 9)

9

9. detectarCambio() ↺

El control de seguimiento detecta internamente el evento descrito: Se produce un cambio en una actividad de la atención, como pasar de Pendiente a Completada o No realizada.

BLOQUE 6 · El sistema detecta el cambio registrado en la atención (Pasos 10–12)

10

10. detectarCambioRegistrado(criterio) ——►

El control de seguimiento solicita a la entidad de atenciones (base de datos) la información necesaria para detectar el cambio registrado en la atención.

11

◄ - - 11. datosActualizarInformacion(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

12

12. detectarCambioRegistradoResultado() ↺

El control de seguimiento procesa internamente los datos recibidos y detecta el cambio registrado en la atención, determinando el resultado de esta etapa.

BLOQUE 7 · El sistema actualiza automáticamente la información correspondiente del seguimiento (Pasos 13–14)

13

13. actualizarAutomaticamenteInformacion(datos) ——►

El control de seguimiento solicita a la entidad de atenciones (base de datos) actualizar automáticamente la información correspondiente del seguimiento.

14

◄ - - **14. operacionConfirmada(estado)**

La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 8 · El sistema muestra el nuevo estado de la actividad y el porcentaje de avance actualizado sin que el usuario tenga que recargar la pantalla (Pasos 15–16)

15

◄—— **15. mostrarNuevoEstado()**

El control de seguimiento ordena a la interfaz de seguimiento mostrar el nuevo estado de la actividad y el porcentaje de avance actualizado sin que el usuario tenga que recargar la pantalla.

16

◄ - - **16. mostrarActualizarInformacion()**

La interfaz de seguimiento presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF044-GESTIONAR CUENTAS DE USUARIO | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Administrador	I_Administracion	C_Administracion	E_Usuario

BLOQUE 1 · El Administrador selecciona “Administración” desde el menú de usuario (Pasos 1–2)

1 — **1. seleccionarAdministracionMenu(administracion)** ——►

El administrador selecciona “Administración” desde el menú de usuario en la interfaz de administración, utilizando los elementos: Administración.

2 — **2. enviarGestionarCuentas(administracion)** ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las opciones de administración disponibles (Pasos 3–4)

3 — ◄—— **3. mostrarOpcionesAdministracion()**

El control de administración ordena a la interfaz de administración mostrar las opciones de administración disponibles.

4 — ◄ - - **4. mostrarGestionarCuentas()**

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El Administrador selecciona “Gestión de usuarios” (Pasos 5–6)

5

5. seleccionarGestionUsuarios(gestionUsuarios) ——►

El administrador selecciona "Gestión de usuarios" en la interfaz de administración, utilizando los elementos: Gestión de usuarios.

6

6. enviarGestionarCuentas(gestionUsuarios) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la lista de usuarios registrados y la opción "Nuevo usuario" (Pasos 7–8)

7

◄—— 7. mostrarListaUsuarios(nuevoUsuario)

El control de administración ordena a la interfaz de administración mostrar la lista de usuarios registrados y la opción "Nuevo usuario".

8

◄ - - 8. mostrarGestionarCuentas()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El Administrador selecciona "Nuevo usuario" para crear una cuenta, o selecciona "Ver más información" para consultar una cuenta existente (Pasos 9–10)

9

9. seleccionarNuevoUsuario(nuevoUsuario, verInformacion) ——►

El administrador selecciona "Nuevo usuario" para crear una cuenta, o selecciona "Ver más información" para consultar una cuenta existente en la interfaz de administración, utilizando los elementos: Nuevo usuario, Ver más información.

10

10. enviarGestionarCuentas(nuevoUsuario) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · Para una nueva cuenta, el sistema muestra el formulario y el Administrador ingresa los datos requeridos y asigna el rol correspondiente; para una cuenta existente, el sistema muestra sus datos y las opciones “Editar” y “Desactivar” o “Activar” (Pasos 11–13)

11

11. pararNuevaCuenta() ↺

El control de administración realiza internamente la lógica necesaria para parar una nueva cuenta, el sistema muestra el formulario y el Administrador ingresa los datos requeridos y asigna el rol correspondiente; para una cuenta existente, el sistema muestra sus datos y las opciones “Editar” y “Desactivar” o “Activar”.

12

◄—— 12. mostrarResultadoGestionarCuentas()

El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.

13

◄ - - 13. mostrarGestionarCuentas()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El Administrador completa o modifica la información y confirma la operación seleccionada (Pasos 14–15)

14

14. confirmarOperacion() ——►

El administrador completa o modifica la información y confirma la operación seleccionada en la interfaz de administración.

15

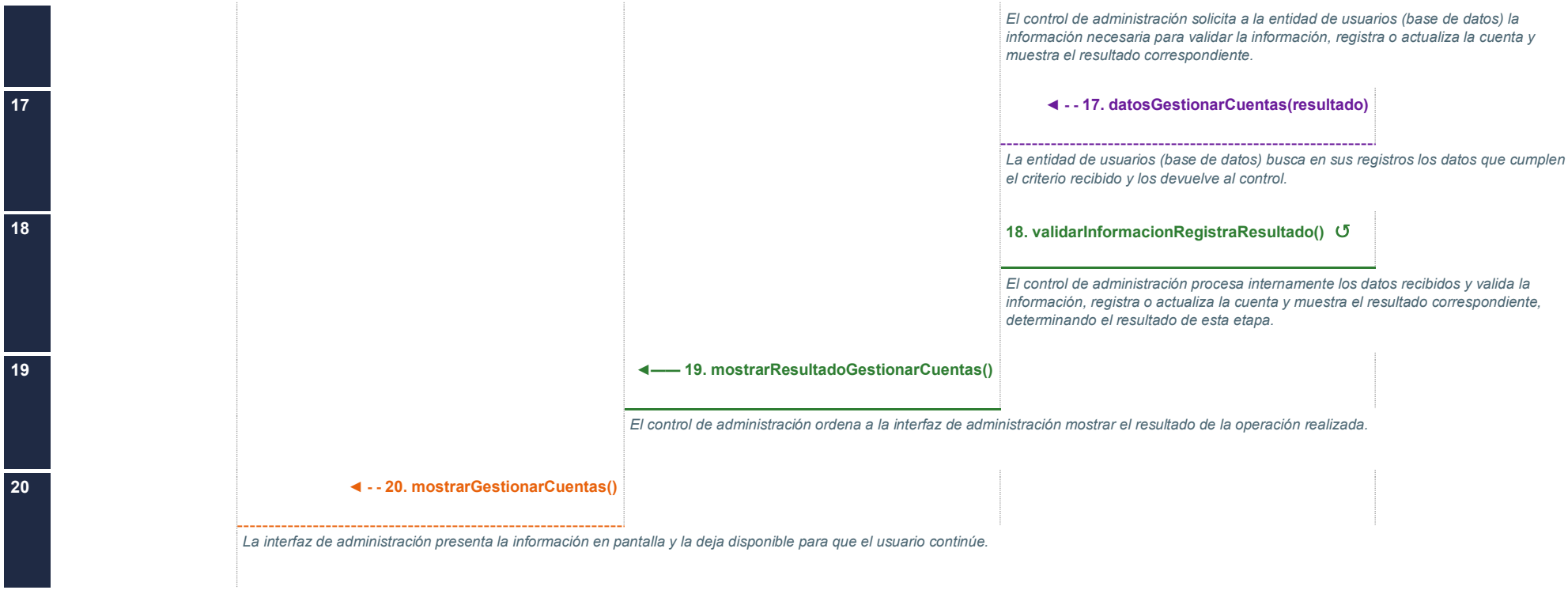
15. enviarGestionarCuentas(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema valida la información, registra o actualiza la cuenta y muestra el resultado correspondiente (Pasos 16–20)

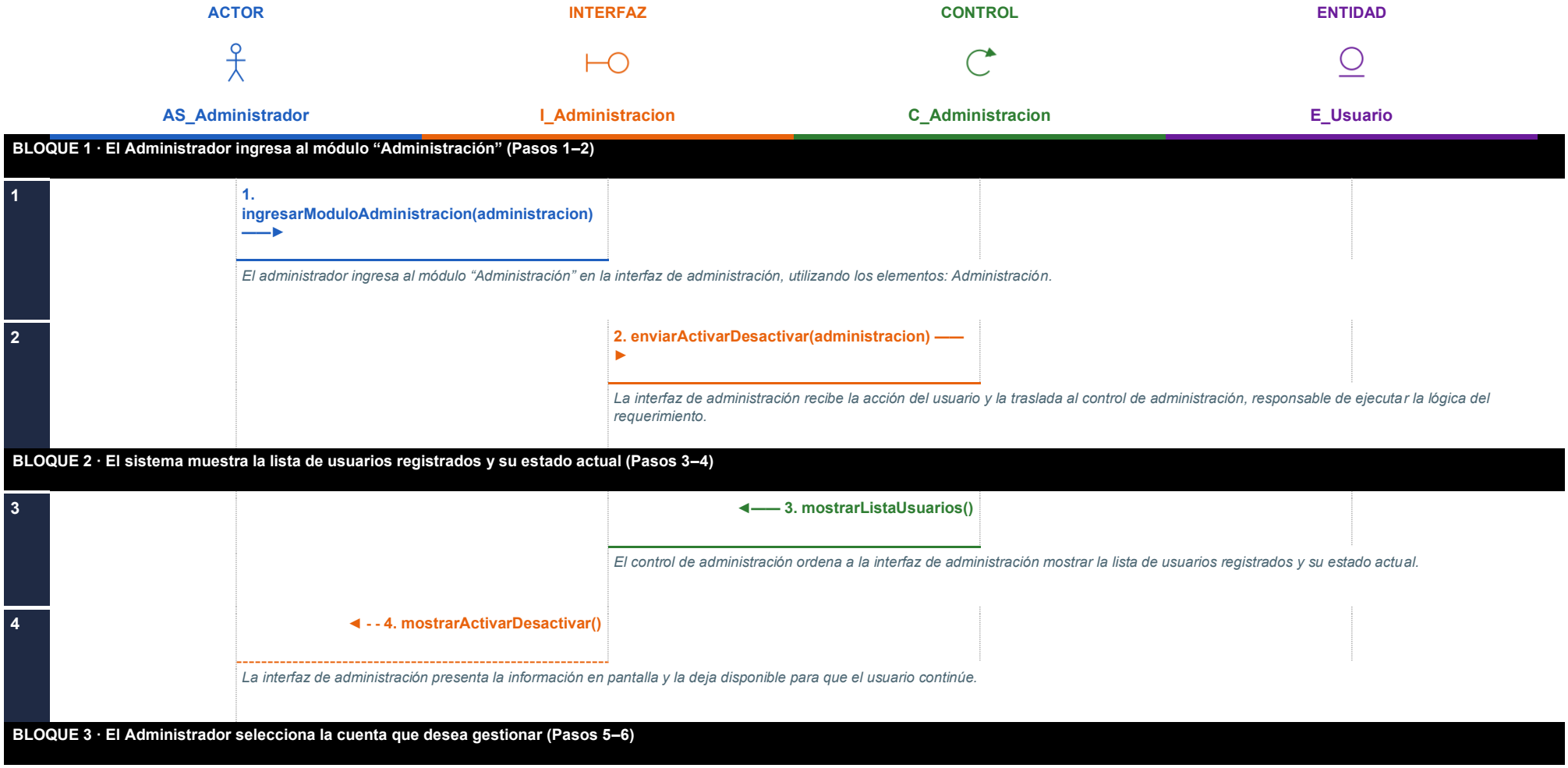
16

16. validarInformacionRegistra(criterio) ——►


El control de administración solicita a la entidad de usuarios (base de datos) la información necesaria para validar la información, registra o actualiza la cuenta y muestra el resultado correspondiente.
17
◄ - - 17. datosGestionarCuentas(resultado)
La entidad de usuarios (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.
18
18. validarInformacionRegistraResultado()
El control de administración procesa internamente los datos recibidos y valida la información, registra o actualiza la cuenta y muestra el resultado correspondiente, determinando el resultado de esta etapa.
19
◄—— 19. mostrarResultadoGestionarCuentas()
El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.
20
◄ - - 20. mostrarGestionarCuentas()
La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF045-ACTIVAR O DESACTIVAR CUENTA DE USUARIO | ESCENARIO DE ÉXITO



5

5. seleccionarCuentaDesea() ——►

El administrador selecciona la cuenta que desea gestionar en la interfaz de administración.

6

6. enviarActivarDesactivar(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la información de la cuenta y su estado actual (Pasos 7–8)

7

◄—— 7. mostrarInformacionCuenta()

El control de administración ordena a la interfaz de administración mostrar la información de la cuenta y su estado actual.

8

◄ - - 8. mostrarActivarDesactivar()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El Administrador selecciona la opción “Activar” o “Desactivar” (Pasos 9–10)

9

9. seleccionarOpcionActivar(activar, desactivar) ——►

El administrador selecciona la opción “Activar” o “Desactivar” en la interfaz de administración, utilizando los elementos: Activar, Desactivar.

10

10. enviarActivarDesactivar(activar) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema solicita confirmar la acción (Pasos 11–12)

11 ◄—— 11. solicitarConfirmarAccion()

El control de administración ordena a la interfaz de administración solicitar confirmar la acción.

12 ◄ - - 12. mostrarActivarDesactivar()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El Administrador confirma la modificación (Pasos 13–14)

13 13. confirmarOperacion() ——►

El administrador confirma la modificación en la interfaz de administración.

14 14. enviarActivarDesactivar(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema actualiza el estado de la cuenta y muestra un mensaje de confirmación (Pasos 15–18)

15 15. actualizarEstadoCuenta(datos) ——►

El control de administración solicita a la entidad de usuarios (base de datos) actualizar el estado de la cuenta y muestra un mensaje de confirmación.

16 ◄ - - 16. operacionConfirmada(estado)

La entidad de usuarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

17 ◄—— 17. mostrarResultadoActivarDesactivar()

El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.

◄ - - 18. mostrarActivarDesactivar()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF046-GESTIONAR ESTADO DE MASCOTAS | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Administrador	I_Administracion	C_Administracion	E_Mascota

BLOQUE 1 · El Administrador selecciona "Administración" desde el menú de usuario (Pasos 1–2)

1. **1. seleccionarAdministracionMenu(administracion) ——►**

El administrador selecciona "Administración" desde el menú de usuario en la interfaz de administración, utilizando los elementos: Administración.

2. **2. enviarGestionarEstado(administracion) ——►**

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las opciones de administración disponibles (Pasos 3–4)

3. **◄—— 3. mostrarOpcionesAdministracion()**

El control de administración ordena a la interfaz de administración mostrar las opciones de administración disponibles.

4. **◄ - - 4. mostrarGestionarEstado()**

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El Administrador selecciona "Gestión de mascotas" (Pasos 5–6)

5

5. seleccionarGestionMascotas(gestionMascotas) ——►

El administrador selecciona “Gestión de mascotas” en la interfaz de administración, utilizando los elementos: Gestión de mascotas.

6

6. enviarGestionarEstado(gestionMascotas) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra una barra de búsqueda para localizar una mascota registrada (Pasos 7–8)

7

◄—— 7. mostrarBarraBusqueda()

El control de administración ordena a la interfaz de administración mostrar una barra de búsqueda para localizar una mascota registrada.

8

◄ - - 8. mostrarGestionarEstado()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El Administrador ingresa el nombre, código o dato de identificación de la mascota (Pasos 9–10)

9

9. ingresarNombreCodigo() ——►

El administrador ingresa el nombre, código o dato de identificación de la mascota en la interfaz de administración.

10

◄ - - 10. camposCompletados()

La interfaz de administración refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 6 · El sistema muestra la mascota encontrada con sus datos principales y su estado actual (Pasos 11–12)

11 ◄—— 11. mostrarMascotaEncontrada()

El control de administración ordena a la interfaz de administración mostrar la mascota encontrada con sus datos principales y su estado actual.

12 ◄ - - 12. mostrarGestionarEstado()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El Administrador selecciona “Desactivar” o “Activar”, según el estado actual de la mascota, y confirma la operación (Pasos 13–14)

13 **13. seleccionarDesactivarActivar(desactivar, activar) ——►**

El administrador selecciona “Desactivar” o “Activar”, según el estado actual de la mascota, y confirma la operación en la interfaz de administración, utilizando los elementos: Desactivar, Activar.

14 **14. enviarGestionarEstado(desactivar) ——►**

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema actualiza el estado de la mascota y muestra el resultado de la operación (Pasos 15–18)

15 **15. actualizarEstadoMascota(datos) ——►**

El control de administración solicita a la entidad de mascotas (base de datos) actualizar el estado de la mascota y muestra el resultado de la operación.

16 ◄ - - 16. operacionConfirmada(estado)

La entidad de mascotas (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

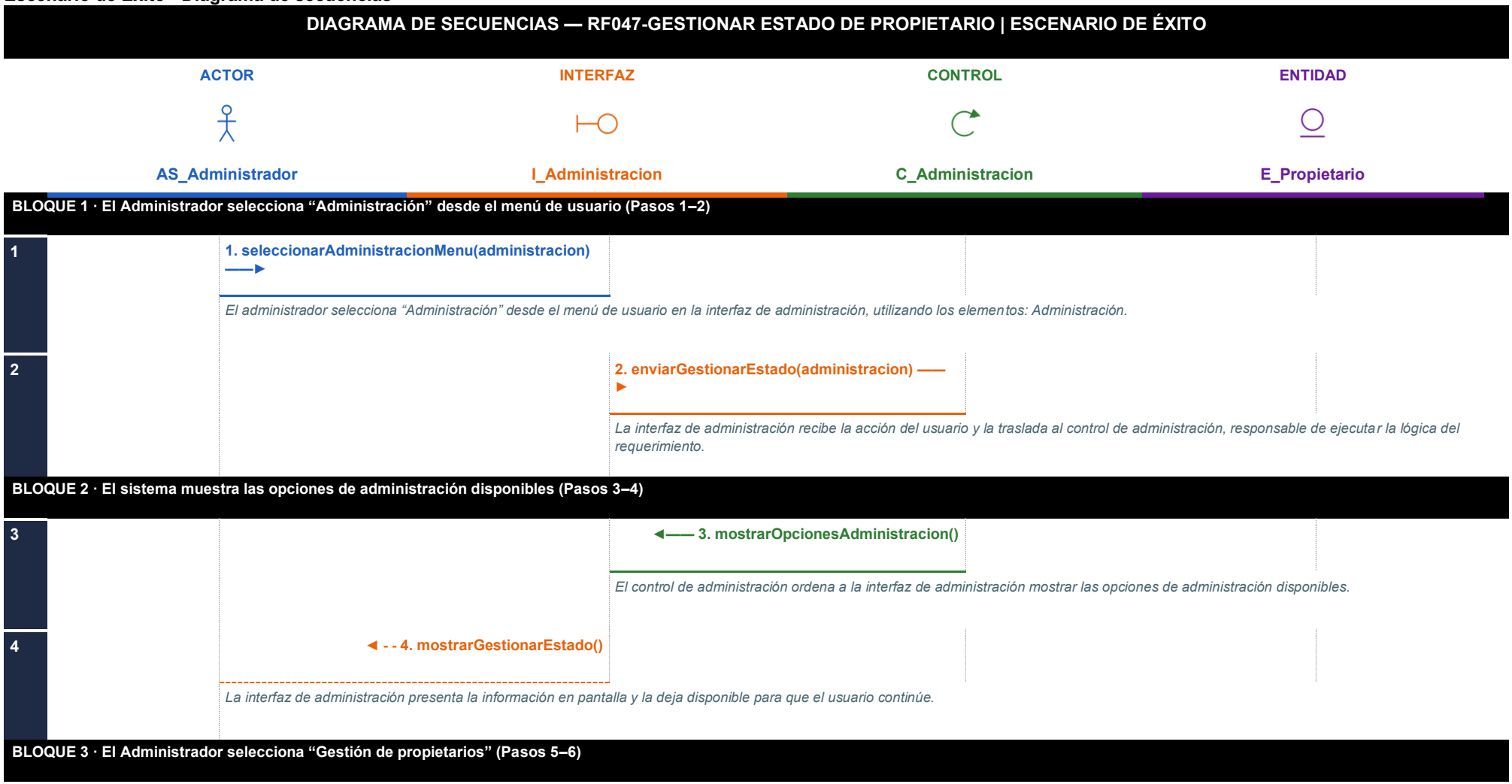
17 ◄—— 17. mostrarResultadoGestionarEstado()

El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.

◄ - - 18. mostrarGestionarEstado()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias



5

5. seleccionarGestionPropietarios(gestionPropietarios) ——►

El administrador selecciona “Gestión de propietarios” en la interfaz de administración, utilizando los elementos: Gestión de propietarios.

6

6. enviarGestionarEstado(gestionPropietarios) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra una barra de búsqueda para localizar un propietario registrado (Pasos 7–8)

7

◄—— 7. mostrarBarraBusqueda()

El control de administración ordena a la interfaz de administración mostrar una barra de búsqueda para localizar un propietario registrado.

8

◄ - - 8. mostrarGestionarEstado()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El Administrador ingresa el nombre, documento o dato de identificación del propietario (Pasos 9–10)

9

9. ingresarNombreDocumento() ——►

El administrador ingresa el nombre, documento o dato de identificación del propietario en la interfaz de administración.

10

◄ - - 10. camposCompletados()

La interfaz de administración refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 6 · El sistema muestra el propietario encontrado con sus datos principales y su estado actual (Pasos 11–12)

11 ◄—— 11. mostrarPropietarioEncontrado()

El control de administración ordena a la interfaz de administración mostrar el propietario encontrado con sus datos principales y su estado actual.

12 ◄ - - 12. mostrarGestionarEstado()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El Administrador selecciona “Desactivar” o “Activar”, según el estado actual del propietario, y confirma la operación (Pasos 13–14)

13 **13. seleccionarDesactivarActivar(desactivar, activar) ——►**

El administrador selecciona “Desactivar” o “Activar”, según el estado actual del propietario, y confirma la operación en la interfaz de administración, utilizando los elementos: Desactivar, Activar.

14 **14. enviarGestionarEstado(desactivar) ——►**

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema actualiza el estado del propietario y muestra el resultado de la operación (Pasos 15–18)

15 **15. actualizarEstadoPropietario(datos) ——►**

El control de administración solicita a la entidad de propietarios (base de datos) actualizar el estado del propietario y muestra el resultado de la operación.

16 **◄ - - 16. operacionConfirmada(estado)**

La entidad de propietarios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

17 **◄—— 17. mostrarResultadoGestionarEstado()**

El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.

◄ - - 18. mostrarGestionarEstado()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF048-CONSULTAR RESUMEN GENERAL | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Administrador	I_Administracion	C_Seguimiento	E_Atencion

BLOQUE 1 · El Administrador selecciona “Administración” desde el menú de usuario (Pasos 1–2)

1 — **1. seleccionarAdministracionMenu(administracion)** ——▶ (AS_Administrador → I_Administracion)

El administrador selecciona “Administración” desde el menú de usuario en la interfaz de administración, utilizando los elementos: Administración.

2 — **2. enviarConsultarResumen(administracion) ——▶** (I_Administracion → C_Seguimiento)

La interfaz de administración recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las opciones de administración disponibles (Pasos 3–4)

3 — **◀—— 3. mostrarOpcionesAdministracion()** (C_Seguimiento → I_Administracion)

El control de seguimiento ordena a la interfaz de administración mostrar las opciones de administración disponibles.

4 — **◀ - - 4. mostrarConsultarResumen()** (I_Administracion → AS_Administrador)

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El Administrador selecciona “Resumen general” (Pasos 5–6)

5

5. seleccionarResumenGeneral(resumenGeneral) ——►

El administrador selecciona “Resumen general” en la interfaz de administración, utilizando los elementos: Resumen general.

6

6. enviarConsultarResumen(resumenGeneral) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema consulta la información registrada de usuarios, mascotas, propietarios, atenciones y actividades (Pasos 7–9)

7

7. consultarInformacionRegistrada(criterio) ——►

El control de seguimiento solicita a la entidad de atenciones (base de datos) la información necesaria para consultar la información registrada de usuarios, mascotas, propietarios, atenciones y actividades.

8

◄ - - 8. datosConsultarResumen(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

9

9. consultarInformacionRegistradaResultado() ↺

El control de seguimiento procesa internamente los datos recibidos y consulta la información registrada de usuarios, mascotas, propietarios, atenciones y actividades, determinando el resultado de esta etapa.

BLOQUE 5 · El sistema calcula las cantidades correspondientes a cada categoría y estado (Pasos 10–12)

10

10. calcularCantidadesCorrespondientes(criterio) ——►

El control de seguimiento solicita a la entidad de atenciones (base de datos) la información necesaria para calcular las cantidades correspondientes a cada categoría y estado.

11 **◄ - - 11. datosConsultarResumen(resultado)**

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

12 **12. calcularCantidadesCorrespondientesResultado() ↺**

El control de seguimiento procesa internamente los datos recibidos y calcula las cantidades correspondientes a cada categoría y estado, determinando el resultado de esta etapa.

BLOQUE 6 · El sistema muestra los indicadores generales organizados por Usuarios, Mascotas, Propietarios, Atenciones y Actividades (Pasos 13–14)

13 **◄—— 13. mostrarIndicadoresGenerales()**

El control de seguimiento ordena a la interfaz de administración mostrar los indicadores generales organizados por Usuarios, Mascotas, Propietarios, Atenciones y Actividades.

14 **◄ - - 14. mostrarConsultarResumen()**

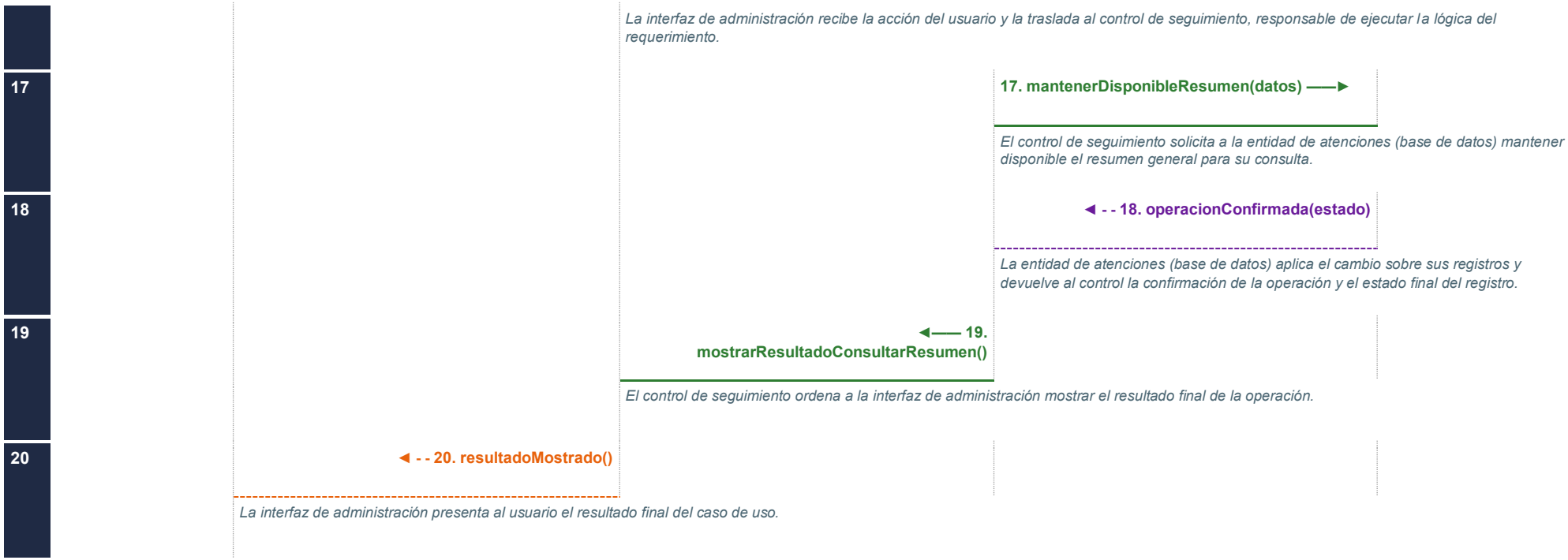
La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El Administrador consulta los indicadores mostrados. El sistema mantiene disponible el resumen general para su consulta (Pasos 15–20)

15 **15. consultarIndicadoresMostrados() ——►**

El administrador consulta los indicadores mostrados en la interfaz de administración.

16 **16. enviarConsultarResumen(solicitud) ——►**


La interfaz de administración recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.
17
17. mantenerDisponibleResumen(datos) ——►
El control de seguimiento solicita a la entidad de atenciones (base de datos) mantener disponible el resumen general para su consulta.
18
◄ - - 18. operacionConfirmada(estado)
La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.
19
◄—— 19. mostrarResultadoConsultarResumen()
El control de seguimiento ordena a la interfaz de administración mostrar el resultado final de la operación.
20
◄ - - 20. resultadoMostrado()
La interfaz de administración presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF049-CONSULTAR INDICADORES HISTÓRICOS | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Administrador	I_Administracion	C_Seguimiento	E_Atencion

BLOQUE 1 · El Administrador selecciona “Administración” desde el menú de usuario (Pasos 1–2)

1 — **1. seleccionarAdministracionMenu(administracion) ——►** (AS_Administrador → I_Administracion)

El administrador selecciona “Administración” desde el menú de usuario en la interfaz de administración, utilizando los elementos: Administración.

2 — **2. enviarConsultarIndicadores(administracion) ——►** (I_Administracion → C_Seguimiento)

La interfaz de administración recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las opciones de administración disponibles (Pasos 3–4)

3 — **◄—— 3. mostrarOpcionesAdministracion()** (C_Seguimiento → I_Administracion)

El control de seguimiento ordena a la interfaz de administración mostrar las opciones de administración disponibles.

4 — **◄ - - 4. mostrarConsultarIndicadores()** (I_Administracion → AS_Administrador)

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El Administrador selecciona “Indicadores históricos” (Pasos 5–6)

5

5. seleccionarIndicadoresHistoricos(indicadoresHistoricos) ——►

El administrador selecciona “Indicadores históricos” en la interfaz de administración, utilizando los elementos: Indicadores históricos.

6

6. enviarConsultarIndicadores(indicadoresHistoricos) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra los indicadores históricos, los filtros de periodo y el filtro por estado (Pasos 7–8)

7

◄—— 7. mostrarIndicadoresHistoricos()

El control de seguimiento ordena a la interfaz de administración mostrar los indicadores históricos, los filtros de periodo y el filtro por estado.

8

◄ - - 8. mostrarConsultarIndicadores()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El Administrador selecciona el periodo y, opcionalmente, el estado que desea consultar (Pasos 9–10)

9

9. seleccionarPeriodoOpcionalmente() ——►

El administrador selecciona el periodo y, opcionalmente, el estado que desea consultar en la interfaz de administración.

10

10. enviarConsultarIndicadores(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema actualiza los indicadores y la representación gráfica de acuerdo con los filtros seleccionados (Pasos 11–12)

11

11. actualizarIndicadoresRepresentacion(datos) ——►

El control de seguimiento solicita a la entidad de atenciones (base de datos) actualizar los indicadores y la representación gráfica de acuerdo con los filtros seleccionados.

12

◄ - - 12. operacionConfirmada(estado)

La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 7 · El Administrador selecciona uno de los indicadores mostrados para consultar su detalle (Pasos 13–14)

13

13. seleccionarUnoIndicadores() ——►

El administrador selecciona uno de los indicadores mostrados para consultar su detalle en la interfaz de administración.

14

14. enviarConsultarIndicadores(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de seguimiento, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema muestra el detalle de las atenciones relacionadas con el indicador seleccionado (Pasos 15–16)

15

◄—— 15. mostrarDetalleAtenciones()

El control de seguimiento ordena a la interfaz de administración mostrar el detalle de las atenciones relacionadas con el indicador seleccionado.

16

◄ - - 16. mostrarConsultarIndicadores()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF050-CONSULTAR CAMBIOS REALIZADOS | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Administrador	I_Administracion	C_Administracion	E_HistorialCambio

BLOQUE 1 · El Administrador selecciona “Administración” desde el menú de usuario (Pasos 1–2)

1 — **1. seleccionarAdministracionMenu(administracion)** ——►

El administrador selecciona “Administración” desde el menú de usuario en la interfaz de administración, utilizando los elementos: Administración.

2 — **2. enviarConsultarCambios(administracion)** ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las opciones de administración disponibles (Pasos 3–4)

3 — ◄—— **3. mostrarOpcionesAdministracion()**

El control de administración ordena a la interfaz de administración mostrar las opciones de administración disponibles.

4 — ◄ - - **4. mostrarConsultarCambios()**

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El Administrador selecciona “Cambios realizados” (Pasos 5–6)

5

5. seleccionarCambiosRealizados(cambiosRealizados) ——►

El administrador selecciona “Cambios realizados” en la interfaz de administración, utilizando los elementos: Cambios realizados.

6

6. enviarConsultarCambios(cambiosRealizados) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra el historial de cambios registrados y las opciones de búsqueda y filtrado (Pasos 7–8)

7

◄—— 7. mostrarHistorialCambios()

El control de administración ordena a la interfaz de administración mostrar el historial de cambios registrados y las opciones de búsqueda y filtrado.

8

◄ - - 8. mostrarConsultarCambios()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El Administrador selecciona los filtros que desea aplicar, como usuario, tipo de información, acción o periodo (Pasos 9–10)

9

9. seleccionarFiltrosDesea() ——►

El administrador selecciona los filtros que desea aplicar, como usuario, tipo de información, acción o periodo en la interfaz de administración.

10

10. enviarConsultarCambios(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema actualiza la lista mostrando los cambios que coinciden con los filtros seleccionados (Pasos 11–12)

11

11. actualizarListaMostrando(datos) ——►

El control de administración solicita a la entidad de historial de cambios (base de datos) actualizar la lista mostrando los cambios que coinciden con los filtros seleccionados.

12

◄ - - 12. operacionConfirmada(estado)

La entidad de historial de cambios (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

BLOQUE 7 · El Administrador selecciona un cambio para consultar su información detallada (Pasos 13–14)

13

13. seleccionarCambioConsultar() ——►

El administrador selecciona un cambio para consultar su información detallada en la interfaz de administración.

14

14. enviarConsultarCambios(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema muestra el usuario que realizó el cambio, la acción realizada, la información afectada y la fecha y hora del cambio (Pasos 15–16)

15

◄—— 15. mostrarUsuarioRealizo()

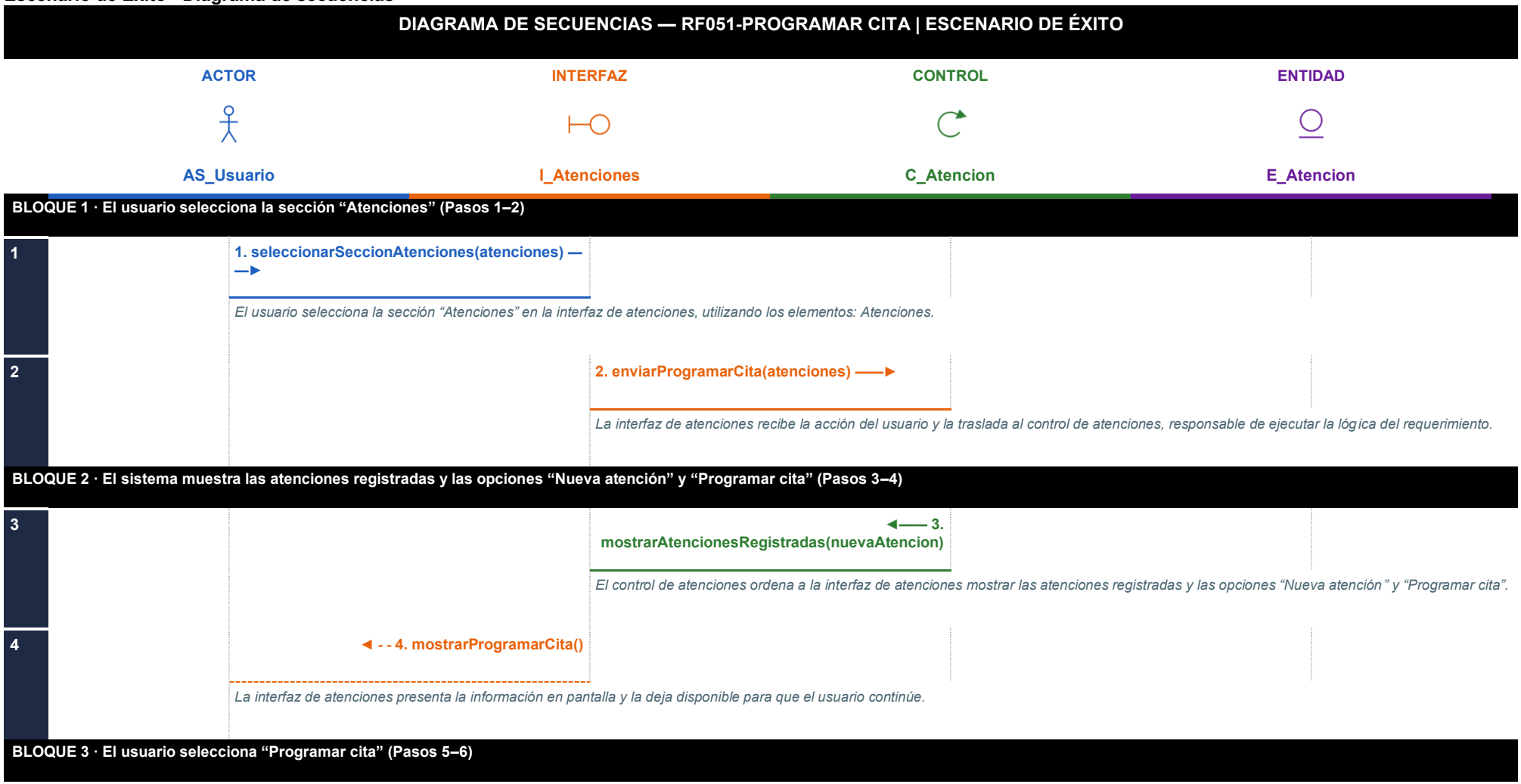
El control de administración ordena a la interfaz de administración mostrar el usuario que realizó el cambio, la acción realizada, la información afectada y la fecha y hora del cambio.

16

◄ - - 16. mostrarConsultarCambios()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias



5

5. seleccionarProgramarCita(programarCita) ——►

El usuario selecciona “Programar cita” en la interfaz de atenciones, utilizando los elementos: Programar cita.

6

6. enviarProgramarCita(programarCita) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra una barra de búsqueda para seleccionar la mascota y los campos “Fecha”, “Hora” y “Motivo” (Pasos 7–8)

7

◄—— 7. mostrarBarraBusqueda(fecha)

El control de atenciones ordena a la interfaz de atenciones mostrar una barra de búsqueda para seleccionar la mascota y los campos “Fecha”, “Hora” y “Motivo”.

8

◄ - - 8. mostrarProgramarCita()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El usuario selecciona la mascota e ingresa la fecha, la hora y el motivo (Pasos 9–11)

9

9. seleccionarMascotaE() ——►

El usuario selecciona la mascota e ingresa la fecha, la hora y el motivo en la interfaz de atenciones.

10

◄ - - 10. camposCompletados()

La interfaz de atenciones refleja en pantalla los datos ingresados para que el usuario pueda continuar.

11

11. enviarProgramarCita(solicitud) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El usuario selecciona “Guardar cita” (Pasos 12–13)

12

12. seleccionarGuardarCita(guardarCita) ——►

El usuario selecciona “Guardar cita” en la interfaz de atenciones, utilizando los elementos: Guardar cita.

13

13. enviarProgramarCita(guardarCita) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema valida que los campos obligatorios estén completos y que la fecha y hora sean posteriores al momento actual (Pasos 14–16)

14

14. validarCamposObligatorios(criterio) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) la información necesaria para validar que los campos obligatorios estén completos y que la fecha y hora sean posteriores al momento actual.

15

◄ - - 15. datosProgramarCita(resultado)

La entidad de atenciones (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

16

16. validarCamposObligatoriosResultado() ↺

El control de atenciones procesa internamente los datos recibidos y valida que los campos obligatorios estén completos y que la fecha y hora sean posteriores al momento actual, determinando el resultado de esta etapa.

BLOQUE 8 · El sistema registra la cita con estado “Programada” y muestra el mensaje: “La cita se programó correctamente.” (Pasos 17–20)

17

17. registrarCitaEstado(datos) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) registrar la cita con estado “Programada” y muestra el mensaje: “La cita se programó correctamente.”.

18 ◄ - - 18. operacionConfirmada(estado)

La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

19 ◄—— 19. mostrarResultadoProgramarCita()

El control de atenciones ordena a la interfaz de atenciones mostrar el resultado de la operación realizada.

20 20. vistaDesplegada() - -►

La interfaz de atenciones confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 9 · El sistema muestra la cita en el listado de atenciones con su fecha, hora y estado “Programada” (Pasos 21–22)

21 ◄—— 21. mostrarCitaListado(programada)

El control de atenciones ordena a la interfaz de atenciones mostrar la cita en el listado de atenciones con su fecha, hora y estado “Programada”.

22 22. vistaDesplegada() - -►

La interfaz de atenciones confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 10 · Para cancelar, el usuario selecciona una cita “Programada” y la opción “Cancelar cita” (Paso 23)

23 23. pararCancelarUsuario() ↺

El control de atenciones realiza internamente la lógica necesaria para parar cancelar, el usuario selecciona una cita “Programada” y la opción “Cancelar cita”.

BLOQUE 11 · El sistema solicita confirmación mediante "¿Deseas cancelar esta cita?", con las opciones "Confirmar" y "Cancelar" (Pasos 24–25)

24 ◄—— 24. solicitarConfirmacionDeseas(deseasCancelar)

El control de atenciones ordena a la interfaz de atenciones solicitar confirmación mediante "¿Deseas cancelar esta cita?", con las opciones "Confirmar" y "Cancelar".

25 ◄ - - 25. mostrarProgramarCita()

La interfaz de atenciones presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 12 · Si el usuario confirma, el sistema cambia el estado de la cita a "Cancelada" y muestra: "La cita se canceló correctamente." (Pasos 26–31)

26 26. confirmarOperacion() ——►

Si el usuario confirma en la interfaz de atenciones.

27 27. enviarProgramarCita(solicitud) ——►

La interfaz de atenciones recibe la acción del usuario y la traslada al control de atenciones, responsable de ejecutar la lógica del requerimiento.

28 28. cambiarEstadoCita(datos) ——►

El control de atenciones solicita a la entidad de atenciones (base de datos) cambiar el estado de la cita a "Cancelada" y muestra: "La cita se canceló correctamente.".

29 ◄ - - 29. operacionConfirmada(estado)

La entidad de atenciones (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

30 ◄—— 30. mostrarResultadoProgramarCita()

El control de atenciones ordena a la interfaz de atenciones mostrar el resultado de la operación realizada.

31

31. vistaDesplegada() - -►

La interfaz de atenciones confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 13 · Desde el detalle de una atención finalizada, el usuario puede seleccionar “Programar próxima cita”; el sistema abre el mismo formulario con la mascota ya seleccionada y guarda la fecha como próxima cita de la atención (Pasos 32–34)

32

32. desdeerDetalleAtencion() ↺

El control de atenciones realiza internamente la lógica necesaria para desdeer el detalle de una atención finalizada, el usuario puede seleccionar “Programar próxima cita”; el sistema abre el mismo formulario con la mascota ya seleccionada y guarda la fecha como próxima cita de la atención.

33

◄—— 33. mostrarResultadoProgramarCita()

El control de atenciones ordena a la interfaz de atenciones mostrar el resultado final de la operación.

34

◄ - - 34. resultadoMostrado()

La interfaz de atenciones presenta al usuario el resultado final del caso de uso.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF052-GESTIONAR CATÁLOGO DE MEDICAMENTOS | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Administrador	I_Administracion	C_Administracion	E_Medicamento

BLOQUE 1 · El Administrador selecciona “Administración” desde el menú de usuario (Pasos 1–2)

1. **1. seleccionarAdministracionMenu(administracion) ——►** (AS_Administrador → I_Administracion)
   El administrador selecciona “Administración” desde el menú de usuario en la interfaz de administración, utilizando los elementos: Administración.
2. **2. enviarGestionarCatalogo(administracion) ——►** (I_Administracion → C_Administracion)
   La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las opciones de administración disponibles (Pasos 3–4)

3. **◄—— 3. mostrarOpcionesAdministracion()** (C_Administracion → I_Administracion)
   El control de administración ordena a la interfaz de administración mostrar las opciones de administración disponibles.
4. **◄ - - 4. mostrarGestionarCatalogo()** (I_Administracion → AS_Administrador)
   La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El Administrador selecciona “Catálogo de medicamentos” (Pasos 5–6)

5

5. seleccionarCatalogoMedicamentos(catalogoMedicamentos) ——►

El administrador selecciona “Catálogo de medicamentos” en la interfaz de administración, utilizando los elementos: Catálogo de medicamentos.

6

6. enviarGestionarCatalogo(catalogoMedicamentos) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la lista de medicamentos registrados con su nombre y concentración, una barra de búsqueda y la opción “Nuevo medicamento” (Pasos 7–8)

7

◄—— 7. mostrarListaMedicamentos(nuevoMedicamento)

El control de administración ordena a la interfaz de administración mostrar la lista de medicamentos registrados con su nombre y concentración, una barra de búsqueda y la opción “Nuevo medicamento”.

8

◄ - - 8. mostrarGestionarCatalogo()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El Administrador selecciona “Nuevo medicamento” o selecciona un medicamento existente y la opción “Editar” (Pasos 9–10)

9

9. seleccionarNuevoMedicamento(nuevoMedicamento, editar) ——►

El administrador selecciona “Nuevo medicamento” o selecciona un medicamento existente y la opción “Editar” en la interfaz de administración, utilizando los elementos: Nuevo medicamento, Editar.

10

10. enviarGestionarCatalogo(nuevoMedicamento) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema muestra el formulario con los campos “Nombre” y “Concentración” (con los datos actuales si es una edición) (Pasos 11–12)

11 ◄—— 11. mostrarFormularioCampos(nombre)

El control de administración ordena a la interfaz de administración mostrar el formulario con los campos “Nombre” y “Concentración” (con los datos actuales si es una edición).

12 ◄ - - 12. mostrarGestionarCatalogo()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El Administrador ingresa o modifica los datos y selecciona “Guardar” (Pasos 13–15)

13 13. ingresarModificaDatos(guardar) ——►

El administrador ingresa o modifica los datos y selecciona “Guardar” en la interfaz de administración.

14 ◄ - - 14. camposCompletados()

La interfaz de administración refleja en pantalla los datos ingresados para que el usuario pueda continuar.

15 15. enviarGestionarCatalogo(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema valida la información y muestra el mensaje: “¿Deseas guardar los cambios del medicamento?”, con las opciones “Confirmar” y “Cancelar” (Pasos 16–20)

16 16. validarInformacionMuestra(criterio) ——►

El control de administración solicita a la entidad de medicamentos (base de datos) la información necesaria para validar la información y muestra el mensaje: “¿Deseas guardar los cambios del medicamento?”, con las opciones “Confirmar” y “Cancelar”.

17

◄ - - 17. datosGestionarCatalogo(resultado)

La entidad de medicamentos (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

18

18. validarInformacionMuestraResultado() ↺

El control de administración procesa internamente los datos recibidos y valida la información y muestra el mensaje: "¿Deseas guardar los cambios del medicamento?", con las opciones "Confirmar" y "Cancelar", determinando el resultado de esta etapa.

19

◄—— 19. mostrarResultadoGestionarCatalogo()

El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.

20

◄ - - 20. mostrarGestionarCatalogo()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 9 · Si el Administrador confirma, el sistema guarda el medicamento y muestra: "El medicamento se guardó correctamente." (Pasos 21–26)

21

21. confirmarOperacion() ——►

Si el Administrador confirma en la interfaz de administración.

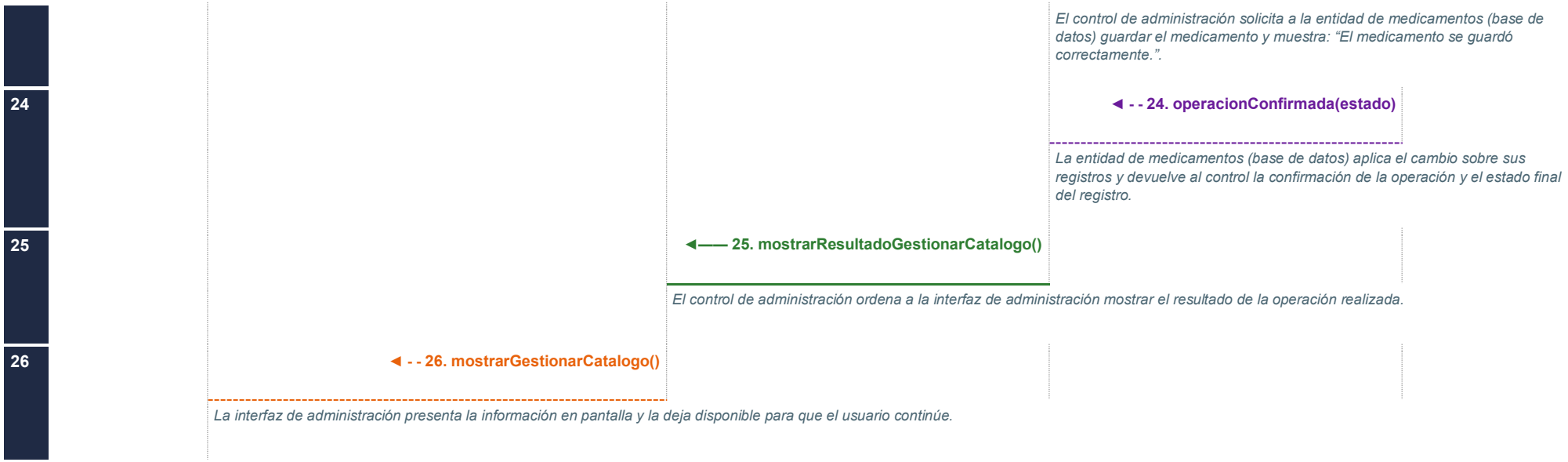
22

22. enviarGestionarCatalogo(solicitud) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

23

23. guardarMedicamentoMuestra(datos) ——►


El control de administración solicita a la entidad de medicamentos (base de datos) guardar el medicamento y muestra: “El medicamento se guardó correctamente.”.
24
◄ - - 24. operacionConfirmada(estado)
La entidad de medicamentos (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.
25
◄—— 25. mostrarResultadoGestionarCatalogo()
El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.
26
◄ - - 26. mostrarGestionarCatalogo()
La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF053-REGISTRAR POSOLOGÍA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_NuevaActividad	C_Actividad	E_Actividad

BLOQUE 1 · El usuario ingresa al detalle de una atención activa y selecciona “Nueva actividad” (Pasos 1–4)

1. **1. ingresarDetalleAtencion(nuevaActividad) ——►**

   El usuario ingresa al detalle de una atención activa y selecciona “Nueva actividad” en la interfaz de nueva actividad, utilizando los elementos: Nueva actividad.

2. **2. enviarRegistrarPosologia(nuevaActividad) ——►**

   La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

3. **◄ - - 3. solicitudRecibida()**

   El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

4. **◄ - - 4. mostrarSiguientePaso()**

   La interfaz de nueva actividad habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 2 · El usuario selecciona el tipo de actividad “Medicación” (Pasos 5–6)

5. **5. seleccionarTipoActividad(medicacion) ——►**

El usuario selecciona el tipo de actividad “Medicación” en la interfaz de nueva actividad, utilizando los elementos: Medicación.

6

6. enviarRegistrarPosologia(medicacion) ——►

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra el formulario con los campos “Medicamento”, “Dosis”, “Frecuencia”, “Vía de administración”, “Duración del tratamiento”, “Condición de ingesta”, “Fecha prevista” y “Observaciones” (Pasos 7–8)

7

◄—— 7. mostrarFormularioCampos(medicamento)

El control de actividades ordena a la interfaz de nueva actividad mostrar el formulario con los campos “Medicamento”, “Dosis”, “Frecuencia”, “Vía de administración”, “Duración del tratamiento”, “Condición de ingesta”, “Fecha prevista” y “Observaciones”.

8

◄ - - 8. mostrarRegistrarPosologia()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona un medicamento del catálogo e ingresa los datos de posología (Pasos 9–11)

9

9. seleccionarMedicamentoCatalogo() ——►

El usuario selecciona un medicamento del catálogo e ingresa los datos de posología en la interfaz de nueva actividad.

10

◄ - - 10. camposCompletados()

La interfaz de nueva actividad refleja en pantalla los datos ingresados para que el usuario pueda continuar.

11

11. enviarRegistrarPosologia(solicitud) ——►

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · Si la actividad incluye otro medicamento, el usuario selecciona “Agregar otro medicamento” y repite el paso 4 (Paso 12)

12

12. seleccionarAgregarOtro() ↺

El control de actividades realiza internamente la lógica necesaria para seleccionar “Agregar otro medicamento” y repite el paso 4.

BLOQUE 6 · El usuario selecciona “Registrar actividad” (Pasos 13–14)

13

13. seleccionarRegistrarActividad(registrarActividad) ——►

El usuario selecciona “Registrar actividad” en la interfaz de nueva actividad, utilizando los elementos: Registrar actividad.

14

14. enviarRegistrarPosologia(registrarActividad) ——►

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema valida que el medicamento y la dosis estén completos y que los datos tengan formato válido (Pasos 15–17)

15

15. validarMedicamentoDosis(criterio) ——►

El control de actividades solicita a la entidad de actividades (base de datos) la información necesaria para validar que el medicamento y la dosis estén completos y que los datos tengan formato válido.

16

◄ - - 16. datosRegistrarPosologia(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

17

17. validarMedicamentoDosisResultado() ↺

El control de actividades procesa internamente los datos recibidos y valida que el medicamento y la dosis estén completos y que los datos tengan formato válido, determinando el resultado de esta etapa.

BLOQUE 8 · El sistema registra la actividad con estado “Pendiente” y muestra la posología de cada medicamento en el detalle de la actividad (Pasos 18–21)

18

18. registrarActividadEstado(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) registrar la actividad con estado “Pendiente” y muestra la posología de cada medicamento en el detalle de la actividad.

19

◄ - - 19. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

20

◄—— 20. mostrarResultadoRegistrarPosologia()

El control de actividades ordena a la interfaz de nueva actividad mostrar el resultado de la operación realizada.

21

◄ - - 21. mostrarRegistrarPosologia()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF054-REGISTRAR DIETA INDICADA | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Usuario	I_NuevaActividad	C_Actividad	E_Actividad

BLOQUE 1 · El usuario ingresa al detalle de una atención activa y selecciona “Nueva actividad” (Pasos 1–4)

1	1. ingresarDetalleAtencion(nuevaActividad) ——►			*El usuario ingresa al detalle de una atención activa y selecciona “Nueva actividad” en la interfaz de nueva actividad, utilizando los elementos: Nueva actividad.*
2		2. enviarRegistrarDieta(nuevaActividad) ——►		*La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.*
3			◄ - - 3. solicitudRecibida()	*El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.*
4	◄ - - 4. mostrarSiguientePaso()			*La interfaz de nueva actividad habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.*

BLOQUE 2 · El usuario selecciona el tipo de actividad “Dieta” (Pasos 5–6)

5	5. seleccionarTipoActividad(dieta) ——►			

El usuario selecciona el tipo de actividad “Dieta” en la interfaz de nueva actividad, utilizando los elementos: Dieta.

6 **6. enviarRegistrarDieta(dieta) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra el campo “Tipo de dieta”, la fecha prevista y observaciones (Pasos 7–8)

7 **◄—— 7. mostrarCampoTipo(tipoDieta)**

El control de actividades ordena a la interfaz de nueva actividad mostrar el campo “Tipo de dieta”, la fecha prevista y observaciones.

8 **◄ - - 8. mostrarRegistrarDieta()**

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario selecciona un tipo de dieta (Pasos 9–10)

9 **9. seleccionarTipoDieta() ——►**

El usuario selecciona un tipo de dieta en la interfaz de nueva actividad.

10 **10. enviarRegistrarDieta(solicitud) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema muestra la restricción nutricional de la dieta y los alimentos permitidos agrupados por categoría (Proteína, Vegetales, Carbohidratos y Otro) (Pasos 11–12)

11 **◄—— 11. mostrarRestriccionNutricional()**

El control de actividades ordena a la interfaz de nueva actividad mostrar la restricción nutricional de la dieta y los alimentos permitidos agrupados por categoría (Proteína, Vegetales, Carbohidratos y Otro).

12 ◄ - - 12. mostrarRegistrarDieta()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario selecciona “Registrar actividad” (Pasos 13–14)

13 13. seleccionarRegistrarActividad(registrarActividad) ——►

El usuario selecciona “Registrar actividad” en la interfaz de nueva actividad, utilizando los elementos: Registrar actividad.

14 14. enviarRegistrarDieta(registrarActividad) ——►

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 7 · El sistema valida que se haya seleccionado un tipo de dieta (Pasos 15–17)

15 15. validarHayaSeleccionado(criterio) ——►

El control de actividades solicita a la entidad de actividades (base de datos) la información necesaria para validar que se haya seleccionado un tipo de dieta.

16 ◄ - - 16. datosRegistrarDieta(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

17 17. validarHayaSeleccionadoResultado() ↺

El control de actividades procesa internamente los datos recibidos y valida que se haya seleccionado un tipo de dieta, determinando el resultado de esta etapa.

BLOQUE 8 · El sistema registra la actividad con estado “Pendiente” y muestra la dieta indicada en el detalle de la atención (Pasos 18–21)

18

18. registrarActividadEstado(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) registrar la actividad con estado “Pendiente” y muestra la dieta indicada en el detalle de la atención.

19

◄ - - 19. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

20

◄—— 20. mostrarResultadoRegistrarDieta()

El control de actividades ordena a la interfaz de nueva actividad mostrar el resultado de la operación realizada.

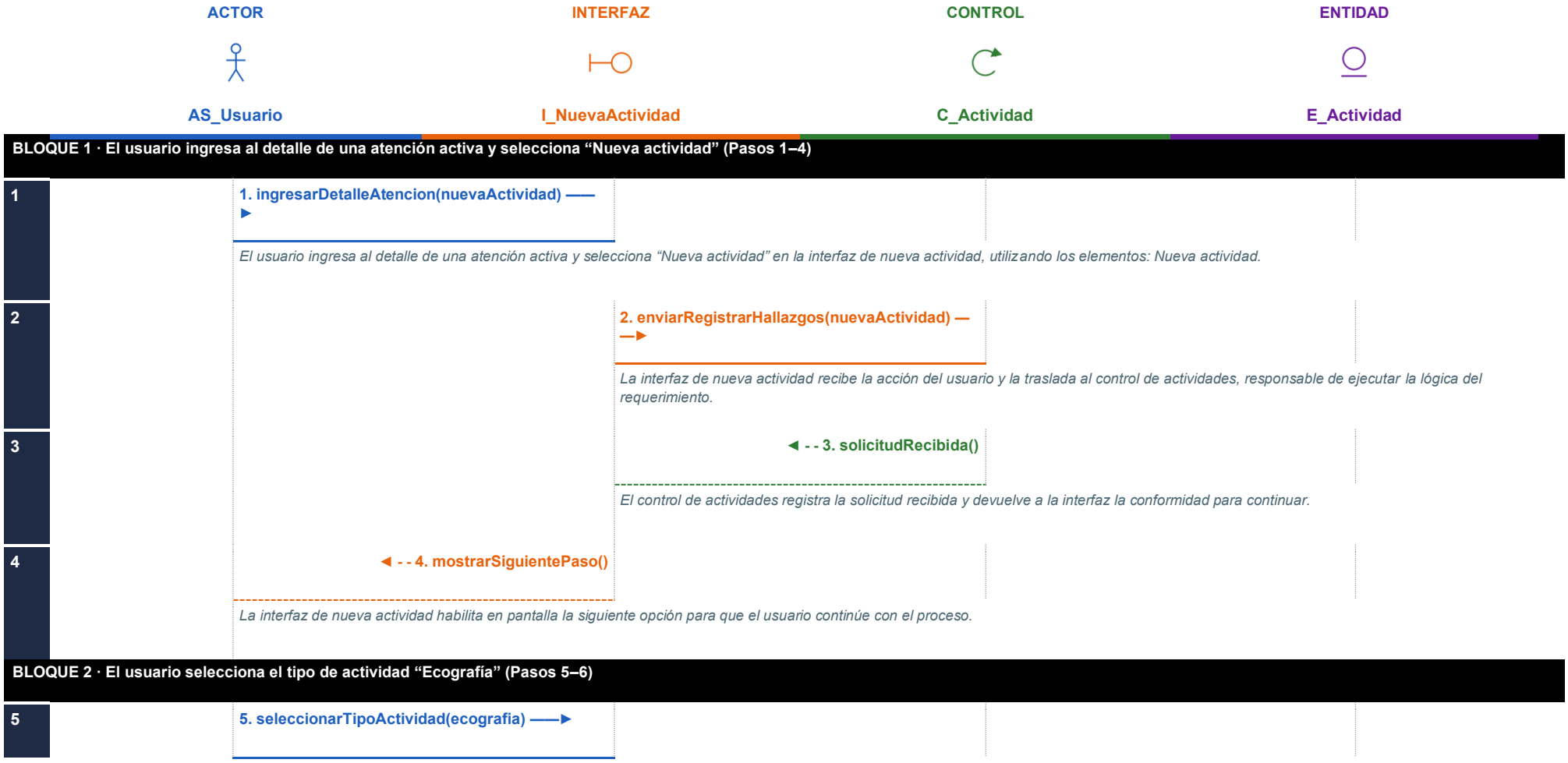
21

◄ - - 21. mostrarRegistrarDieta()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF055-REGISTRAR HALLAZGOS POR ÓRGANO | ESCENARIO DE ÉXITO



El usuario selecciona el tipo de actividad “Ecografía” en la interfaz de nueva actividad, utilizando los elementos: Ecografía.

6 **6. enviarRegistrarHallazgos(ecografia) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 3 · El sistema muestra los campos “Área a evaluar”, “Fecha prevista” y “Observaciones”, y la opción “Agregar órgano” (Pasos 7–8)

7 **◄—— 7. mostrarCamposArea(areaEvaluar)**

El control de actividades ordena a la interfaz de nueva actividad mostrar los campos “Área a evaluar”, “Fecha prevista” y “Observaciones”, y la opción “Agregar órgano”.

8 **◄ - - 8. mostrarRegistrarHallazgos()**

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 4 · El usuario ingresa el área a evaluar y selecciona “Agregar órgano” (Pasos 9–11)

9 **9. ingresarAreaEvaluar(agregarOrgano) ——►**

El usuario ingresa el área a evaluar y selecciona “Agregar órgano” en la interfaz de nueva actividad.

10 **◄ - - 10. camposCompletados()**

La interfaz de nueva actividad refleja en pantalla los datos ingresados para que el usuario pueda continuar.

11 **11. enviarRegistrarHallazgos(solicitud) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 5 · El sistema muestra la lista de órganos del catálogo (Pasos 12–13)

12 ◄—— 12. mostrarListaOrganos()

El control de actividades ordena a la interfaz de nueva actividad mostrar la lista de órganos del catálogo.

13 ◄ - - 13. mostrarRegistrarHallazgos()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 6 · El usuario selecciona un órgano; el sistema muestra los campos “Hallazgos”, “Mediciones específicas” y “Conclusión sugerida” para ese órgano (Pasos 14–17)

14 **14. seleccionarOrganoSistema(hallazgos, medicionesEspecificas) ——►**

El usuario selecciona un órgano; el sistema muestra los campos “Hallazgos”, “Mediciones específicas” y “Conclusión sugerida” para ese órgano en la interfaz de nueva actividad, utilizando los elementos: Hallazgos, Mediciones específicas, Conclusión sugerida.

15 **15. enviarRegistrarHallazgos(hallazgos) ——►**

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

16 ◄ - - 16. solicitudRecibida()

El control de actividades registra la solicitud recibida y devuelve a la interfaz la conformidad para continuar.

17 ◄ - - 17. mostrarSiguientePaso()

La interfaz de nueva actividad habilita en pantalla la siguiente opción para que el usuario continúe con el proceso.

BLOQUE 7 · El usuario completa los campos y repite los pasos 4 a 6 para cada órgano evaluado (Pasos 18–19)

18

18. completarCamposRepite() ——►

El usuario completa los campos y repite los pasos 4 a 6 para cada órgano evaluado en la interfaz de nueva actividad.

19

◄ - - 19. camposCompletados()

La interfaz de nueva actividad refleja en pantalla los datos ingresados para que el usuario pueda continuar.

BLOQUE 8 · El usuario selecciona “Registrar actividad” (Pasos 20–21)

20

20. seleccionarRegistrarActividad(registrarActividad) ——►

El usuario selecciona “Registrar actividad” en la interfaz de nueva actividad, utilizando los elementos: Registrar actividad.

21

21. enviarRegistrarHallazgos(registrarActividad) ——►

La interfaz de nueva actividad recibe la acción del usuario y la traslada al control de actividades, responsable de ejecutar la lógica del requerimiento.

BLOQUE 9 · El sistema valida que exista al menos un órgano con hallazgos (Pasos 22–24)

22

22. validarExistaMenos(criterio) ——►

El control de actividades solicita a la entidad de actividades (base de datos) la información necesaria para validar que exista al menos un órgano con hallazgos.

23

◄ - - 23. datosRegistrarHallazgos(resultado)

La entidad de actividades (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

24

24. validarExistaMenosResultado() ↺

El control de actividades procesa internamente los datos recibidos y valida que exista al menos un órgano con hallazgos, determinando el resultado de esta etapa.

BLOQUE 10 · El sistema registra la actividad con estado “Pendiente” y muestra los hallazgos agrupados por órgano en el detalle de la actividad (Pasos 25–28)

25

25. registrarActividadEstado(datos) ——►

El control de actividades solicita a la entidad de actividades (base de datos) registrar la actividad con estado “Pendiente” y muestra los hallazgos agrupados por órgano en el detalle de la actividad.

26

◄ - - 26. operacionConfirmada(estado)

La entidad de actividades (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

27

◄—— 27. mostrarResultadoRegistrarHallazgos()

El control de actividades ordena a la interfaz de nueva actividad mostrar el resultado de la operación realizada.

28

◄ - - 28. mostrarRegistrarHallazgos()

La interfaz de nueva actividad presenta la información en pantalla y la deja disponible para que el usuario continúe.

Escenario de Éxito · Diagrama de secuencias

DIAGRAMA DE SECUENCIAS — RF056-GESTIONAR TIPOS DE ACTIVIDAD | ESCENARIO DE ÉXITO

ACTOR	INTERFAZ	CONTROL	ENTIDAD
AS_Administrador	I_Administracion	C_Administracion	E_TipoActividad

BLOQUE 1 · El Administrador selecciona “Administración” desde el menú de usuario (Pasos 1–2)

1 — **1. seleccionarAdministracionMenu(administracion)** ——►

El administrador selecciona “Administración” desde el menú de usuario en la interfaz de administración, utilizando los elementos: Administración.

2 — **2. enviarGestionarTipos(administracion)** ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 2 · El sistema muestra las opciones de administración disponibles (Pasos 3–4)

3 — ◄—— **3. mostrarOpcionesAdministracion()**

El control de administración ordena a la interfaz de administración mostrar las opciones de administración disponibles.

4 — ◄ - - **4. mostrarGestionarTipos()**

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 3 · El Administrador selecciona “Tipos de actividad” (Pasos 5–6)

5

5. seleccionarTiposActividad(tiposActividad) ——►

El administrador selecciona “Tipos de actividad” en la interfaz de administración, utilizando los elementos: Tipos de actividad.

6

6. enviarGestionarTipos(tiposActividad) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 4 · El sistema muestra la lista de tipos de actividad con su nombre, descripción y estado, y la opción “Nuevo tipo” (Pasos 7–8)

7

◄—— 7. mostrarListaTipos(nuevoTipo)

El control de administración ordena a la interfaz de administración mostrar la lista de tipos de actividad con su nombre, descripción y estado, y la opción “Nuevo tipo”.

8

◄ - - 8. mostrarGestionarTipos()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 5 · El Administrador selecciona “Nuevo tipo” o selecciona un tipo existente y la opción “Editar” (Pasos 9–10)

9

9. seleccionarNuevoTipo(nuevoTipo, editar) ——►

El administrador selecciona “Nuevo tipo” o selecciona un tipo existente y la opción “Editar” en la interfaz de administración, utilizando los elementos: Nuevo tipo, Editar.

10

10. enviarGestionarTipos(nuevoTipo) ——►

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 6 · El sistema muestra el formulario con los campos “Nombre”, “Descripción” y “¿Requiere medicamento?” (Pasos 11–12)

11 ◄—— 11. mostrarFormularioCampos(nombre)

El control de administración ordena a la interfaz de administración mostrar el formulario con los campos “Nombre”, “Descripción” y “¿Requiere medicamento?”.

12 ◄ - - 12. mostrarGestionarTipos()

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 7 · El Administrador completa o modifica los datos y selecciona “Guardar” (Pasos 13–14)

13 **13. completarModificaDatos(guardar) ——►**

El administrador completa o modifica los datos y selecciona “Guardar” en la interfaz de administración, utilizando los elementos: Guardar.

14 **14. enviarGestionarTipos(guardar) ——►**

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

BLOQUE 8 · El sistema valida la información y muestra el mensaje: “¿Deseas guardar los cambios del tipo de actividad?”, con las opciones “Confirmar” y “Cancelar” (Pasos 15–19)

15 **15. validarInformacionMuestra(criterio) ——►**

El control de administración solicita a la entidad de tipos de actividad (base de datos) la información necesaria para validar la información y muestra el mensaje: “¿Deseas guardar los cambios del tipo de actividad?”, con las opciones “Confirmar” y “Cancelar”.

16 **◄ - - 16. datosGestionarTipos(resultado)**

La entidad de tipos de actividad (base de datos) busca en sus registros los datos que cumplen el criterio recibido y los devuelve al control.

17 **17. validarInformacionMuestraResultado() ↺**

El control de administración procesa internamente los datos recibidos y valida la información y muestra el mensaje: "¿Deseas guardar los cambios del tipo de actividad?", con las opciones "Confirmar" y "Cancelar", determinando el resultado de esta etapa.

18 **◄—— 18. mostrarResultadoGestionarTipos()**

El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.

19 **◄ - - 19. mostrarGestionarTipos()**

La interfaz de administración presenta la información en pantalla y la deja disponible para que el usuario continúe.

BLOQUE 9 · Si el Administrador confirma, el sistema guarda el tipo de actividad y muestra: "El tipo de actividad se guardó correctamente." (Pasos 20–25)

20 **20. confirmarOperacion() ——►**

Si el Administrador confirma en la interfaz de administración.

21 **21. enviarGestionarTipos(solicitud) ——►**

La interfaz de administración recibe la acción del usuario y la traslada al control de administración, responsable de ejecutar la lógica del requerimiento.

22 **22. guardarTipoActividad(datos) ——►**

El control de administración solicita a la entidad de tipos de actividad (base de datos) guardar el tipo de actividad y muestra: "El tipo de actividad se guardó correctamente.".

23 **◄ - - 23. operacionConfirmada(estado)**

La entidad de tipos de actividad (base de datos) aplica el cambio sobre sus registros y devuelve al control la confirmación de la operación y el estado final del registro.

24 ◄—— 24. mostrarResultadoGestionarTipos()

El control de administración ordena a la interfaz de administración mostrar el resultado de la operación realizada.

25 25. vistaDesplegada() - -►

La interfaz de administración confirma al control que la información fue desplegada, para que este continúe con el siguiente paso.

BLOQUE 10 · Para cambiar su disponibilidad, el Administrador selecciona “Desactivar” o “Activar” y confirma la operación; un tipo desactivado deja de ofrecerse al registrar nuevas actividades, pero se conservan las ya registradas (Pasos 26–28)

26 26. pararCambiarDisponibilidad() ↺

El control de administración realiza internamente la lógica necesaria para parar cambiar su disponibilidad, el Administrador selecciona “Desactivar” o “Activar” y confirma la operación; un tipo desactivado deja de ofrecerse al registrar nuevas actividades, pero se conservan las ya registradas.

27 ◄—— 27. mostrarResultadoGestionarTipos()

El control de administración ordena a la interfaz de administración mostrar el resultado final de la operación.

28 ◄ - - 28. resultadoMostrado()

La interfaz de administración presenta al usuario el resultado final del caso de uso.

2. Uso de recursos del lenguaje elegido

2.1. Lista de librerías utilizadas

PetCare está desarrollada en JavaScript con React Native y Expo. Para el lenguaje JavaScript la rúbrica espera librerías equivalentes en cuatro propósitos: procesamiento de datos, manejo de archivos, registros (logging) y seguridad. La siguiente tabla resume las librerías de apoyo del proyecto. Más abajo se detalla cada una con su justificación, sus consideraciones de seguridad y un fragmento del código real donde se usa.

1. Resumen por propósito

Propósito	Librería (versión)	Uso en PetCare	Equivalente en la rúbrica
Procesamiento de datos	@supabase/supabase-js (^2.117)	Consultas, registro y actualización de datos en PostgreSQL desde los DAO	axios / fetch
Manejo de archivos	expo-print (~57.0) + expo-sharing (~57.0)	Exportar el detalle de una atención a PDF y compartirlo	xlsx
Registros (logging)	react-native-logs (^5.6)	Registro centralizado de errores con niveles y ocultamiento de datos sensibles	winston
Seguridad	expo-secure-store (~57.0)	Guardar la sesión cifrada (Android Keystore / iOS Keychain)	bcrypt / helmet

Complementan a estas librerías: **Supabase Auth** (contraseñas con hash bcrypt en el servidor y tokens JWT), **jest + @testing-library/react-native** (pruebas unitarias, TDD), **@react-native-community/datetimepicker** (ingreso de fechas validado) y **expo-router** (navegación por pantallas).

2. Detalle por librería

2.1 Procesamiento de datos: @supabase/supabase-js

Aspecto	Detalle
Propósito	Cliente oficial para conectarse a la base de datos PostgreSQL de Supabase. Todas las consultas se hacen desde la capa DAO (mascotaDAO, propietarioDAO, atencionesDAO, actividadesDAO, perfilDAO) y ninguna pantalla consulta la base de datos directamente.
Contribución a la calidad	Las consultas se construyen con métodos encadenados (from, select, eq, or, order), sin concatenar SQL a mano. Devuelve los errores de forma estructurada ({ data, error }) para manejarlos en cada DAO.

Aspecto	Detalle
Consideraciones de seguridad	Las consultas son parametrizadas, lo que evita la inyección SQL. Cada petición viaja por HTTPS con el token JWT del usuario. Las políticas RLS (Row Level Security) de la base de datos solo permiten operar a usuarios autenticados con cuenta activa y rol Administrador o Asistente. La app solo contiene la clave pública (publishable), nunca la clave de servicio.

Archivo: src/dao/mascotaDAO.js

```
export async function buscarMascotas(termino) {
  const texto = validarTerminoBusqueda(termino);
  if (!texto) return consultarMascotas();

  const patron = `%${texto}%`;

  const { data, error } = await supabase
    .from('mascotas')
    .select(relacionPropietario)
    .eq('estado', 'Activo')
    .or(`nombre.ilike.${patron},numero_historia.ilike.${patron},especie.ilike.${patron}`)
    .order('nombre', { ascending: true });

  if (error) {
    logger.error('Error al buscar mascotas:', error);
    throw error;
  }
  return data ?? [];
}
```

2.2 Manejo de archivos: expo-print y expo-sharing

Aspecto	Detalle
Propósito	expo-print genera un archivo PDF a partir de una plantilla HTML con los datos de la mascota, la atención y sus actividades. expo-sharing abre el menú del sistema para enviar el PDF por WhatsApp, correo o Drive. Se usa en el botón «Exportar PDF» de la pantalla Detalle de atención.
Contribución a la calidad	La lógica de generación está en un servicio aparte (exportarPdfServicio.js) y no en la pantalla, lo que respeta MVC y SRP. Tiene pruebas unitarias en src/tests/librerias/exportarPdf.test.js.
Consideraciones de seguridad	Todos los textos se escapan con escaparHtml() antes de insertarse en el HTML, lo que evita la inyección de código. El PDF se crea en la carpeta temporal privada de la app y solo sale del dispositivo si el usuario decide compartirlo. Si no hay atención cargada, se lanza un error controlado.

Archivo: src/services/exportarPdfServicio.js

```
export async function exportarAtencionPdf({ atencion, mascota, actividades }) {
  if (!atencion) {
    throw new Error('No hay una atención cargada para exportar.');
  }

  const html = construirHtmlAtencion({ atencion, mascota, actividades });
  const { uri } = await Print.printToFileAsync({ html });

  log.info('PDF generado para la atención', atencion.atencion_id);

  if (await Sharing.isAvailableAsync()) {
    await Sharing.shareAsync(uri, {
      mimeType: 'application/pdf',
      dialogTitle: 'Compartir detalle de atención',
    });
  }
  return uri;
}
```

2.3 Registros (logging): react-native-logs

Aspecto	Detalle
Propósito	Logger centralizado (src/utils/logger.js) con niveles debug, info, warn y error. Reemplaza los 116 console.error/console.log que había repartidos por las pantallas, servicios y DAO.
Contribución a la calidad	Un único punto de registro con fecha, hora, nivel y módulo (logger.modulo('ExportarPDF')). En desarrollo muestra todos los niveles y en la APK de producción solo advertencias y errores, lo que reduce el ruido y mejora el rendimiento.
Consideraciones de seguridad	Antes de registrar, la función sanitizar() reemplaza por *** las contraseñas, tokens, documentos de identidad y teléfonos, oculta los correos (jh***@utp.edu.pe) y elimina los tokens JWT. Así ningún dato personal queda expuesto en los registros. Está cubierto por 5 pruebas unitarias en src/tests/librerias/logger.test.js.

Archivo: src/utils/logger.js

```
const CLAVES_SENSIBLES =
  /pass|contrasena|token|secret|clave|correo|email|documento|dni|telefono/i;


export function sanitizar(valor) {
  if (typeof valor === 'string') {
    return valor.replace(PATRON_JWT, '***')
                .replace(PATRON_CORREO, (c) => ocultarCorreo(c));
  }
  if (typeof valor === 'object') {
    const copia = {};
    Object.keys(valor).forEach((clave) => {
      copia[clave] = CLAVES_SENSIBLES.test(clave) ? '***' : sanitizar(valor[clave]);
    });
    return copia;
  }
  return valor;
}


const registro = rnLogger.createLogger({
  severity: __DEV__ ? 'debug' : 'warn',
  transport: consoleTransport,
  printLevel: true, printDate: true,
});
```

2.4 Seguridad: expo-secure-store

Aspecto	Detalle
Propósito	Almacena la sesión del usuario (token de acceso y de actualización de Supabase) de forma cifrada en el dispositivo. Se integra como storage del cliente de Supabase en src/services/supabase.js.

Aspecto	Detalle
Contribución a la calidad	Mantiene la sesión activa (RF04) sin pedir el login en cada apertura, pero de forma segura. El adaptador almacenamientoSeguro.js divide la sesión en fragmentos, porque SecureStore admite valores de máximo 2 KB, y tiene 4 pruebas unitarias.
Consideraciones de seguridad	Antes la sesión se guardaba en AsyncStorage como texto plano. Ahora se cifra con el Android Keystore (en iOS, Keychain), de modo que otra app o alguien con acceso a los archivos del teléfono no puede leer el token. Al cerrar sesión (RF07) se eliminan todos los fragmentos. Las contraseñas nunca se guardan en el teléfono: las valida Supabase Auth, que las almacena con hash bcrypt.

Archivo: src/services/almacenamientoSeguro.js y src/services/supabase.js

```
const almacenamientoNativo = {
  async setItem(clave, valor) {
    await eliminarFragmentos(clave);
    const texto = String(valor);
    const total = Math.ceil(texto.length / TAMANO_FRAGMENTO);
    for (let i = 0; i < total; i += 1) {
      await SecureStore.setItemAsync(claveFragmento(clave, i),
        texto.slice(i * TAMANO_FRAGMENTO, (i + 1) * TAMANO_FRAGMENTO));
    }
    await SecureStore.setItemAsync(claveTotal(clave), String(total));
  },
  // getItem() une los fragmentos; removeItem() los elimina todos
};

// supabase.js
export const supabase = createClient(supabaseUrl, supabaseKey, {
  auth: {
    storage: almacenamientoSeguro,   // antes: AsyncStorage (sin cifrar)
    autoRefreshToken: true,
    persistSession: true,
  },
});
```

3. Consideraciones de seguridad en el tratamiento de la información

- **Autenticación:** Supabase Auth valida las credenciales y guarda las contraseñas con hash bcrypt. La app nunca almacena contraseñas.
- **Autorización:** las políticas RLS en PostgreSQL restringen cada tabla a usuarios autenticados, con cuenta activa y rol Administrador o Asistente. La función es_administrador_activo() protege la gestión de usuarios.
- **Sesión cifrada:** los tokens se guardan con expo-secure-store (Keystore/Keychain) y se renuevan automáticamente (autoRefreshToken).
- **Datos en tránsito:** toda la comunicación con Supabase usa HTTPS.
- **Prevención de inyección:** consultas parametrizadas con supabase-js (SQL) y escape de HTML en los PDF.
- **Registros sin datos personales:** el logger oculta contraseñas, tokens, correos, documentos y teléfonos.
- **Validación de entradas:** las validaciones en src/utils (validacionesAutenticacion, validacionesPropietario, validacionesAtencion) se ejecutan antes de enviar datos a la base de datos.
- **Claves:** la app solo incluye la clave pública (publishable) de Supabase. La clave de servicio no está en el código ni en el repositorio.

2.1.1. Resumen por propósito

Propósito	Librería (versión)	Uso en PetCare	Equivalente en la rúbrica
Procesamiento de datos	@supabase/supabase-js (^2.117)	Consultas, registro y actualización de datos en PostgreSQL desde los DAO	axios / fetch
Manejo de archivos	expo-print (~57.0) + expo-sharing (~57.0)	Exportar el detalle de una atención a PDF y compartirlo	xlsx
Registros (logging)	react-native-logs (^5.6)	Registro centralizado de errores con niveles y ocultamiento de datos sensibles	winston
Seguridad	expo-secure-store (~57.0)	Guardar la sesión cifrada (Android Keystore / iOS Keychain)	bcrypt / helmet

Complementan a estas librerías: **Supabase Auth** (contraseñas con hash bcrypt en el servidor y tokens JWT), **jest + @testing-library/react-native** (pruebas unitarias, TDD), **@react-native-community/datetimepicker** (ingreso de fechas validado) y **expo-router** (navegación por pantallas).

2. Detalle por librería

2.2.1 Procesamiento de datos: @supabase/supabase-js

Aspecto	Detalle
Propósito	Cliente oficial para conectarse a la base de datos PostgreSQL de Supabase. Todas las consultas se hacen desde la capa DAO (mascotaDAO, propietarioDAO, atencionesDAO, actividadesDAO, perfilDAO) y ninguna pantalla consulta la base de datos directamente.

Contribución a la calidad	Las consultas se construyen con métodos encadenados (from, select, eq, or, order), sin concatenar SQL a mano. Devuelve los errores de forma estructurada ({ data, error }) para manejarlos en cada DAO.
Consideraciones de seguridad	Las consultas son parametrizadas, lo que evita la inyección SQL. Cada petición viaja por HTTPS con el token JWT del usuario. Las políticas RLS (Row Level Security) de la base de datos solo permiten operar a usuarios autenticados con cuenta activa y rol Administrador o Asistente. La app solo contiene la clave pública (publishable), nunca la clave de servicio.

Archivo: `src/dao/mascotaDAO.js`

```
export async function buscarMascotas(termino) {
  const texto = validarTerminoBusqueda(termino);
  if (!texto) return consultarMascotas();

  const patron = `%${texto}%`;

  const { data, error } = await supabase
    .from('mascotas')
    .select(relacionPropietario)
    .eq('estado', 'Activo')
    .or(`nombre.ilike.${patron},numero_historia.ilike.${patron},especie.ilike.${patron}`)
    .order('nombre', { ascending: true });

  if (error) {
    logger.error('Error al buscar mascotas:', error);
    throw error;
  }
  return data ?? [];
}
```

2.2 Manejo de archivos: expo-print y expo-sharing

Aspecto	Detalle
Propósito	expo-print genera un archivo PDF a partir de una plantilla HTML con los datos de la mascota, la atención y sus actividades. expo-sharing abre el menú del sistema para enviar el PDF por WhatsApp, correo o Drive. Se usa en el botón «Exportar PDF» de la pantalla Detalle de atención.
Contribución a la calidad	La lógica de generación está en un servicio aparte (exportarPdfServicio.js) y no en la pantalla, lo que respeta MVC y SRP. Tiene pruebas unitarias en src/tests/librerias/exportarPdf.test.js.
Consideraciones de seguridad	Todos los textos se escapan con escaparHtml() antes de insertarse en el HTML, lo que evita la inyección de código. El PDF se crea en la carpeta temporal privada de la app y solo sale del dispositivo si el usuario decide compartirlo. Si no hay atención cargada, se lanza un error controlado.

Archivo: `src/services/exportarPdfServicio.js`

```
export async function exportarAtencionPdf({ atencion, mascota, actividades }) {
  if (!atencion) {
    throw new Error('No hay una atención cargada para exportar.');
  }

  const html = construirHtmlAtencion({ atencion, mascota, actividades });
  const { uri } = await Print.printToFileAsync({ html });

  log.info('PDF generado para la atención', atencion.atencion_id);

  if (await Sharing.isAvailableAsync()) {
```

```
    await Sharing.shareAsync(uri, {
      mimeType: 'application/pdf',
      dialogTitle: 'Compartir detalle de atención',
    });
  }
  return uri;
}
```

2.3 Registros (logging): react-native-logs

Aspecto	Detalle
Propósito	Logger centralizado (src/utils/logger.js) con niveles debug, info, warn y error. Reemplaza los 116 console.error/console.log que había repartidos por las pantallas, servicios y DAO.
Contribución a la calidad	Un único punto de registro con fecha, hora, nivel y módulo (logger.modulo('ExportarPDF')). En desarrollo muestra todos los niveles y en la APK de producción solo advertencias y errores, lo que reduce el ruido y mejora el rendimiento.
Consideraciones de seguridad	Antes de registrar, la función sanitizar() reemplaza por *** las contraseñas, tokens, documentos de identidad y teléfonos, oculta los correos (jh***@utp.edu.pe) y elimina los tokens JWT. Así ningún dato personal queda expuesto en los registros. Está cubierto por 5 pruebas unitarias en src/tests/librerias/logger.test.js.

Archivo: `src/utils/logger.js`

```
const CLAVES_SENSIBLES =
  /pass|contrasena|token|secret|clave|correo|email|documento|dni|telefono/i;

export function sanitizar(valor) {
  if (typeof valor === 'string') {
    return valor.replace(PATRON_JWT, '***')
                .replace(PATRON_CORREO, (c) => ocultarCorreo(c));
  }
  if (typeof valor === 'object') {
    const copia = {};
    Object.keys(valor).forEach((clave) => {
      copia[clave] = CLAVES_SENSIBLES.test(clave) ? '***' : sanitizar(valor[clave]);
    });
    return copia;
  }
  return valor;
}

const registro = rnLogger.createLogger({
  severity: __DEV__ ? 'debug' : 'warn',
  transport: consoleTransport,
  printLevel: true, printDate: true,
});
```

2.4 Seguridad: expo-secure-store

Aspecto	Detalle
Propósito	Almacena la sesión del usuario (token de acceso y de actualización de Supabase) de forma cifrada en el dispositivo. Se integra como storage del cliente de Supabase en src/services/supabase.js.

Contribución a la calidad	Mantiene la sesión activa (RF04) sin pedir el login en cada apertura, pero de forma segura. El adaptador almacenamientoSeguro.js divide la sesión en fragmentos, porque SecureStore admite valores de máximo 2 KB, y tiene 4 pruebas unitarias.
Consideraciones de seguridad	Antes la sesión se guardaba en AsyncStorage como texto plano. Ahora se cifra con el Android Keystore (en iOS, Keychain), de modo que otra app o alguien con acceso a los archivos del teléfono no puede leer el token. Al cerrar sesión (RF07) se eliminan todos los fragmentos. Las contraseñas nunca se guardan en el teléfono: las valida Supabase Auth, que las almacena con hash bcrypt.

Archivo: `src/services/almacenamientoSeguro.js y src/services/supabase.js`

```
const almacenamientoNativo = {
  async setItem(clave, valor) {
    await eliminarFragmentos(clave);
    const texto = String(valor);
    const total = Math.ceil(texto.length / TAMANO_FRAGMENTO);
    for (let i = 0; i < total; i += 1) {
      await SecureStore.setItemAsync(claveFragmento(clave, i),
        texto.slice(i * TAMANO_FRAGMENTO, (i + 1) * TAMANO_FRAGMENTO));
    }
    await SecureStore.setItemAsync(claveTotal(clave), String(total));
  },
  // getItem() une los fragmentos; removeItem() los elimina todos
};

// supabase.js
export const supabase = createClient(supabaseUrl, supabaseKey, {
  auth: {
    storage: almacenamientoSeguro,   // antes: AsyncStorage (sin cifrar)
    autoRefreshToken: true,
    persistSession: true,
  },
});
```

3. Consideraciones de seguridad en el tratamiento de la información

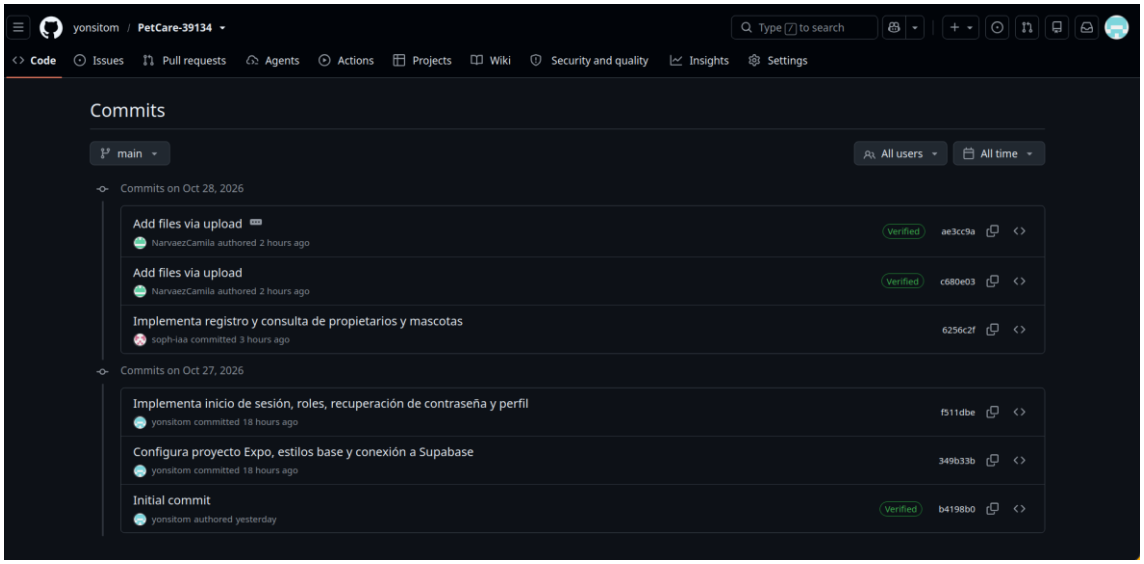
- **Autenticación:** Supabase Auth valida las credenciales y guarda las contraseñas con hash bcrypt. La app nunca almacena contraseñas.
- **Autorización:** las políticas RLS en PostgreSQL restringen cada tabla a usuarios autenticados, con cuenta activa y rol Administrador o Asistente. La función es_administrador_activo() protege la gestión de usuarios.
- **Sesión cifrada:** los tokens se guardan con expo-secure-store (Keystore/Keychain) y se renuevan automáticamente (autoRefreshToken).
- **Datos en tránsito:** toda la comunicación con Supabase usa HTTPS.
- **Prevención de inyección:** consultas parametrizadas con supabase-js (SQL) y escape de HTML en los PDF.
- **Registros sin datos personales:** el logger oculta contraseñas, tokens, correos, documentos y teléfonos.
- **Validación de entradas:** las validaciones en src/utils (validacionesAutenticacion, validacionesPropietario, validacionesAtencion) se ejecutan antes de enviar datos a la base de datos.
- **Claves:** la app solo incluye la clave pública (publishable) de Supabase. La clave de servicio no está en el código ni en el repositorio.

3. Control de versiones (GitHub)

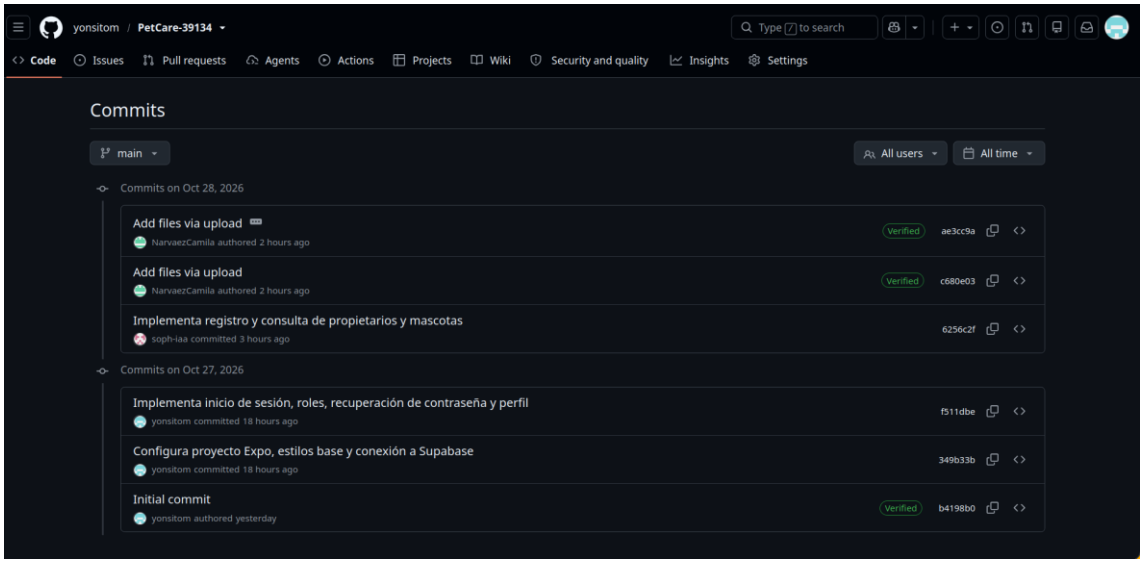
3.1. Enlace al repositorio

https://github.com/yonsitom/PetCare-39134.git

3.2. Captura del historial

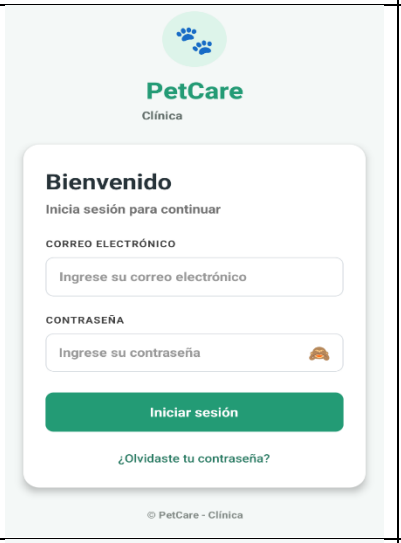


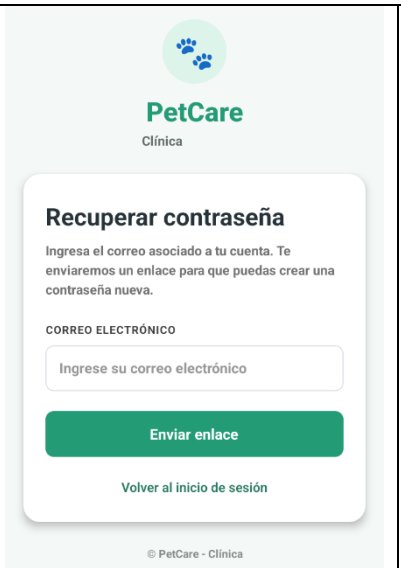
3.3. Captura de participantes

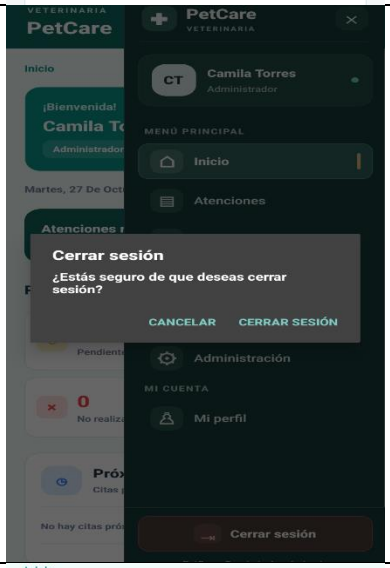


4. Implementación de interfaces gráficas

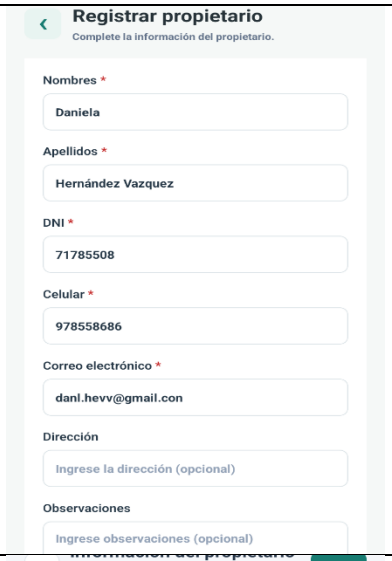
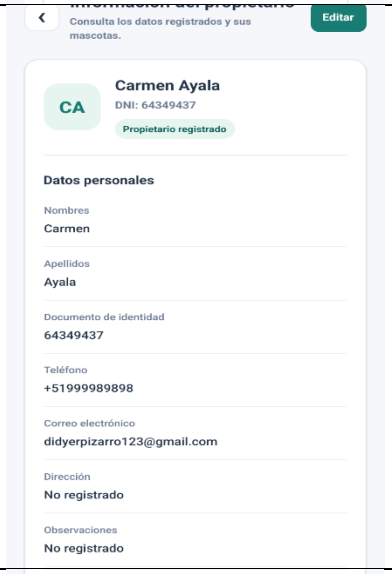
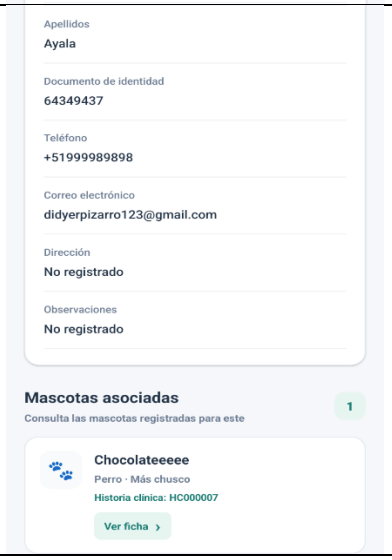
Como la aplicación ya está funcionando, la evidencia documental son las capturas de pantalla organizadas por requerimiento más una tabla de cobertura. No se repite la normalización de vistas (las correcciones realizadas a las observaciones de la entregada de avance 2).

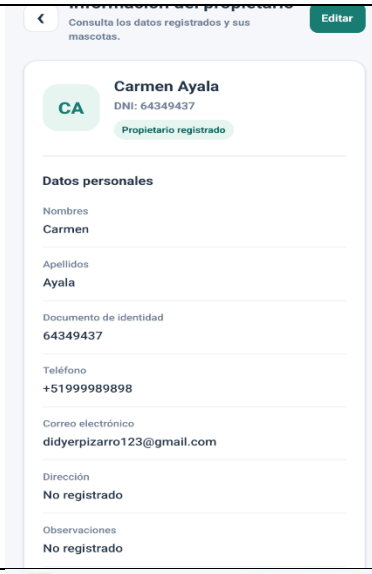
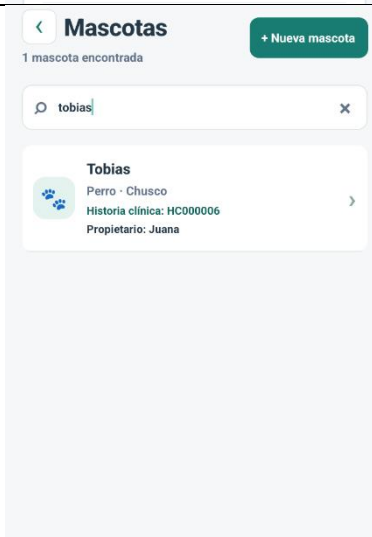
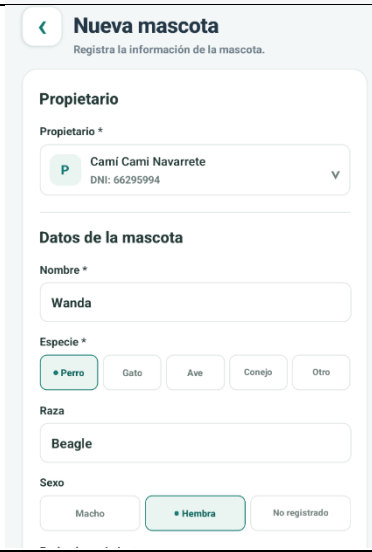
RF	Nombre del requerimiento	Pantalla implementada	Funciona
RF01	Iniciar sesión con correo electrónico y contraseña		Sí
RF02	Identificar el rol del usuario y mostrar sus funciones		Sí

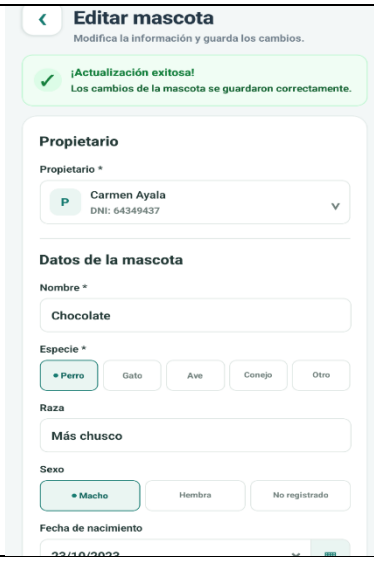
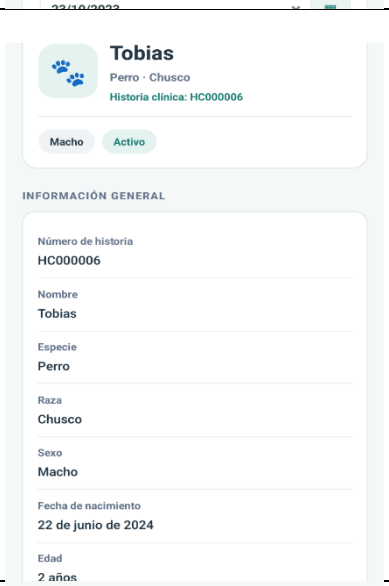
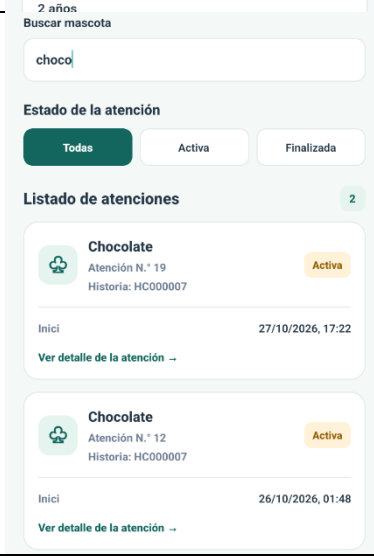
RF03	Recuperar la contraseña mediante el correo electrónico registrado		Sí
RF04	Mantener activa la sesión mientras las credenciales sean válidas		Sí
RF05	Consultar y actualizar la información del perfil del usuario		Sí

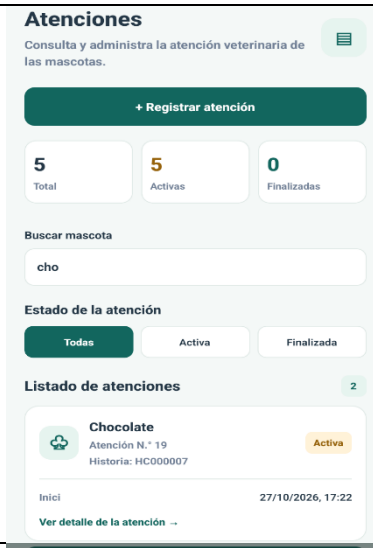
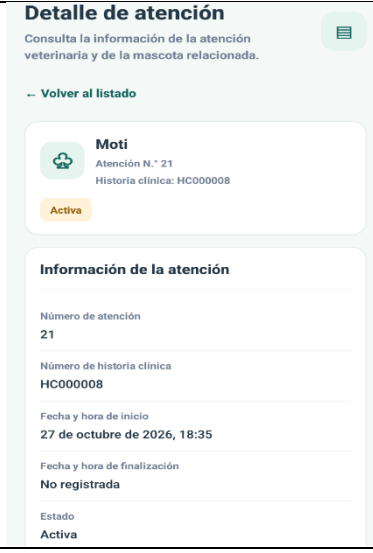
RF06	Cambiar la contraseña del usuario		Sí
RF07	Cerrar sesión de forma segura		Sí
RF08	Visualizar el panel principal con el resumen de atenciones y actividades		Sí

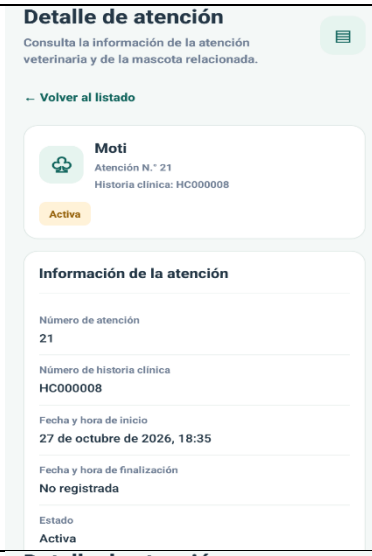
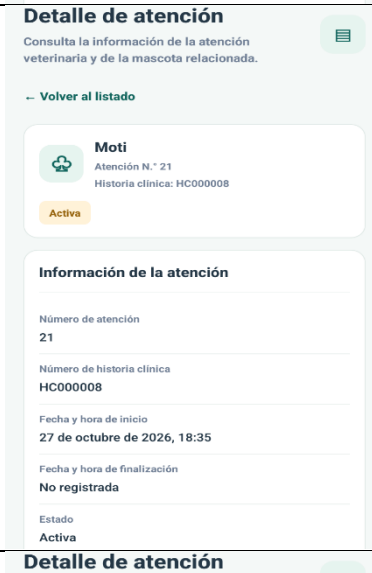
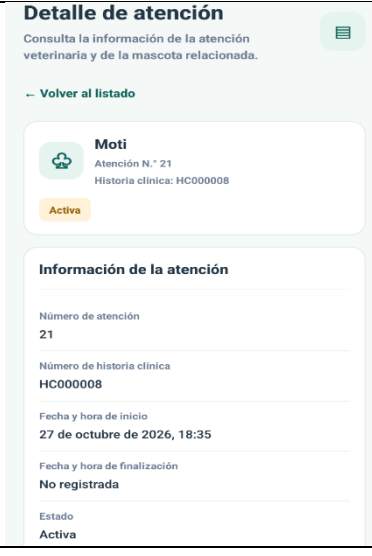
RF09	Visualizar en el panel principal las atenciones activas		Sí
RF10	Visualizar la cantidad de actividades pendientes, completadas y no realizadas		Sí
RF11	Mostrar avisos visuales cuando una actividad pendiente está próxima a vencer		Sí

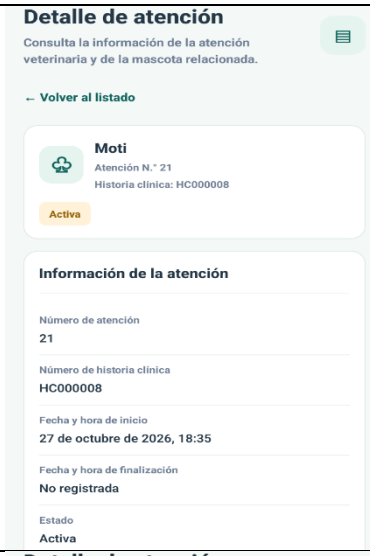
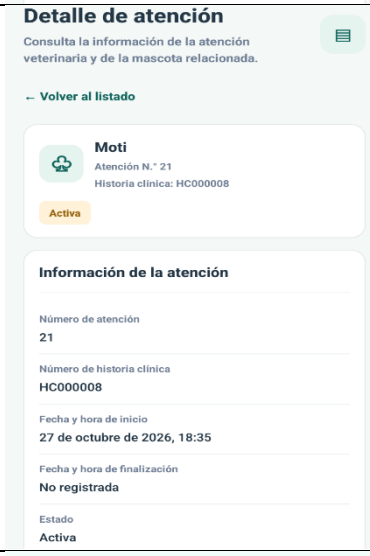
RF12	Registrar los datos de un propietario		Sí
RF13	Consultar la información de un propietario		Sí
RF14	Consultar las mascotas asociadas a un propietario		Sí

RF15	Editar los datos de un propietario		Sí
RF16	Buscar una mascota mediante la barra de búsqueda		Sí
RF17	Registrar una mascota y asociarla a un propietario		Sí

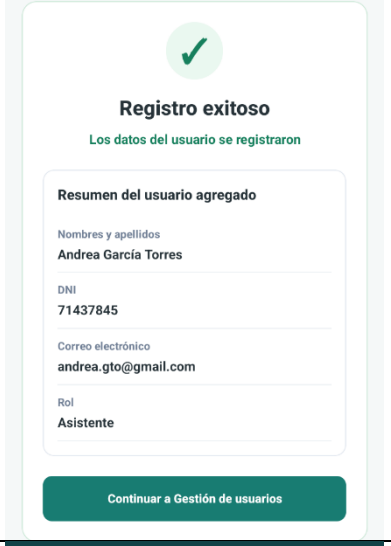
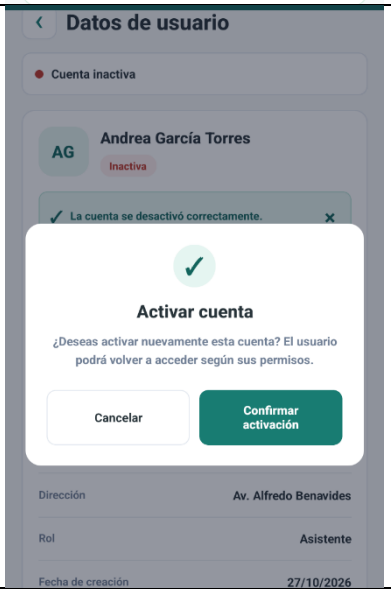
RF18	Modificar la información registrada de una mascota		Sí
RF19	Consultar la información de una mascota		Sí
RF20	Consultar el historial de atención de una mascota		Sí

RF21	Filtrar las atenciones según su estado activo o finalizado		Sí
RF22	Registrar una nueva atención veterinaria		Sí
RF23	Registrar automáticamente la fecha y hora de inicio de una atención		Sí

RF24	Consultar el detalle completo de una atención veterinaria		Sí
RF25	Identificar si una atención está activa o finalizada		Sí
RF26	Mostrar aviso de atención lista para finalizar		Sí

RF27	Finalizar una atención veterinaria mediante confirmación		Sí
RF28	Registrar automáticamente la fecha y hora de finalización		Sí
RF29	Registrar una nueva actividad dentro de una atención		Sí
RF30	Seleccionar el tipo de actividad y mostrar sus campos		Sí
RF31	Consultar las actividades registradas en una atención	(no hay actividad aun)	Sí

RF32	Modificar la información registrada de una actividad	(no hay actividad aun)	Sí
RF33	Eliminar una actividad registrada por error	(no hay actividad aun)	Sí
RF34	Asignar automáticamente el estado “Pendiente” a una nueva actividad	(no hay actividad aun)	Sí
RF35	Cambiar el estado de una actividad de “Pendiente” a “Completada”	(no hay actividad aun)	Sí
RF36	Marcar como “No realizada” una actividad pendiente con su motivo	(no hay actividad aun)	Sí
RF37	Reactivar una actividad “No realizada” devolviéndola a “Pendiente”	(no hay actividad aun)	Sí
RF38	Calcular el porcentaje de avance de una atención	(no hay actividad aun)	Sí
RF39	Mostrar una barra de progreso del avance de una atención	(no hay actividad aun)	Sí
RF40	Visualizar las actividades clasificadas según su estado	(no hay actividad aun)	Sí
RF41	Visualizar una línea de tiempo con los eventos de una atención	(no hay eventos)	Sí
RF42	Registrar automáticamente los cambios realizados en la información		Sí
RF43	Actualizar en tiempo real el seguimiento de una atención	(no hay seguimiento aun)	Sí

RF44	Crear y editar las cuentas de los usuarios (Administrador)		Sí
RF45	Activar o desactivar las cuentas de los usuarios		Sí
RF46	Gestionar el estado de las mascotas registradas (Administrador)	(no se puede modificar el estado)	Sí
RF47	Activar o desactivar el registro de un propietario (Administrador)	(se cierra)	Sí
RF48	Consultar un resumen general de la información de la veterinaria		Sí
RF49	Consultar indicadores históricos de las atenciones		Sí
RF50	Consultar los cambios realizados en la información registrada		Sí
RF51	Programar y cancelar citas de atención veterinaria		Sí

RF52	Gestionar el catálogo de medicamentos (Administrador)		Sí
RF53	Registrar la posología de los medicamentos en una actividad		Sí
RF54	Registrar la dieta indicada a una mascota dentro de una atención		Sí
RF55	Registrar los hallazgos de una ecografía por órgano evaluado		Sí
RF56	Gestionar el catálogo de tipos de actividad (Administrador)		Sí

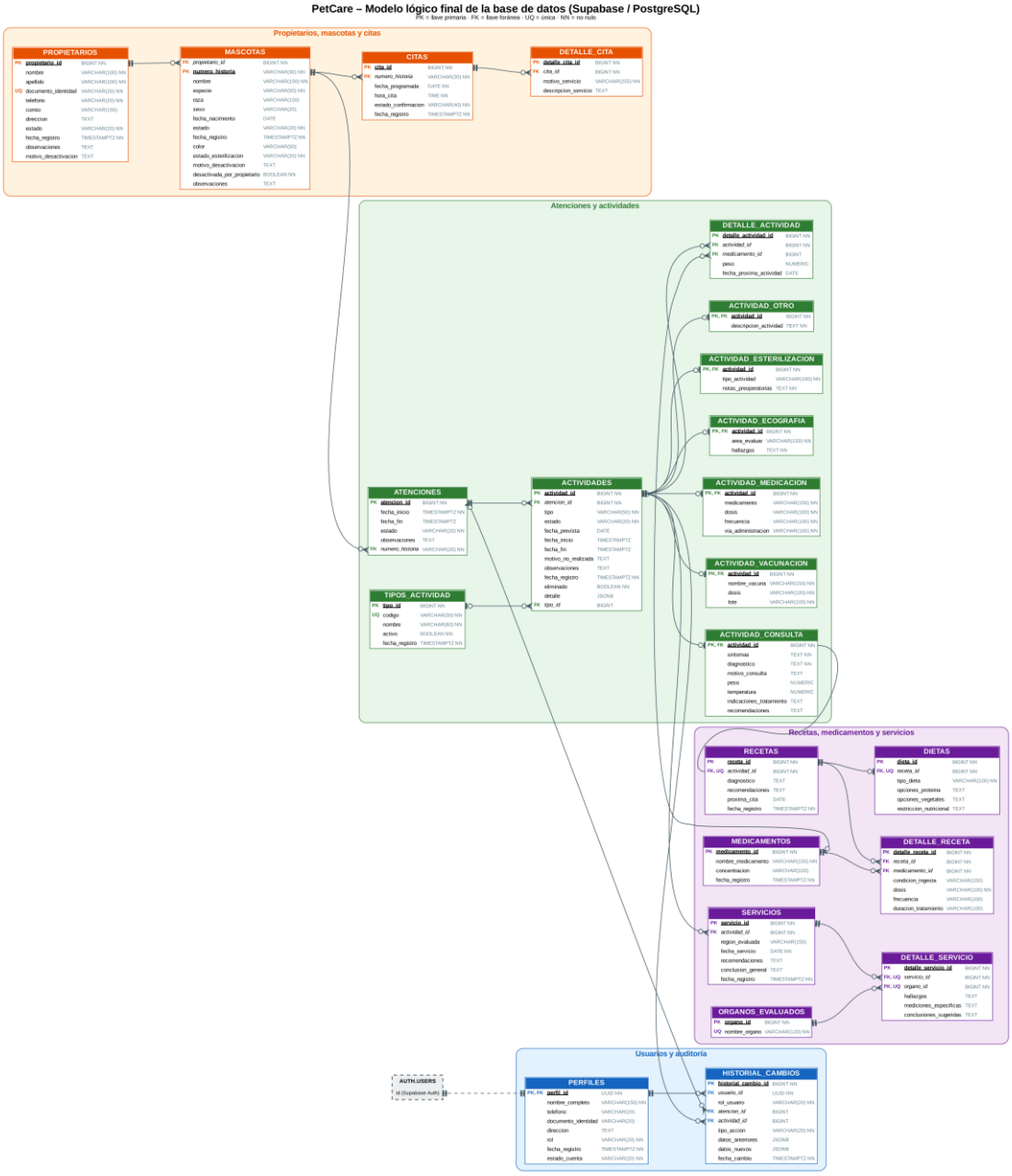
4.1. Capturas por requerimiento

Una captura de pantalla de cada pantalla funcional. Cada imagen debe tener un tamaño estándar de 5 × 7 cm dentro del documento y estar rotulada con el número y nombre del RF correspondiente. Se recomienda insertar las imágenes con esas dimensiones fijas para mantener uniformidad.

4.2. Modelo lógico final de BD

El modelo se obtuvo directamente del catálogo de la base de datos implementada en Supabase, por lo que refleja exactamente las tablas que usa el producto final. Está compuesto por 23 tablas organizadas en 4 módulos y 25 llaves foráneas. La normalización realizada en el APF2 (vistas Receta médica, Historia clínica, Seguimiento de citas, Lista de clientes y Reportes de servicio) se ve reflejada en las tablas de detalle (detalle_receta, detalle_cita, detalle_servicio, detalle_actividad), en los catálogos (medicamentos, organos_evaluados, tipos_actividad) y en la separación de propietarios y mascotas.

4.3.1. Diagrama del modelo lógico



4.3.2. Notación relacional

Convención: subrayado = llave primaria; **#** = llave foránea.

Usuarios y auditoría

PERFILES (**#perfil_id**, nombre_completo, telefono, documento_identidad, direccion, rol, fecha_registro, estado_cuenta)

HISTORIAL_CAMBIOS (**historial_cambio_id**, #usuario_id, rol_usuario, #atencion_id, #actividad_id, tipo_accion, datos_anteriores, datos_nuevos, fecha_cambio)

Propietarios, mascotas y citas

PROPIETARIOS (**propietario_id**, nombre, apellido, documento_identidad, telefono, correo, direccion, estado, fecha_registro, observaciones, motivo_desactivacion)

MASCOTAS (#propietario_id, **numero_historia**, nombre, especie, raza, sexo, fecha_nacimiento, estado, fecha_registro, color, estado_esterilizacion, motivo_desactivacion, desactivada_por_propietario, observaciones)

CITAS (**cita_id**, #numero_historia, fecha_programada, hora_cita, estado_confirmacion, fecha_registro)

DETALLE_CITA (**detalle_cita_id**, #cita_id, motivo_servicio, descripcion_servicio)

Atenciones y actividades

ATENCIONES (**atencion_id**, fecha_inicio, fecha_fin, estado, observaciones, #numero_historia)

TIPOS_ACTIVIDAD (**tipo_id**, codigo, nombre, activo, fecha_registro)

ACTIVIDADES (**actividad_id**, #atencion_id, tipo, estado, fecha_prevista, fecha_inicio, fecha_fin, motivo_no_realizada, observaciones, fecha_registro, eliminado, detalle, #tipo_id)

ACTIVIDAD_CONSULTA (**#actividad_id**, sintomas, diagnostico, motivo_consulta, peso, temperatura, indicaciones_tratamiento, recomendaciones)

ACTIVIDAD_VACUNACION (**#actividad_id**, nombre_vacuna, dosis, lote)

ACTIVIDAD_MEDICACION (**#actividad_id**, medicamento, dosis, frecuencia, via_administracion)

ACTIVIDAD_ECOGRAFIA (**#actividad_id**, area_evaluar, hallazgos)

ACTIVIDAD_ESTERILIZACION (**#actividad_id**, tipo_actividad, notas_preoperatorias)

ACTIVIDAD_OTRO (**#actividad_id**, descripcion_actividad)

DETALLE_ACTIVIDAD (**detalle_actividad_id**, #actividad_id, #medicamento_id, peso, fecha_proxima_actividad)

Recetas, medicamentos y servicios

MEDICAMENTOS (**medicamento_id**, nombre_medicamento, concentracion, fecha_registro)

RECETAS (**receta_id**, #actividad_id, diagnostico, recomendaciones, proxima_cita, fecha_registro)

DETALLE_RECETA (**detalle_receta_id**, #receta_id, #medicamento_id, condicion_ingesta, dosis, frecuencia, duracion_tratamiento)

DIETAS (**dieta_id**, #receta_id, tipo_dieta, opciones_proteina, opciones_vegetales, restriccion_nutricional)

SERVICIOS (**servicio_id**, #actividad_id, region_evaluada, fecha_servicio, recomendaciones, conclusion_general, fecha_registro)

DETALLE_SERVICIO (**detalle_servicio_id**, #servicio_id, #organo_id, hallazgos, mediciones_especificas, conclusiones_sugeridas)

ORGANOS_EVALUADOS (**organo_id**, nombre_organo)

4.3.3. Relaciones y cardinalidad

Tabla padre (PK)	Tabla hija (FK)	Relación	Cardinalidad	Tipo
actividad_consulta.actividad_id	recetas.actividad_id	emite	1 : 0..1	Uno a uno (FK única)
actividades.actividad_id	actividad_consulta.actividad_id	especializa	1 : 0..1	Identificadora (subtipo)
actividades.actividad_id	actividad_ecografia.actividad_id	especializa	1 : 0..1	Identificadora (subtipo)
actividades.actividad_id	actividad_esterilizacion.actividad_id	especializa	1 : 0..1	Identificadora (subtipo)
actividades.actividad_id	actividad_medicacion.actividad_id	especializa	1 : 0..1	Identificadora (subtipo)
actividades.actividad_id	actividad_otro.actividad_id	especializa	1 : 0..1	Identificadora (subtipo)
actividades.actividad_id	actividad_vacunacion.actividad_id	especializa	1 : 0..1	Identificadora (subtipo)
actividades.actividad_id	detalle_actividad.actividad_id	registra detalle de	1 : N	No identificadora
actividades.actividad_id	historial_cambios.actividad_id	registra cambios de	0..1 : N	Opcional (FK admite nulo)
actividades.actividad_id	servicios.actividad_id	genera	1 : N	No identificadora

Tabla padre (PK)	Tabla hija (FK)	Relación	Cardinalidad	Tipo
atenciones.atencion_id	actividades.atencion_id	incluye	1 : N	No identificadora
atenciones.atencion_id	historial_cambios.atencion_id	registra cambios de	0..1 : N	Opcional (FK admite nulo)
auth.users.id	perfiles.perfil_id	autentica	1 : 1	Uno a uno (FK única)
citas.cita_id	detalle_cita.cita_id	detalla	1 : N	No identificadora
mascotas.numero_historia	atenciones.numero_historia	recibe	1 : N	No identificadora
mascotas.numero_historia	citas.numero_historia	tiene programada	1 : N	No identificadora
medicamentos.medicamento_id	detalle_actividad.medicamento_id	se usa en	0..1 : N	Opcional (FK admite nulo)
medicamentos.medicamento_id	detalle_receta.medicamento_id	se prescribe en	1 : N	No identificadora
organos_evaluados.organo_id	detalle_servicio.organo_id	es evaluado en	1 : N	No identificadora
perfiles.perfil_id	historial_cambios.usuario_id	realiza	1 : N	No identificadora
propietarios.propietario_id	mascotas.propietario_id	es dueño de	1 : N	No identificadora

Tabla padre (PK)	Tabla hija (FK)	Relación	Cardinalidad	Tipo
recetas.receta_id	detalle_receta.receta_id	contiene	1 : N	No identificadora
recetas.receta_id	dietas.receta_id	indica	1 : 0..1	Uno a uno (FK única)
servicios.servicio_id	detalle_servicio.servicio_id	evalúa	1 : N	No identificadora
tipos_actividad.tipo_id	actividades.tipo_id	clasifica	0..1 : N	Opcional (FK admite nulo)

4.3.4. Reglas de integridad implementadas

- **Dominios (CHECK):** estado de atenciones (Activa/Finalizada), actividades (Pendiente/Completada/No realizada), mascotas y propietarios (Activo/Inactivo), perfiles (rol Administrador/Asistente y estado de cuenta), sexo (Macho/Hembra), estado de esterilización y tipo de acción del historial.
- **Unicidad (UQ):** documento de identidad del propietario, código de tipo de actividad, nombre de órgano, una receta por consulta, una dieta por receta y un mismo órgano por servicio (servicio_id, organo_id).
- **Integridad referencial:** las tablas de subtipo de actividad se eliminan en cascada con su actividad; el resto usa RESTRICT para no dejar registros huérfanos.
- **Valores por defecto:** fechas de registro con now(), estados iniciales (Pendiente, Activa, Activo) y número de historia clínica autogenerado (HC000001…).
- **Seguridad:** perfiles se vincula 1:1 con auth.users de Supabase Auth y las políticas RLS restringen el acceso a usuarios autenticados y activos según su rol.

5. Construcción del producto

Tabla de alcance implementado (RF comprometidos vs. implementados) + modelo lógico final de base de datos con PK y FK.